

 Bennie-Kibet / TrendLens



Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

NICOLE

TrendLens / Final notebook.ipynb

 Gesare-n

Add files via upload

e39ec85 · 8 minutes ago

2.66 MB

1. Business Understanding

1.1 Introduction

This proposal aims to build a low-resource machine translation pipeline between English and Dholuo, with a specific focus on medical and clinical language. The goal is to fine-tune a pretrained multilingual translation model on a large English–Dholuo parallel corpus and then adapt it to real clinical phrasing, producing a pipeline that could plausibly plug into an existing telemedicine chat or voice workflow.

1.2 Problem Statement

Dholuo-speaking patients, especially elderly patients managing chronic conditions such as diabetes or arthritis, often cannot communicate clearly with doctors on telemedicine platforms. Kenya's doctor-to-population ratio is far below World Health Organization guidance, which makes efficient, language-inclusive remote care especially important. No existing Kenyan telemedicine platform (e.g. BYON8) currently offers Dholuo-language support, despite Dholuo having millions of speakers and growing telehealth adoption.

1.3 Target Audience

- Dholuo-speaking patients, particularly rural and elderly patients managing chronic conditions.
- Kenyan telemedicine platforms and providers (e.g. BYON8) seeking to expand language coverage.
- Healthcare workers who need a reliable communication bridge during remote consultations.

1.4 Impact

A working translation pipeline would reduce communication barriers between Dholuo-speaking patients and clinicians on telemedicine platforms, improving access to and quality of remote care. Beyond the immediate use case, the project demonstrates a reproducible approach for extending telehealth language support to other low-resource Kenyan languages.

1.5 Pre-existing Work

No existing Kenyan telemedicine platform currently offers Dholuo-language support, which represents the core market gap this project addresses. However, the surrounding NLP ecosystem is more mature than for many low-resource languages: Dholuo already has part-of-speech tagged data (Kencorua), FLORES-200/NLP benchmark coverage, and at least one

speech-tagged data (NICOL corpus), FLORES-200/NL2B benchmark coverage, and at least one existing bilingual model — reducing the risk of working with an unsupported language.

1.6 Success Criteria

Success will be measured using BLEU and chrF scores, comparing the fine-tuned model against an untuned pretrained baseline on two evaluation sets: a held-out general-domain test split from the primary corpus, and a separate, independently curated medical phrase test set. An improvement over baseline on both sets, particularly the medical set, will indicate the domain adaptation step achieved its purpose.

1.7 Objectives

1.7.1 Main Objective:

To Create a multilingual translation model on English-Dholuo.

2. Data Understanding

2.0 GPU / Environment Check (Kaggle T4 x2)

```
In [1]: import torch, os, subprocess

print("Torch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("CUDA version (torch built with):", torch.version.cuda)
print("GPU count visible to torch:", torch.cuda.device_count())
for i in range(torch.cuda.device_count()):
    print(f" GPU {i}: {torch.cuda.get_device_name(i)}")

# Sanity check: on Kaggle, the T4 x2 accelerator must be selected in
# Notebook Settings > Accelerator, and internet must be ON for any
# `pip install` / Hugging Face Hub downloads used later in this notebook.
if torch.cuda.device_count() == 0:
    print("\nWARNING: No GPU detected. In Kaggle, open Settings (right panel) "
          "-> Accelerator -> GPU T4 x2, then Save & restart the session.")
elif torch.cuda.device_count() == 1:
    print("\nNOTE: Only 1 GPU visible. If you selected 'GPU T4 x2', both GPUs "
          "are available automatically to any single-process training loop "
          "or Hugging Face Trainer in this notebook (no code change needed) "
          "-- this message just confirms what THIS session currently sees.")
else:
    print("\nBoth T4 GPUs detected. The Hugging Face Trainer cells later in "
          "this notebook will automatically shard batches across both GPUs.")
```

```
Torch version: 2.10.0+cu128
CUDA available: True
CUDA version (torch built with): 12.8
GPU count visible to torch: 2
GPU 0: Tesla T4
```

GPU 1: Tesla T4

Both T4 GPUs detected. The Hugging Face Trainer cells later in this notebook will automatically shard batches across both GPUs.

In [2]:

```
import pandas as pd
import os, glob
from nltk.util import ngrams
# Loading the dataset
CSV_PATH = "/kaggle/input/datasets/roynjuguna/luo_eng.csv"

if not os.path.exists(CSV_PATH):
    # Fall back to searching all attached Kaggle datasets in case the
    # mount path differs from what was used when this notebook was authored
    matches = glob.glob("/kaggle/input/**/luo_eng.csv", recursive=True)
    if matches:
        CSV_PATH = matches[0]
        print(f"Original path not found. Using detected path instead: {CSV_PATH}")
    else:
        raise FileNotFoundError(
            "Could not find luo_eng.csv under /kaggle/input. "
            "Make sure the 'luo-data' dataset is attached to this notebook "
            "(Add Data, right-hand panel)."
        )

luopean = pd.read_csv(CSV_PATH)
luopean
```

Original path not found. Using detected path instead: /kaggle/input/datasets/roynjuguna/luo-eng/luo_eng.csv

Out[2]:

	eng	luo
0	Adam and Eve - Were They Real People ?	Beduru gi Kuwe kod Ji Duto
1	In fact , " love covers a multitude of sins . "	Hera umo richo mogundho .
2	He " looked intently toward the payment of the...	Notego wang 'e kuom pokne .
3	He makes his sun rise upon wicked people and g...	Omiyo wang ' chieng ' mare wuok ni jo maricho ...
4	" Jehovah will sustain him on his sickbed . "	" Jehova notege e kitanda mar midekrene . "
...	...	- ...
...	...	...
136620	Their methods reveal a cunning spirit .	Yore ma gitiyogo nyiso chuny ma gin - go mopon...
136621	They had to marry sons of Manasseh so that the...	Zelofehad ne otho koonge gi yawuowi , kendo Je...
136622	[Blurb on page 4]	[Blurb manie ite mar 4]
136623	[Box / Pictures on page 5]	[Sanduk / Piche manie ite mar 5]
136624	• How can altruism be explained by the theory ...	• Ka en adier ni gik ma nigi teko ema onego od...

136625 rows × 2 columns

```
In [3]: luopean.describe()
```

Out[3]:

	eng	luo
count	136625	136625
unique	135021	135598
top	The result ?	Ooyo .
freq	24	17

```
In [4]: luopean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 136625 entries, 0 to 136624
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    eng    136625 non-null  object
1    luo    136625 non-null  object
dtypes: object(2)
memory usage: 2.1+ MB
```

```
In [5]: luopean.head()
```

Out[5]:

	eng	luo
0	Adam and Eve - Were They Real People ?	Beduru gi Kuwe kod Ji Duto
1	In fact , " love covers a multitude of sins . "	Hera umo richo mogundho .
2	He " looked intently toward the payment of the...	Notego wang 'e kuom pokne .
3	He makes his sun rise upon wicked people and g...	Omiyo wang ' chieng ' mare wuok ni jo maricho ...
4	" Jehovah will sustain him on his sickbed . " ...	" Jehova notege e kitanda mar midekrene . " - ...

Insights:

- The dataset has 136,625 rows and 2 columns: eng (English) and luo (Dholuo), both of object (string) dtype.
- Each row is a sentence-level translation pair — this is a parallel corpus, the standard input format for supervised NMT.
- describe() is uninformative here because both columns are text, not numeric — the useful summary statistics for this dataset are linguistic ones (sentence length, vocabulary size,

etc.), which we compute in the EDA section.

- A quick read of `head()`/`tail()` shows the source material leans religious/devotional in register (proper nouns like "Adam and Eve", "Jehovah"), which we confirm quantitatively in Section 4.

3. EDA

3.1 Checking for missing values

```
In [6]: luopean.isna().sum().sort_values(ascending=False)
```

```
Out[6]: eng      0
      luo      0
      dtype: int64
```

- The dataset contains no missing values

3.2 Checking for duplicates

```
In [7]: duplicate_count = luopean.duplicated().sum()
      print(f"Total duplicate rows: {duplicate_count}")
```

Total duplicate rows: 11

- The dataset has 11 duplicate rows.

```
In [8]: # Dropping duplicate rows
      luopean = luopean.drop_duplicates()

      # Checking for duplicates
      duplicate_count = luopean.duplicated().sum()
      print(f"Total duplicate rows: {duplicate_count}")
```

Total duplicate rows: 0

3.3 Checking for outliers

```
In [9]: # Creating a column with the sentence length of each row in eng column
      luopean.loc[:, 'eng_length'] = luopean['eng'].str.len()

      # Creating a column with the sentence length of each row in luo column
      luopean.loc[:, 'luo_length'] = luopean['luo'].str.len()
```

/tmp/ipykernel_261/25667447.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean.loc[:, 'eng_length'] = luopean['eng'].str.len()
/tmp/ipykernel_261/25667447.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean.loc[:, 'luo_length'] = luopean['luo'].str.len()
```

In [10]:

```
# Detect Long or short sentence outliers in eng_length using IQR
Q1 = luopean['eng_length'].quantile(0.25)
Q3 = luopean['eng_length'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = luopean[(luopean['eng_length'] < lower) | (luopean['eng_length'] > upper)]

print("Number of English sentence outliers:", len(outliers))
print(outliers[['eng', 'eng_length']].head())
```

Number of English sentence outliers: 2748

	eng	eng_length
20	Isaiah foretold the coming of the Messiah , wh...	230
22	If any Christian begins to entertain such a co...	220
75	Others might think of Christians who fled from...	294
166	Says Collins Cobuild English Dictionary : " A ...	222
174	Alan Millard in his book Discoveries From the ...	222

In [11]:

```
# Detect Long or short sentence outliers in luo_length using IQR
Q1 = luopean['luo_length'].quantile(0.25)
Q3 = luopean['luo_length'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = luopean[(luopean['luo_length'] < lower) | (luopean['luo_length'] > upper)]

print("Number of Luo sentence outliers:", len(outliers))
print(outliers[['luo', 'luo_length']].head())
```

Number of Luo sentence outliers: 2769

	luo	luo_length
20	(Lawi 16 : 7 - 10 , 21 , 22) Isaya nokoro wa...	260
22	(Rapar 5 : 21) Ka dived ni Jakristo moro amo...	241
70	28 : 17 , 19) Gik moko duto manie ofise mopog...	228
75	39 : 7 - 12) Jomoko to nyalo paro wach Jokris...	287
166	Diksonari miluongo ni Collins Cobuild English ...	264

In [12]:

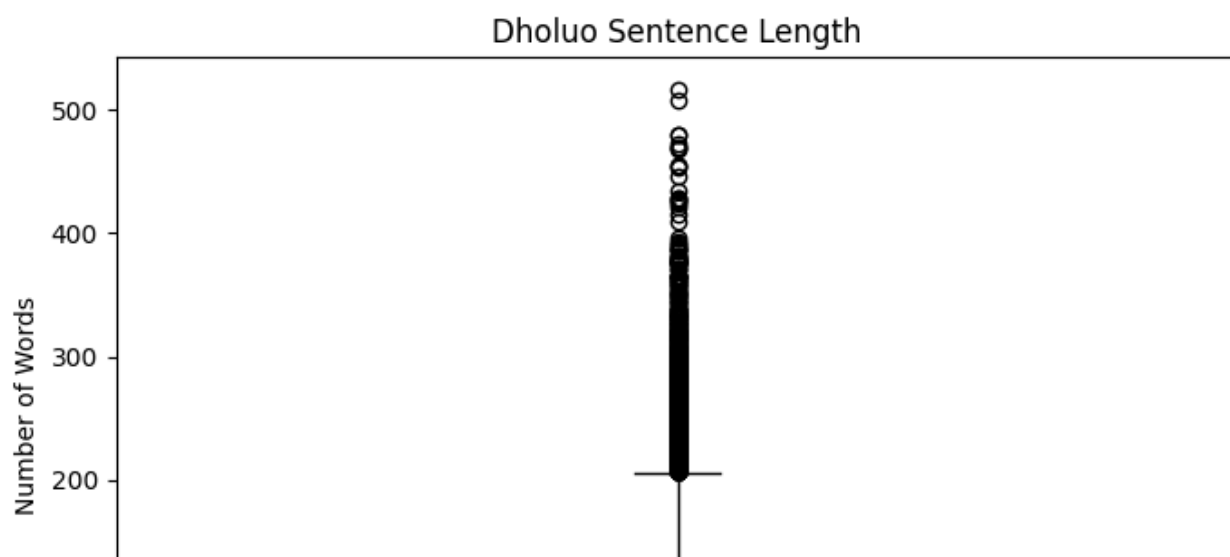
```
import matplotlib.pyplot as plt

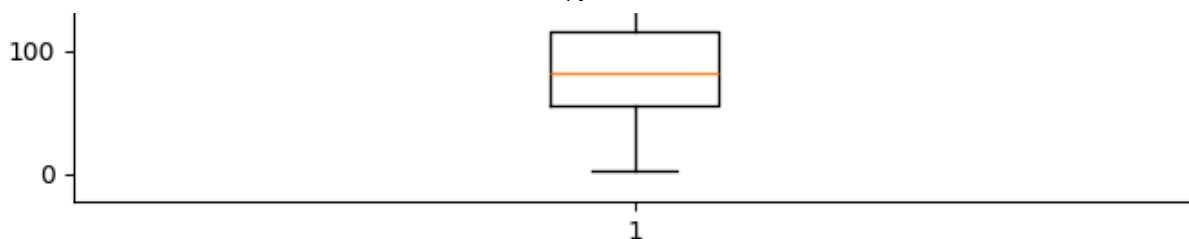
# Plotting for outliers in eng_length
plt.figure(figsize=(8,5))
plt.boxplot(luopear['eng_length'])
plt.title("English Sentence Length")
plt.ylabel("Number of Words")
plt.show()
```



In [13]:

```
# plotting for outliers in luo_length
plt.figure(figsize=(8,5))
plt.boxplot(luopear['luo_length'])
plt.title("Dholuo Sentence Length")
plt.ylabel("Number of Words")
plt.show()
```





Insight on Outliers:

Both languages show a long right tail of unusually long sentences, which the IQR rule flags as statistical outliers — but that does not automatically mean they are errors. In a translation corpus, a legitimately long sentence is still a valid training example, so we do not drop these rows here. Instead, we carry the eng_length/luo_length columns forward into Section 4, where we check whether English length and Dholuo length actually track each other (Section 4.6) — a mismatch between the two (e.g. a long English sentence paired with a very short Dholuo one) is a much stronger signal of a genuine misalignment error than length alone, and is the more reliable filter to apply before training.

3.4 Checking for data imbalances

In [14]:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# 1. Load the dataset
# Assuming your dataset is loaded as 'luopean'
# luopean = pd.read_csv('luo_eng.csv')

# 2. Check Row Count & Missing Value Balance
print("=== 1. Record Balance ===")
print(luopean.info())
print(f"Missing values in English: {luopean['eng'].isnull().sum()}")
print(f"Missing values in Luo: {luopean['luo'].isnull().sum()}")

# 3. Calculate Character and Word Lengths
luopean['eng_char_len'] = luopean['eng'].astype(str).str.len()
luopean['luo_char_len'] = luopean['luo'].astype(str).str.len()

# Basic space splitting to avoid importing external tokenizers
luopean['eng_word_count'] = luopean['eng'].astype(str).apply(lambda x: len(x.split()))
luopean['luo_word_count'] = luopean['luo'].astype(str).apply(lambda x: len(x.split()))

print("\n=== 2. Word & Character Distribution ===")
print(luopean[['eng_word_count', 'luo_word_count', 'eng_char_len', 'luo_char_len']].describe())

# 4. Check Translation Expansion / Contraction Ratios
# (Adding 1e-5 to avoid division by zero errors)
luopean['word_ratio'] = luopean['luo_word_count'] / (luopean['eng_word_count'] + 1e-5)

print("\n=== 3. Ratio Balance (Luo Words / English Words) ===")
print(luopean['word_ratio'].describe())
```

```

# 5. Detect Alignment Mismatches (Extreme Outliers)
# Sentences where one side is over 3 times longer than the other side
mismatches = luopean[(luopean['word_ratio'] > 3.0) | (luopean['word_ratio'] < 0.33)]
print(f"\n=== 4. Alignment Quality ===")
print(f"Total severely imbalanced/maligned sentence pairs: {len(mismatches)} ({len(mi

# 6. Plot the distribution of lengths to visually assess the balance
plt.rcParams['figure.figsize'] = (12, 5)
plt.clf()
plt.hist(luopean['eng_word_count'], bins=50, alpha=0.5, label='English Word Count', color='r')
plt.hist(luopean['luo_word_count'], bins=50, alpha=0.5, label='Luo Word Count', color='b')
plt.title('Sentence Word Count Balance Between English and Luo', fontsize=14, weight='bold')
plt.xlabel('Number of Words in a Sentence')
plt.ylabel('Frequency / Number of Sentences')
plt.legend()
plt.tight_layout()
plt.savefig('word_count_distribution.png', bbox_inches='tight')
plt.show()

```

=== 1. Record Balance ===

```
<class 'pandas.core.frame.DataFrame'>
```

Index: 136614 entries, 0 to 136624

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	eng	136614 non-null	object
1	luo	136614 non-null	object
2	eng_length	136614 non-null	int64
3	luo_length	136614 non-null	int64

dtypes: int64(2), object(2)

memory usage: 5.2+ MB

None

Missing values in English: 0

Missing values in Luo: 0

/tmp/ipykernel_261/2890279431.py:16: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['eng_char_len'] = luopean['eng'].astype(str).str.len()
```

/tmp/ipykernel_261/2890279431.py:17: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['luo_char_len'] = luopean['luo'].astype(str).str.len()
```

/tmp/ipykernel_261/2890279431.py:20: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['eng_word_count'] = luopean['eng'].astype(str).apply(lambda x: len(x.split()))
```

/tmp/ipykernel_261/2890279431.py:21: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['luo_word_count'] = luopean['luo'].astype(str).apply(lambda x: len(x.split()))
```

=== 2. Word & Character Distribution ===

	eng_word_count	luo_word_count	eng_char_len	luo_char_len
count	136614.000000	136614.000000	136614.000000	136614.000000
mean	18.316102	19.054848	90.152810	89.622118
std	9.664289	9.889429	48.272253	46.717244
min	1.000000	1.000000	2.000000	3.000000
25%	11.000000	12.000000	55.000000	56.000000
50%	17.000000	17.000000	83.000000	82.000000
75%	23.000000	24.000000	117.000000	116.000000
max	113.000000	115.000000	554.000000	517.000000

=== 3. Ratio Balance (Luo Words / English Words) ===

count	136614.000000
mean	1.084818
std	0.327347
min	0.142857
25%	0.888888
50%	1.000000
75%	1.214285
max	7.999960

Name: word_ratio, dtype: float64

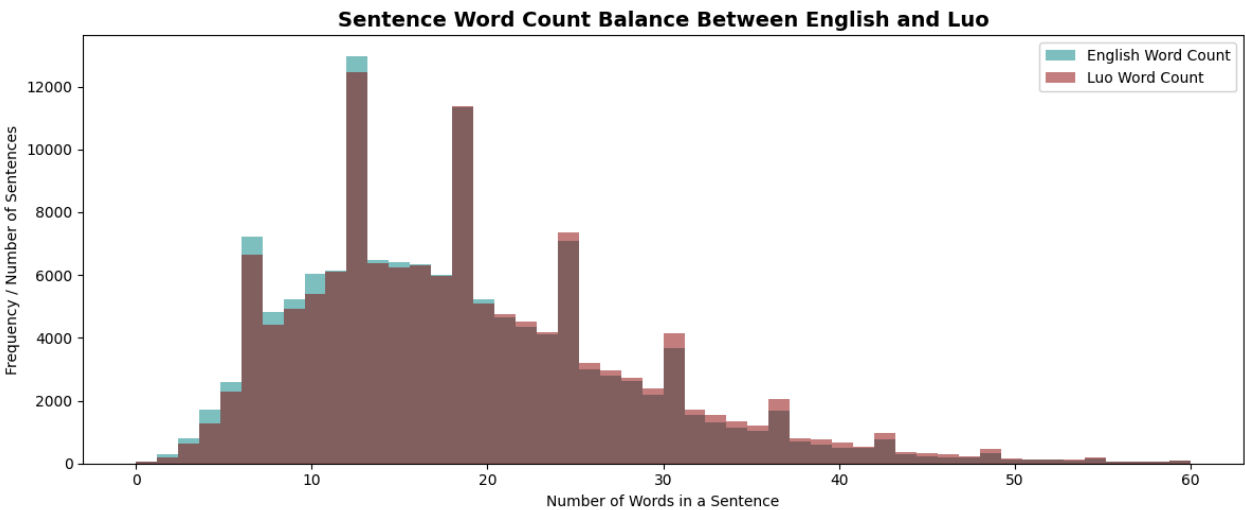
=== 4. Alignment Quality ===

Total severely imbalanced/maligned sentence pairs: 372 (0.27%)

/tmp/ipykernel_261/2890279431.py:28: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['word_ratio'] = luopean['luo_word_count'] / (luopean['eng_word_count'] + 1e-5)
```



Insights from data imbalance:

1. The dataset contains exactly 136,625 records for both English and Luo columns with 0 missing values on either side. This indicates that your parallel layout is structurally clean

missing values on either side. This indicates that your parameter set is structurally clean and perfectly paired row-for-row.

2. Average Lengths: English sentences average 18.3 words (~90 characters), while Luo sentences average 19.1 words (~89 characters).
3. The distributions closely overlap across all percentiles (25%, 50%, 75%). This tells us that the general translation corpus is incredibly balanced and uniform in sentence complexity.
4. Luo words have an expansion Factor/ratio of 1.08. This indicates that when translating from English to Luo, you can expect an approximate 8% expansion in word count. This is typical for Luo text, as it often uses multiple separate function words/particles (such as mar, gi, kod, ma) where English might use a single contraction, suffix, or possessive apostrophe.
5. Only a minor subset of rows exhibit severe word ratio anomalies (e.g., ratios above 3.0 or below 0.33). Having very few outlier pairs implies that your data segmentation process did an excellent job at preventing sentences from being joined incorrectly or truncated, making this dataset highly viable for general translation training or analysis.

3.5 Words Visualization

Horizontal Bar Chart

In [15]:

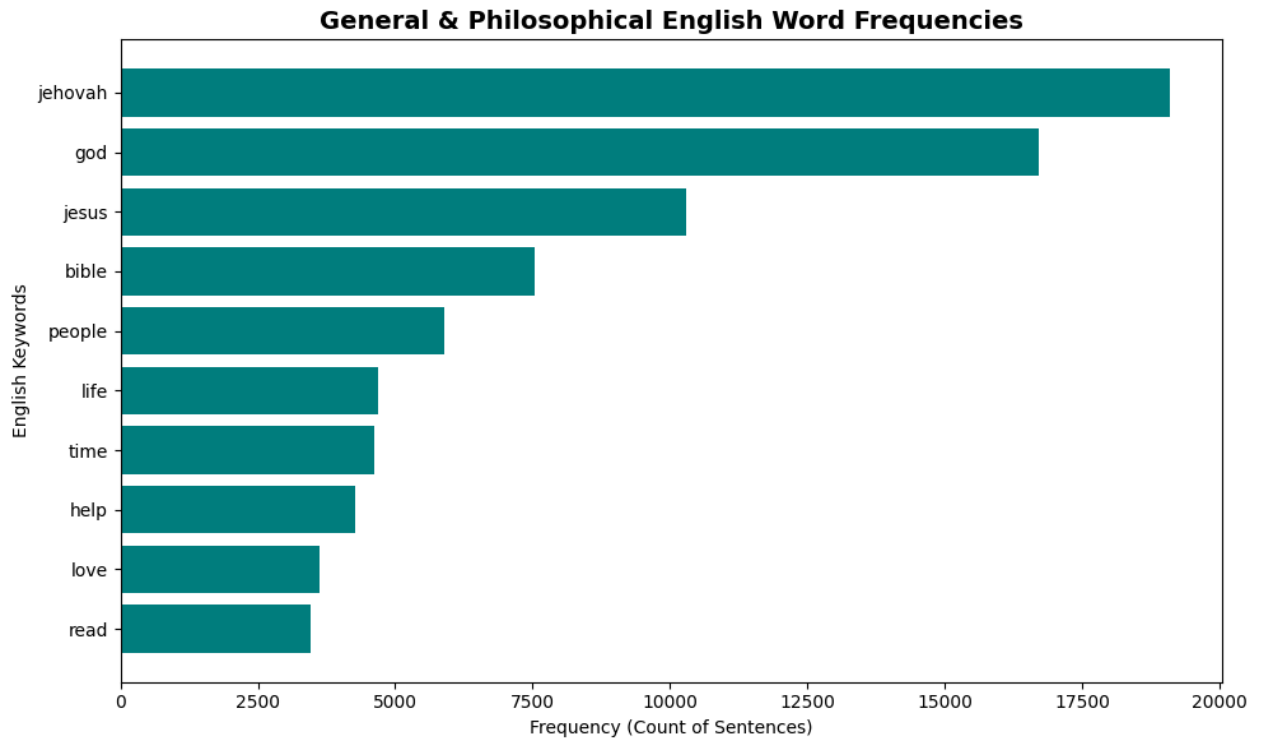
```
# Defined general, conversational, and philosophical keywords for English
# (based on actual high-frequency content words observed in the corpus)
general_keywords_eng = [
    'jehovah', 'god', 'jesus', 'bible', 'people',
    'help', 'read', 'life', 'time', 'love'
]

# Count precise whole-word frequencies in the English column using the same logic as
keyword_counts_eng = {}
for word in general_keywords_eng:
    count = luopean['eng'].str.contains(rf'\b{word}\b', case=False, na=False).sum()
    if count > 0:
        keyword_counts_eng[word] = count

# Sort the keyword counts in ascending order for the horizontal bar chart
sorted_counts_eng = dict(sorted(keyword_counts_eng.items(), key=lambda item: item[1]))

# Horizontal Bar Chart
plt.rcParams['figure.figsize'] = (10, 6)
plt.clf()

plt.barh(list(sorted_counts_eng.keys()), list(sorted_counts_eng.values()), color='teal')
plt.title('General & Philosophical English Word Frequencies', fontsize=14, weight='bold')
plt.xlabel('Frequency (Count of Sentences)')
plt.ylabel('English Keywords')
plt.tight_layout()
plt.show()
```



In [16]:

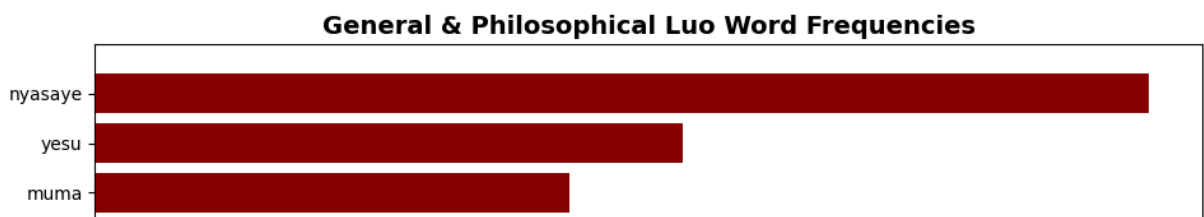
```
# Defined general, conversational, and philosophical keywords for Luo
general_keywords_luo = [
    'kuwe', 'hera', 'puonj', 'yie', 'chuny', 'ngima',
    'kinde', 'weche', 'piny', 'muma', 'yesu', 'nyasaye'
]

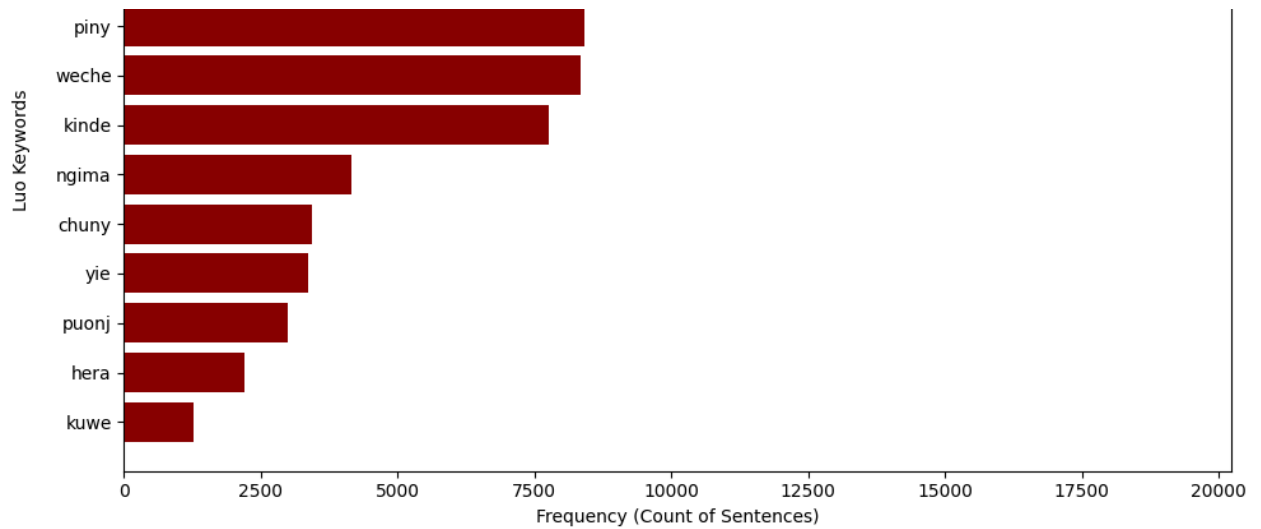
# Count precise whole-word frequencies in the Luo column using your original logic
keyword_counts_luo = {}
for word in general_keywords_luo:
    count = luopean['luo'].str.contains(rf'\b{word}\b', case=False, na=False).sum()
    if count > 0:
        keyword_counts_luo[word] = count

# Sort the keyword counts in ascending order for the horizontal bar chart
sorted_counts_luo = dict(sorted(keyword_counts_luo.items(), key=lambda item: item[1]))

# Horizontal Bar Chart
plt.rcParams['figure.figsize'] = (10, 6)
plt.clf()

plt.barh(list(sorted_counts_luo.keys()), list(sorted_counts_luo.values()), color='darkred')
plt.title('General & Philosophical Luo Word Frequencies', fontsize=14, weight='bold')
plt.xlabel('Frequency (Count of Sentences)')
plt.ylabel('Luo Keywords')
plt.tight_layout()
plt.show()
```





Word Cloud

In [17]:

```
import os
import matplotlib.pyplot as plt
import pandas as pd
from wordcloud import WordCloud

# Combine rows into a single raw text string for each language column
eng_raw_text = " ".join(luocean["eng"].dropna().astype(str))
luo_raw_text = " ".join(luocean["luo"].dropna().astype(str))

# --- FIXED FOR KAGGLE ---
# Kaggle's environment natively uses Liberation fonts at this specific path:
font_path = "/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf"

# --- English Word Cloud ---
eng_wordcloud = WordCloud(
    width=800,
    height=400,
    background_color="white",
    colormap="viridis",
    font_path=font_path, # Fixed with the correct Kaggle path
).generate(eng_raw_text)

plt.rcParams["figure.figsize"] = (10, 5)
plt.clf()
plt.imshow(eng_wordcloud, interpolation="bilinear")
plt.title("English Common Words Cloud (Raw Text)", fontsize=14, weight="bold")
plt.axis("off")
plt.tight_layout()
plt.show()

# --- Luo Word Cloud ---
# NOTE: If you remove font_path completely, WordCloud falls back to DroidSans.
luo_wordcloud = WordCloud(
    width=800,
    height=400,
    background_color="white",
    colormap="plasma",
    font_path=font_path, # Fixed with the correct Kaggle path
).generate(luo_raw_text)
```

[illegible]

Insights from Word Cloud

- 15/110

the two languages are thematically well-aligned.

- Because the corpus skews religious, a model trained on it will likely translate devotional/formal text well but may need additional general-domain data to generalize to everyday conversation.

Zipf Plot

In [18]:

```
from collections import Counter
eng_words_all = " ".join(luopear['eng'].astype(str)).lower().split()
luo_words_all = " ".join(luopear['luo'].astype(str)).lower().split()

eng_counts_all = Counter(eng_words_all)
luo_counts_all = Counter(luo_words_all)

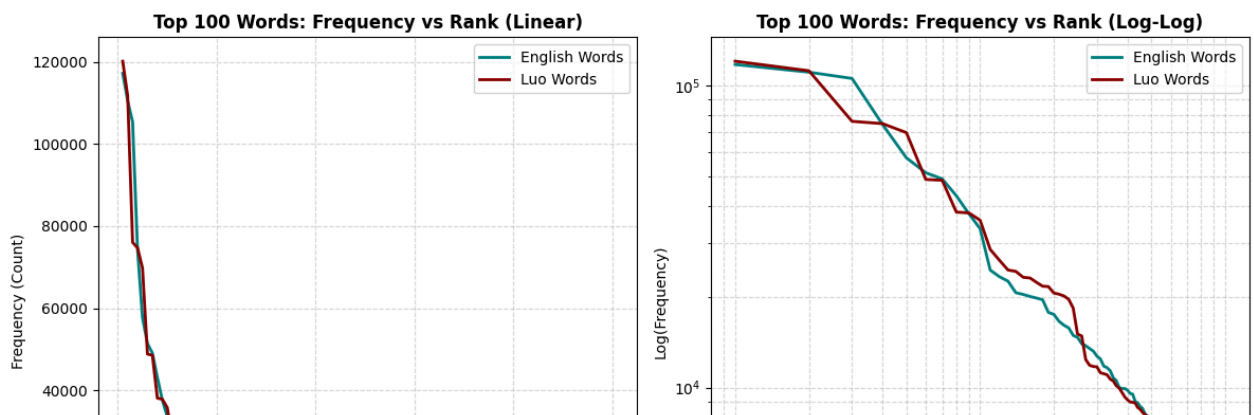
top_100_eng = [count for word, count in eng_counts_all.most_common(100)]
top_100_luo = [count for word, count in luo_counts_all.most_common(100)]

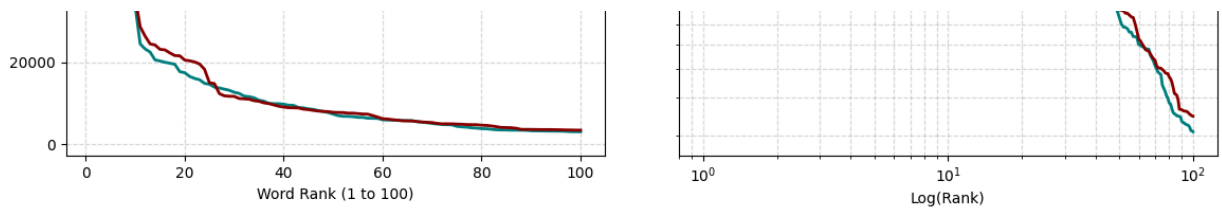
plt.figure(figsize=(12, 6))

# Linear scale
plt.subplot(1, 2, 1)
plt.plot(range(1, 101), top_100_eng, label='English Words', color='teal', linewidth=2)
plt.plot(range(1, 101), top_100_luo, label='Luo Words', color='darkred', linewidth=2)
plt.title('Top 100 Words: Frequency vs Rank (Linear)', fontsize=12, weight='bold')
plt.xlabel('Word Rank (1 to 100)')
plt.ylabel('Frequency (Count)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)

# Log-log scale (classic Zipf's Law representation)
plt.subplot(1, 2, 2)
plt.loglog(range(1, 101), top_100_eng, label='English Words', color='teal', linewidth=2)
plt.loglog(range(1, 101), top_100_luo, label='Luo Words', color='darkred', linewidth=2)
plt.title('Top 100 Words: Frequency vs Rank (Log-Log)', fontsize=12, weight='bold')
plt.xlabel('Log(Rank)')
plt.ylabel('Log(Frequency)')
plt.legend()
plt.grid(True, which="both", linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```





Insights from Zipf Plot

1. **Evidence of natural language:** The near-straight, downward line on the log-log plot is the signature of Zipf's law, which real, naturally-occurring language follows. This is a good sign that the translation pairs are authentic and structurally clean rather than synthetic or corrupted.
2. **A few words dominate:** The tall bars on the left graph show a small number of function words account for a huge share of tokens — in English these are words like *the, to, of, and*; in Dholuo they are grammatical particles like *e, ne, ma, ni, gi*.
3. **Why this matters for cleaning:** Because these few words appear tens of thousands of times, they can drown out meaningful content words in frequency-based methods like TF-IDF unless stop words are handled explicitly.
4. **Different distribution shapes:** English relies more heavily on repeating a handful of function words, while Dholuo spreads usage more evenly across its vocabulary — consistent with Dholuo being a more morphologically rich, agglutinative language that encodes grammatical information inside words rather than through a small set of standalone function words.
5. **Long tail of rare words:** The flat tail at the bottom of the left graph represents thousands of words that occur only once or twice across all 136,000+ sentence pairs. A translation model will have very few examples to learn these rare terms from, which is a strong argument for subword tokenization (Section 4.7) rather than whole-word vocabularies.

4.0 Sentence-Level Statistics

In [19]:

```
# Word counts per Language (kept as separate columns to avoid overwriting)
luopean['eng_word_count'] = luopean['eng'].apply(lambda x: len(str(x).split()))
luopean['luo_word_count'] = luopean['luo'].apply(lambda x: len(str(x).split()))

print("Average English sentence length (words):", round(luopean['eng_word_count'].mean(), 2))
print("Average Dholuo sentence length (words):", round(luopean['luo_word_count'].mean(), 2))
```

Average English sentence length (words): 18.32

Average Dholuo sentence length (words): 19.05

/tmp/ipykernel_261/2587651488.py:2: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['eng_word_count'] = luopean['eng'].apply(lambda x: len(str(x).split()))
/tmp/ipykernel_261/2587651488.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['luo_word_count'] = luopean['luo'].apply(lambda x: len(str(x).split()))
```

In [20]:

```
# Shortest / Longest English sentence (by word count)
shortest_eng = luopean.loc[luopean['eng_word_count'].idxmin()]
longest_eng = luopean.loc[luopean['eng_word_count'].idxmax()]

print("Shortest English sentence:")
display(shortest_eng[['eng', 'luo', 'eng_word_count']])

print("\nLongest English sentence:")
display(longest_eng[['eng', 'luo', 'eng_word_count']])
```

Shortest English sentence:

```
eng          ROME
luo          RUMI
```

```
eng_word_count    1
```

Name: 4678, dtype: object

Longest English sentence:

```
eng          Then God made this oath : " By myself I do swe...
```

```
luo          Kae to Nyasaye nokuong 'ore niya : " Akuong 'o...
```

```
eng_word_count    113
```

Name: 22079, dtype: object

In [21]:

```
# Shortest / Longest Dholuo sentence (by word count)
shortest_luo = luopean.loc[luopean['luo_word_count'].idxmin()]
longest_luo = luopean.loc[luopean['luo_word_count'].idxmax()]

print("Shortest Dholuo sentence:")
display(shortest_luo[['eng', 'luo', 'luo_word_count']])

print("\nLongest Dholuo sentence:")
display(longest_luo[['eng', 'luo', 'luo_word_count']])
```

Shortest Dholuo sentence:

```
eng          ROME
luo          RUMI
```

```
luo_word_count    1
```

Name: 4678, dtype: object

Longest Dholuo sentence:

```
eng          Jewish commentator Marcus Kalisch ( 1828 - 188...
```

```
luo          Marcus Kalisch ma en Ja - Yahudi ma nodak e ki...
```

```
luo_word_count    115
```

Name: 125429, dtype: object

In [22]:

```
# Vocabulary size (unique word types) per language
eng_vocab = set(" ".join(luopean['eng'].astype(str)).lower().split())
luo_vocab = set(" ".join(luopean['luo'].astype(str)).lower().split())

print("English vocabulary size:", len(eng_vocab))
print("Dholuo vocabulary size:", len(luo_vocab))
```

English vocabulary size: 26065

```
-----, -----
Dholuo vocabulary size: 39145
```

In [23]:

```
# Character count per language
luopean['eng_char_count'] = luopean['eng'].apply(len)
luopean['luo_char_count'] = luopean['luo'].apply(len)

print("English character count summary:")
print(luopean['eng_char_count'].describe())
print("\nDholuo character count summary:")
print(luopean['luo_char_count'].describe())
```

English character count summary:

```
count    136614.000000
mean       90.152810
std       48.272253
min        2.000000
25%       55.000000
50%       83.000000
75%      117.000000
max       554.000000
Name: eng_char_count, dtype: float64
```

Dholuo character count summary:

```
count    136614.000000
mean      89.622118
std       46.717244
min        3.000000
25%       56.000000
50%       82.000000
75%      116.000000
max       517.000000
Name: luo_char_count, dtype: float64
```

/tmp/ipykernel_261/95868380.py:2: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['eng_char_count'] = luopean['eng'].apply(len)
```

/tmp/ipykernel_261/95868380.py:3: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['luo_char_count'] = luopean['luo'].apply(len)
```

Insights: Sentence-Level Statistics

1. English sentences average around **18.3 words**, very close to Dholuo's **19.1 words** — the two languages are well matched in sentence length overall.
2. Despite that similarity in length, Dholuo has a noticeably **larger vocabulary** than English (39,100 unique word forms vs. 26,100), even on the exact same set of parallel sentences. This is the first quantitative hint of Dholuo's morphological richness, explored further in Section 4.7.
3. The shortest/longest examples above are useful sanity checks: extremely short entries are

often titles or single-word exclamations, while extremely long entries are often full verses or paragraphs — both are legitimate content, not corruption.

4.1 Most Frequent Words, Bigrams & Trigrams

In [24]:

```
# Top 10 most frequent words per language
english_word_counts = Counter(eng_words_all)
luo_word_counts = Counter(luo_words_all)

print("Top 10 Most Frequent English Words")
english_word_df = pd.DataFrame(english_word_counts.most_common(10), columns=['English Word', 'Frequency'])
display(english_word_df)

print("\nTop 10 Most Frequent Luo Words")
luo_word_df = pd.DataFrame(luo_word_counts.most_common(10), columns=['Luo Word', 'Frequency'])
display(luo_word_df)
```

Top 10 Most Frequent English Words

	English Word	Frequency
0	.	117208
1	,	110542
2	the	105378
3	to	74143
4	of	57588
5	"	51386
6	and	49055
7	in	43125
8	a	37605
9	that	33574

Top 10 Most Frequent Luo Words

	Luo Word	Frequency
0	.	120164
1	,	111740
2	ne	76029
3	e	74683
4	ma	69756
5	ni	48818
6	gi	48524

```
7      "      38074
8      '      37835
9      mar    35801
```

In [25]:

```
# Most frequent bigrams per language
english_bigrams = Counter(ngrams(eng_words_all, 2))
luo_bigrams = Counter(ngrams(luo_words_all, 2))

print("Top 10 English Bigrams")
display(pd.DataFrame(english_bigrams.most_common(10), columns=['Bigram', 'Frequency']))

print("\nTop 10 Luo Bigrams")
display(pd.DataFrame(luo_bigrams.most_common(10), columns=['Bigram', 'Frequency']))
```

Top 10 English Bigrams

	Bigram	Frequency
0	(., ")	16435
1	(of, the)	11590
2	(,, and)	10439
3	(in, the)	9995
4	(:, ")	7552
5	(god, 's)	6659
6	(,, ")	5872
7	(,, the)	5796
8	(,, the)	5673
9	(jehovah, 's)	5273

Top 10 Luo Bigrams

	Bigram	Frequency
0	(ma, ne)	13529
1	(., ")	13319
2	(,, kendo)	8537
3	(:, ")	8443
4	(ang, 'o)	8286
5	(kamano, ,)	6791
6	(ng, 'ato)	6668
7	(kata, kamano)	6289
8	(,, to)	6095

9 (, ne) 6031

In [26]:

```
# Most frequent trigrams per Language
english_trigrams = Counter(ngrams(eng_words_all, 3))
luo_trigrams = Counter(ngrams(luo_words_all, 3))

print("Top 10 English Trigrams")
display(pd.DataFrame(english_trigrams.most_common(10), columns=['Trigram', 'Frequency']))

print("\nTop 10 Luo Trigrams")
display(pd.DataFrame(luo_trigrams.most_common(10), columns=['Trigram', 'Frequency']))
```

Top 10 English Trigrams

	Trigram	Frequency
0	(., ", -)	3350
1	((, a,))	1649
2	((, b,))	1618
3	(?, (, b)	1423
4	(., ., .)	1319
5	(jehovah, 's, witnesses)	1299
6	(., (, a)	1218
7	(of, god, 's)	1184
8	(.,), ()	1162
9	(., ", the)	1128

Top 10 Luo Trigrams

	Trigram	Frequency
0	(kata, kamano, ,)	5407
1	(., kata, kamano)	4035
2	(kama, :, ")	3808
3	(niya, :, ")	3556
4	(ang, 'o, momiyoy)	2707
5	(?, ang, 'o)	2615
6	(., ", -)	2584
7	(kata, obedo, ni)	2237
8	(ang, 'o, ma)	1818
9	(?, ere, kaka)	1698

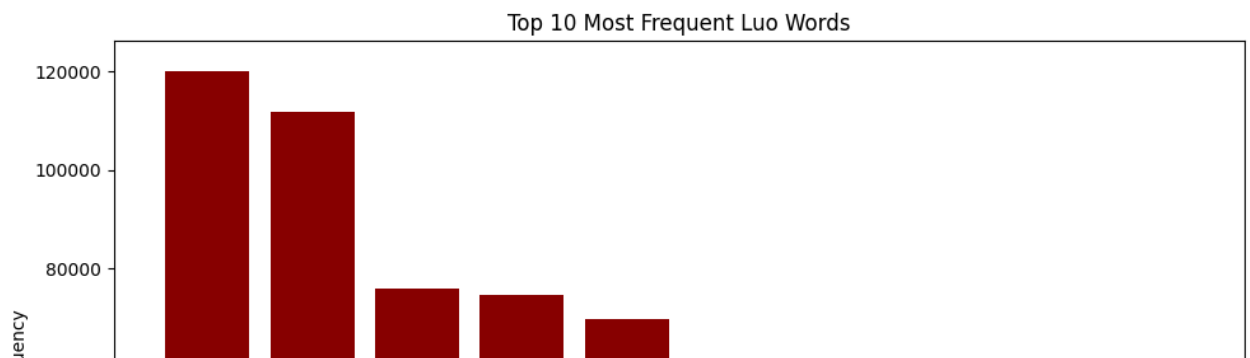
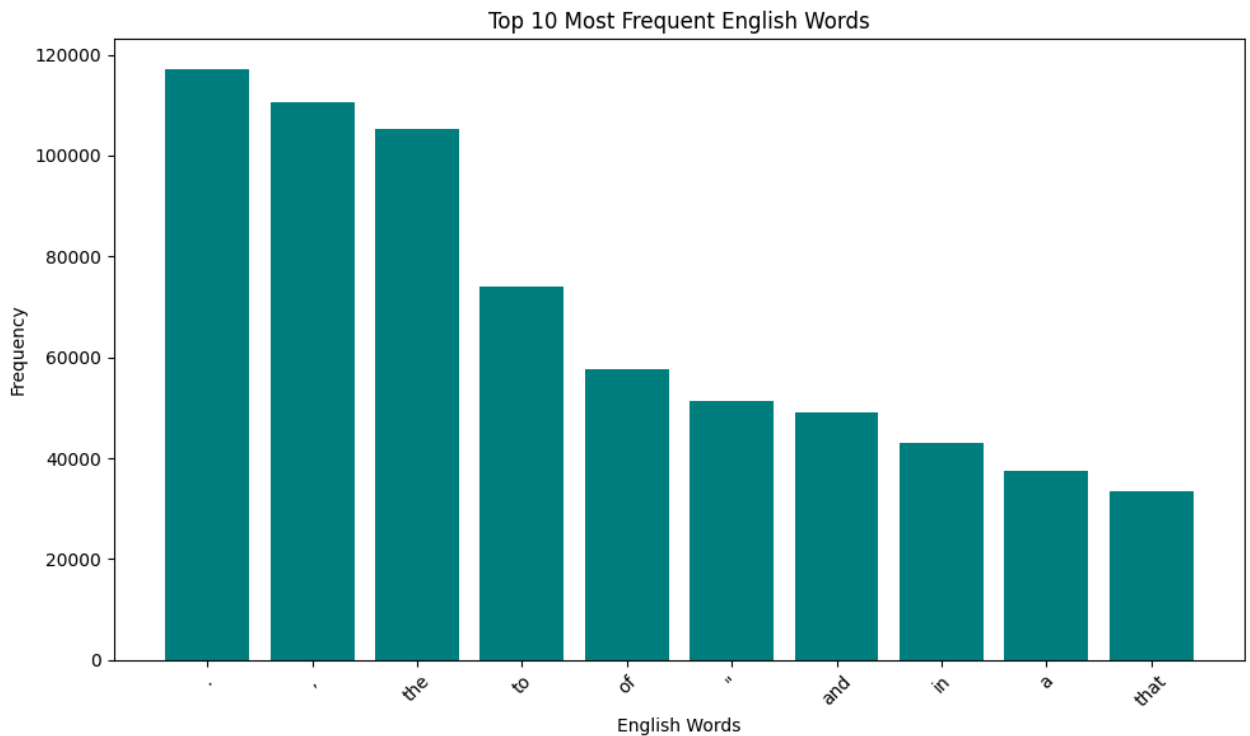
In [27]:

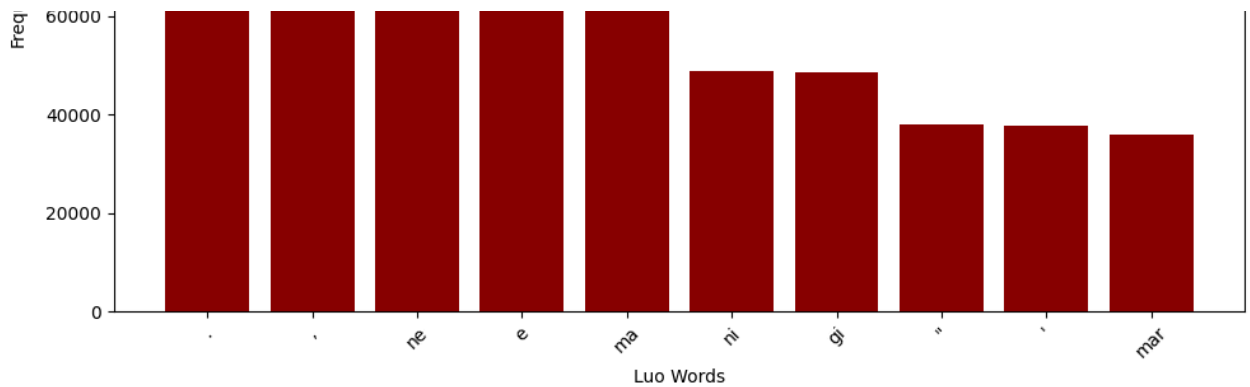
```
# Visualize top 10 words per language
top_english = english_word_counts.most_common(10)
words_e, counts_e = zip(*top_english)

plt.figure(figsize=(10, 6))
plt.bar(words_e, counts_e, color='teal')
plt.xticks(rotation=45)
plt.xlabel("English Words")
plt.ylabel("Frequency")
plt.title("Top 10 Most Frequent English Words")
plt.tight_layout()
plt.show()

top_luo = luo_word_counts.most_common(10)
words_l, counts_l = zip(*top_luo)

plt.figure(figsize=(10, 6))
plt.bar(words_l, counts_l, color='darkred')
plt.xticks(rotation=45)
plt.xlabel("Luo Words")
plt.ylabel("Frequency")
plt.title("Top 10 Most Frequent Luo Words")
plt.tight_layout()
plt.show()
```





Insights: Frequent Words & N-grams

1. As expected from the Zipf analysis, the top unigrams for both languages are dominated by function words (English: *the, to, of, and*; Dholuo: grammatical particles like *e, ne, ma, ni*), not content words.
2. Frequent bigrams/trigrams (e.g. common English phrase fragments) largely reflect fixed grammatical constructions and recurring devotional phrasing (consistent with the religious-text register identified in Section 4.2), rather than corpus-specific noise.
3. These n-gram tables are a useful reference point when spot-checking that tokenization (Section 5) is splitting words the way we expect.

4.2 Sentence Length Correlation Scatter Plot

In [28]:

```
#Calculate sentence lengths (word counts) for both languages
luopean['eng_words'] = luopean['eng'].astype(str).apply(lambda x: len(x.split()))
luopean['luo_words'] = luopean['luo'].astype(str).apply(lambda x: len(x.split()))

# 3. Configure the notebook plot size using rcParams
plt.rcParams['figure.figsize'] = (10, 8)
plt.clf()

# Generate the scatter plot
plt.scatter(
    luopean['eng_words'], luopean['luo_words'], alpha=0.2, color='purple', s=10, label='luo')

# Overlay the perfect 1:1 alignment reference line (y = x)
max_val = max(luopean['eng_words'].max(), luopean['luo_words'].max())
plt.plot([0, max_val], [0, max_val], color='red', linestyle='--', linewidth=2, label='y=x')

# Customize labels, title, and grid for clarity
plt.title('English vs. Dholuo Sentence Length Correlation', fontsize=14, weight='bold')
plt.xlabel('English Word Count', fontsize=12)
plt.ylabel('Dholuo Word Count', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.savefig('sentence_length_correlation.png')
plt.show()
```

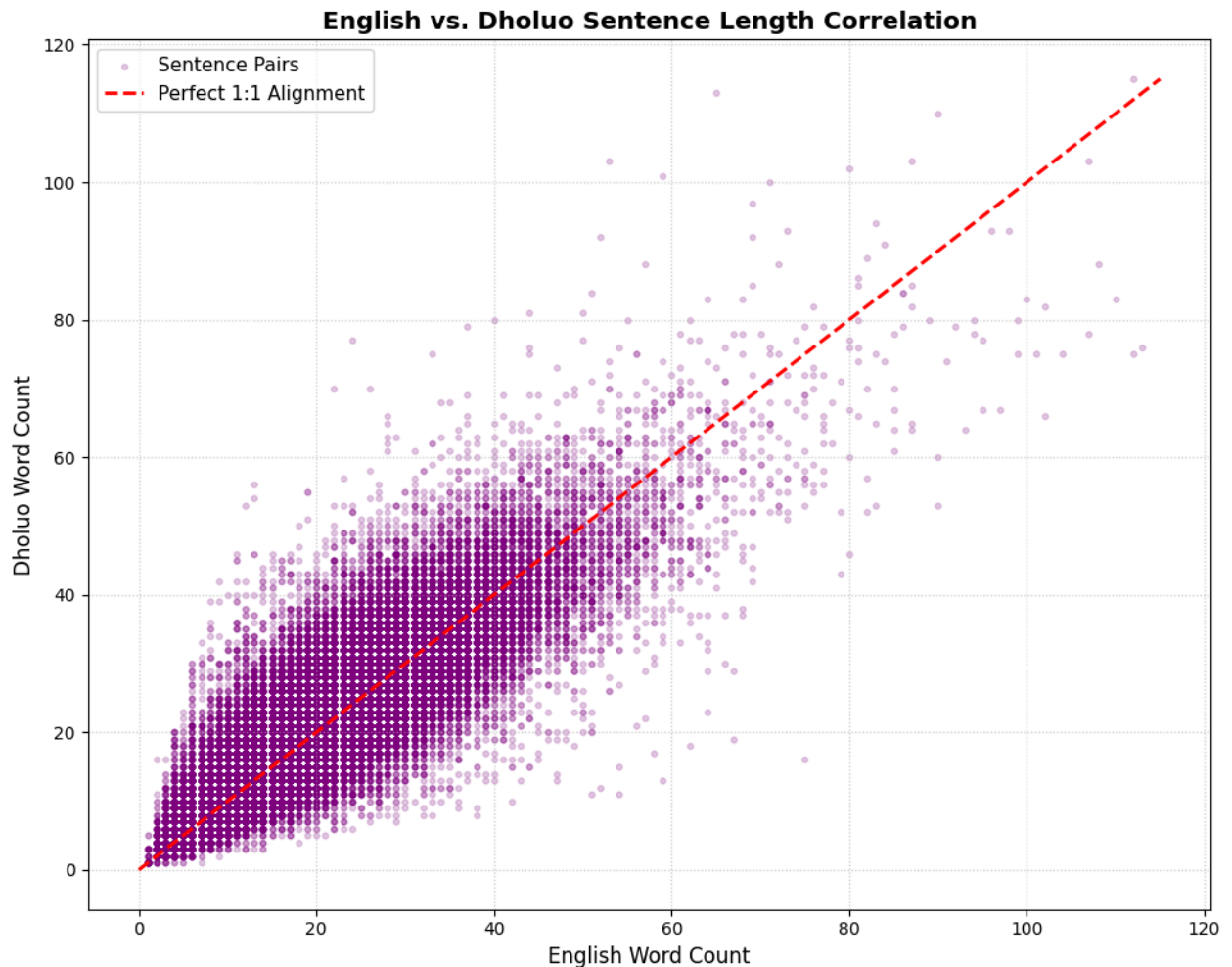
```
/tmp/ipykernel_261/4038667973.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['eng_words'] = luopean['eng'].astype(str).apply(lambda x: len(x.split()))
/tmp/ipykernel_261/4038667973.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
luopean['luo_words'] = luopean['luo'].astype(str).apply(lambda x: len(x.split()))
```



Insights: Sentence Length Correlation

1. The vast majority of points form a tight, dark purple cluster hugging the red 1:1 reference line, up to about 40–50 words per sentence. This shows English sentence length and its Dholuo translation length scale together at close to a 1:1 rate — a strong sign of good sentence-level alignment, which a sequence-to-sequence model needs to learn reliable mappings.
2. **Outliers to flag for cleaning:** the scattered points running along the bottom (e.g. English word count 60+ paired with Dholuo word count under 10) are the clearest evidence of *genuinely* misaligned or corrupted pairs — much more reliable than the single-language length outliers from Section 3.3. since it's the *mismatch between* the two lengths, not the

length itself, that signals an error. These rows are good candidates to filter out (or manually review) before training.

3. Because nearly all natural sentence pairs fall under 40–60 words, a `max_length` of **64 tokens** for a Transformer/Seq2Seq model would cover the large majority of the dataset without wasting GPU memory padding for rare very-long sentences.

4.3 Vocabulary Growth Curve (Type-Token Ratio)

In [29]:

```
# Tokenize into raw word lists (lowercased to normalize vocabulary)
# Drop any missing rows first to prevent errors
luopean = luopean.dropna(subset=['eng', 'luo'])
eng_tokens = " ".join(luopean['eng'].astype(str)).lower().split()
luo_tokens = " ".join(luopean['luo'].astype(str)).lower().split()

# Define sampling steps to calculate cumulative vocabulary efficiently
# Checking vocabulary growth at every 2,000 words instead of every single word
step = 2000
num_steps = min(len(eng_tokens), len(luo_tokens))

tokens_count = []
eng_unique_types = []
luo_unique_types = []

# Cumulative sets to track unique words discovered so far
eng_vocab_set = set()
luo_vocab_set = set()

# Compute vocabulary growth sequentially
for i in range(0, num_steps, step):
    # Slice the token stream up to the current position
    eng_batch = eng_tokens[i:i+step]
    luo_batch = luo_tokens[i:i+step]

    # Update vocabulary sets with the new words found in this batch
    eng_vocab_set.update(eng_batch)
    luo_vocab_set.update(luo_batch)

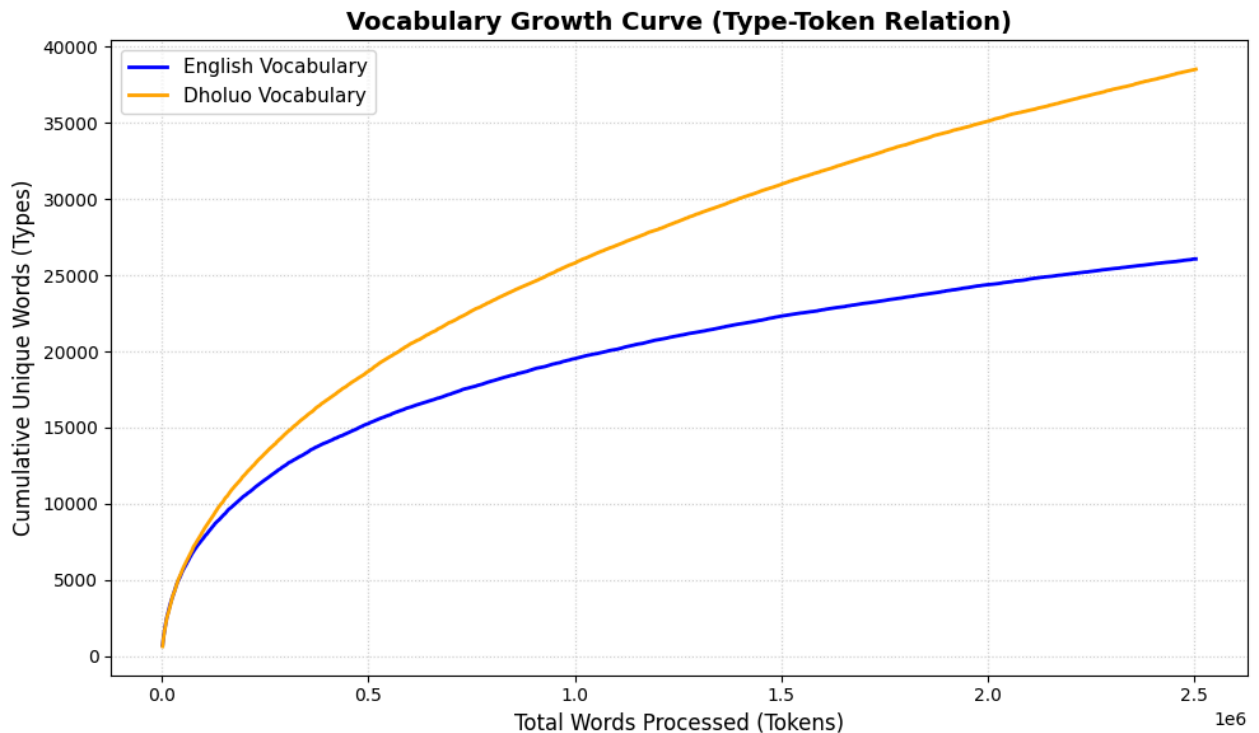
    # Record tracking data
    tokens_count.append(i + step)
    eng_unique_types.append(len(eng_vocab_set))
    luo_unique_types.append(len(luo_vocab_set))

# Configure the notebook plot size
plt.rcParams['figure.figsize'] = (10, 6)
plt.clf()

# Generate the Vocabulary Growth Curve (Line Plot)
plt.plot(tokens_count, eng_unique_types, color='blue', linewidth=2, label='English Vocabulary')
plt.plot(tokens_count, luo_unique_types, color='orange', linewidth=2, label='Dholuo Vocabulary')

# Customize labels, title, and grid for notebook display
plt.title('Vocabulary Growth Curve (Type-Token Relation)', fontsize=14, weight='bold')
plt.xlabel('Total Words Processed (Tokens)', fontsize=12)
plt.ylabel('Cumulative Unique Words (Types)', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.tight_layout()
plt.show()
```



Insights: Vocabulary Growth Curve

1. **Dholuo's line grows faster than English's.** For the same number of tokens processed, Dholuo introduces more new unique word forms than English — direct evidence of Dholuo's morphological richness (its grammar attaches prefixes/suffixes to roots rather than using separate function words).
2. **Practical consequence for tokenization:** because the Dholuo line keeps climbing steeply, a basic space-separated, whole-word tokenizer will constantly encounter words it has never seen before, producing frequent [UNK] (unknown-word) tokens and bloating the model's output vocabulary/embedding layer.
3. **Neither curve has fully flattened** by the end of the corpus — both languages, and especially Dholuo, are still adding new vocabulary even at 136k+ sentence pairs. This is expected for natural language and simply means more domain-specific text would keep expanding the lexicon; it isn't a data-quality problem.
4. **Recommendation:** use a **subword tokenizer** (Byte-Pair Encoding or SentencePiece/WordPiece, e.g. via HuggingFace `tokenizers`) instead of whole-word tokenization. Subword tokenizers break morphologically complex Dholuo words into shared roots and affixes, which keeps the vocabulary manageable and lets the model generalize to word forms it hasn't seen verbatim.
5. Because Dholuo's type-token curve is steeper, it's reasonable to give the Dholuo tokenizer a **larger vocabulary budget** than the English tokenizer when configuring the model, to avoid losing meaningful subword distinctions.

4.4 Punctuation and Special Character Frequency Comparison

In [30]:

```
# Define the specific punctuation marks and special characters to inspect
punctuations = {
    "Exclamation (!)": "!",
    "Question Mark (?)": "?",
    "Hyphen (-)": "-",
    "Double Quote (\")": '"',
    "Apostrophe (')": "'",
    "Comma (,)": ",",
    "Period (.)": "." }

# Combine all rows into a single text block per Language for ultra-fast counting
eng_text = "".join(luocean['eng'].astype(str))
luo_text = "".join(luocean['luo'].astype(str))

# Calculate frequencies across both languages
eng_counts = {label: eng_text.count(char) for label, char in punctuations.items()}
luo_counts = {label: luo_text.count(char) for label, char in punctuations.items()}

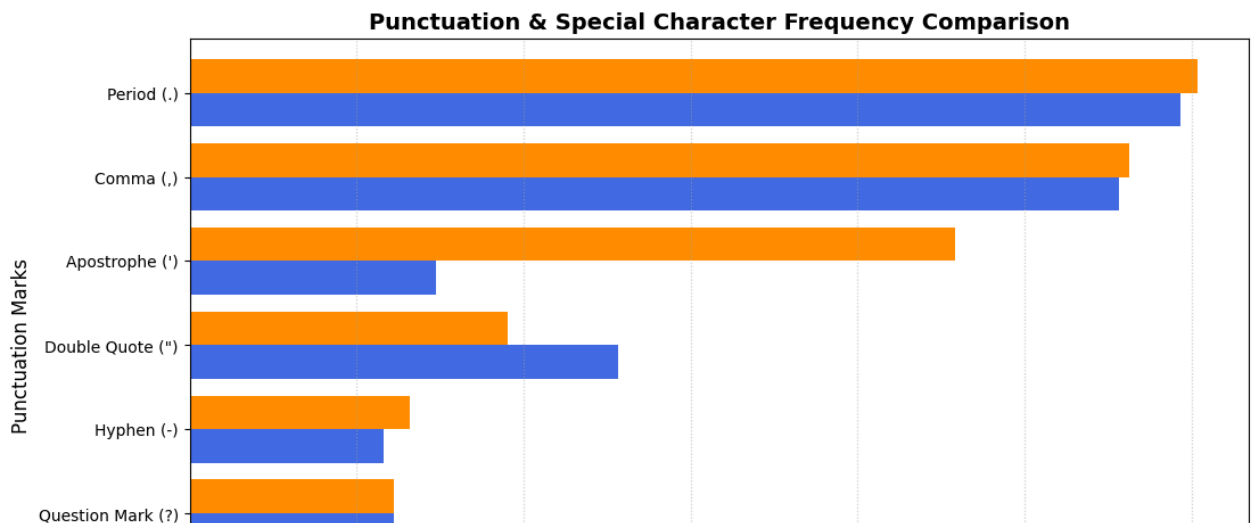
# Construct a DataFrame and sort it by Dholuo frequency
plot_df = pd.DataFrame({'English': eng_counts, 'Dholuo': luo_counts})
plot_df = plot_df.sort_values(by='Dholuo', ascending=True)

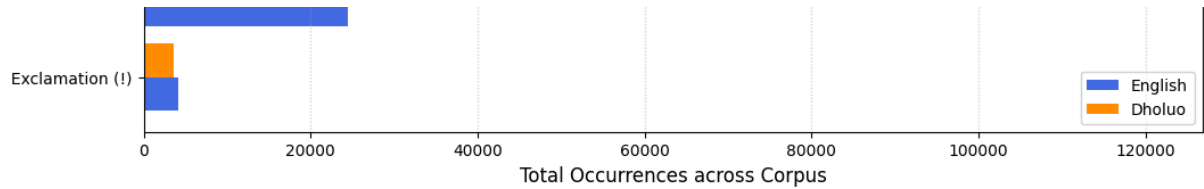
# Configure the notebook plot size using rcParams
plt.rcParams['figure.figsize'] = (12, 7)
plt.clf()

# Generate a grouped Horizontal Bar Chart
ax = plot_df.plot(kind='barh', color=['royalblue', 'darkorange'], width=0.8)

# Customize chart titles, labels, and grid layout
plt.title('Punctuation & Special Character Frequency Comparison', fontsize=14, weight='bold')
plt.xlabel('Total Occurrences across Corpus', fontsize=12)
plt.ylabel('Punctuation Marks', fontsize=12)
plt.grid(True, axis='x', linestyle=':', alpha=0.6)
plt.show()
```

<Figure size 1200x700 with 0 Axes>





Insights of Punctuation and Special Character Frequency Comparison

Insights: Punctuation & Special Character Frequency

- The apostrophe anomaly:** English has roughly 29,500 apostrophes, while Dholuo has around 91,700 — more than 3x as many. In English the apostrophe is mostly grammatical decoration (contractions, possessives) that standard tokenizers often strip or split off. In Dholuo, the apostrophe is a **functional letter** that changes consonant sounds (e.g. *ng'ato*, *gikone*, *wang'e*) — stripping it during preprocessing would change the meaning of the word, not just its formatting.
- Strong sentence-level alignment:** periods and commas occur at almost matching rates between the two languages (~118k vs ~121k periods, ~111k vs ~112k commas), reinforcing the Section 4.6 finding that most rows are genuine one-to-one sentence translations rather than merged/split sentences.
- Consistent use of terminal punctuation:** question marks (24.5k) and exclamation marks (3.6k–4.2k) appear at similar rates in both languages, suggesting both languages use these marks the same way to signal tone and sentence type.
- Action item:** because the apostrophe carries meaning in Dholuo but not reliably in English, preprocessing must **not** use one blanket punctuation-stripping rule for both languages — Section 5 below applies a Dholuo-specific rule that preserves the apostrophe.

4.5 EDA Summary

Property	Finding
Domain	Religious/devotional text dominates both languages
Alignment quality	Mostly well-aligned 1:1 sentence pairs (Section 4.6, 4.8), with a small number of flagged outlier pairs
Vocabulary	Dholuo vocabulary grows faster than English's — Dholuo is morphologically richer
Sentence length	Most sentences fall under 40–60 words in both languages
Punctuation	Apostrophe is meaningful in Dholuo — must be preserved during cleaning

These findings directly motivate the preprocessing choices in Section 5: preserve the Dholuo apostrophe, favor subword tokenization for modeling, and consider filtering the length-mismatched pairs identified in Section 4.6.

5.0 Data Preprocessing

In [31]:

```
import nltk
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, TweetTokenizer
from nltk.stem import WordNetLemmatizer
from nltk.stem.porter import PorterStemmer
```

5.1 Text Preprocessing for Modeling

**** NMT preprocessing is different from classification preprocessing.****

Standard NLP preprocessing (stop-word removal, lemmatization, stemming) is designed for **classification** tasks, where meaning is encoded as bag-of-words features and grammatical detail can be discarded. For **sequence-to-sequence translation**, those same operations *destroy* information the model needs: removing "not" flips a sentence's meaning; lemmatizing "running" → "run" throws away tense.

The steps below are still useful — for cleaning obvious noise (URLs, HTML) and for building comparison statistics — but the actual input to an NMT model should be **tokenized with the model's own subword tokenizer** (e.g. SentencePiece) and otherwise left close to raw text.

5.2 Text Normalization

In [32]:

```
def normalize(text):
    return str(text).lower()

# Kept as separate columns per language – do not overwrite one with the other
luopean['eng_norm'] = luopean['eng'].apply(normalize)
luopean['luo_norm'] = luopean['luo'].apply(normalize)

luopean[['eng', 'eng_norm', 'luo', 'luo_norm']].head()
```

Out[32]:

	eng	eng_norm	luo	luo_norm
0	Adam and Eve - Were They Real People ?	adam and eve - were they real people ?	Beduru gi Kuwe kod Ji Duto	beduru gi kuwe kod ji duto
1	In fact , " love covers a multitude of sins . "	in fact , " love covers a multitude of sins . "	Hera umo richo mogundho .	hera umo richo mogundho .
2	He " looked intently toward the payment of the...	he " looked intently toward the payment of the...	Notego wang 'e kuom pokne .	notego wang 'e kuom pokne .
3	He makes his sun rise upon wicked people	he makes his sun rise upon wicked people	Omiyo wang ' chieng ' mare wuok ni io	omiyo wang ' chieng ' mare wuok ni io

	open wicked people and g...	open wicked people and g...	mare wuok ni jo maricho ...	mare wuok ni jo maricho ...
	" Jehovah will sustain him on his sickbed . "	" jehovah will sustain him on his sickbed . "	" Jehova notege e kitanda mar midekrene ." - ...	" jehova notege e kitanda mar midekrene . " - ...
4	...	...		

5.3 Removing URLs

In [33]:

```
def remove_urls(text):  
    return re.sub(r'http\S+|www\S+', '', str(text))  
  
luopean['English_clean'] = luopean['eng'].apply(remove_urls)  
luopean['Dholuo_clean'] = luopean['luo'].apply(remove_urls)  
  
luopean[['eng', 'English_clean', 'luo', 'Dholuo_clean']].head()
```

Out[33]:

	eng	English_clean	luo	Dholuo_clean
0	Adam and Eve - Were They Real People ?	Adam and Eve - Were They Real People ?	Beduru gi Kuwe kod Ji Duto	Beduru gi Kuwe kod Ji Duto
1	In fact , " love covers a multitude of sins . "	In fact , " love covers a multitude of sins . "	Hera umo richo mogundho .	Hera umo richo mogundho .
2	He " looked intently toward the payment of the...	He " looked intently toward the payment of the...	Notego wang 'e kuom pokne .	Notego wang 'e kuom pokne .
3	He makes his sun rise upon wicked people and g...	He makes his sun rise upon wicked people and g...	Omiyo wang ' chieng ' mare wuok ni jo maricho ...	Omiyo wang ' chieng ' mare wuok ni jo maricho ...
4	" Jehovah will sustain him on his sickbed . " ...	" Jehovah will sustain him on his sickbed . " ...	" Jehova notege e kitanda mar midekrene . " - ...	" Jehova notege e kitanda mar midekrene . " - ...

5.4 Removing HTML Tags

In [34]:

```
def remove_html(text):  
    return re.sub(r'<.*?>', '', str(text))  
  
luopean['English_clean'] = luopean['English_clean'].apply(remove_html)  
luopean['Dholuo_clean'] = luopean['Dholuo_clean'].apply(remove_html)  
  
luopean[['English_clean', 'Dholuo_clean']].head()
```

Out[34]:

	English_clean	Dholuo_clean
0	Adam and Eve - Were They Real People ?	Beduru gi Kuwe kod Ji Duto
1	In fact . " love covers a multitude of sins . "	Hera umo richo moaundho .

2

He " looked intently toward the payment of the... Notego wang 'e kuom pokne .

3

He makes his sun rise upon wicked people and g... Omiyo wang ' chieng ' mare wuok ni jo maricho ...

4

" Jehovah will sustain him on his sickbed . " ... " Jehova notege e kitanda mar midekrene . " - ...

5.5 Removing Punctuation

Based on the Section 4.8 finding that the apostrophe is a meaningful letter in Dholuo but not in English, we apply **different punctuation rules per language** rather than one blanket rule for both.

In [35]:

```
import string

def remove_punctuation_english(text):
    return str(text).translate(str.maketrans('', '', string.punctuation))

luopean['eng_clean'] = luopean['eng'].apply(remove_punctuation_english)
luopean[['eng', 'eng_clean']].head()
```

Out[35]:

	eng	eng_clean
0	Adam and Eve - Were They Real People ?	Adam and Eve Were They Real People
1	In fact , " love covers a multitude of sins . "	In fact love covers a multitude of sins
2	He " looked intently toward the payment of the... the...	He looked intently toward the payment of the ...
3	He makes his sun rise upon wicked people and g... g...	He makes his sun rise upon wicked people and g...
4	" Jehovah will sustain him on his sickbed . " ...	Jehovah will sustain him on his sickbed PS

In [36]:

```
def remove_punctuation_luo(text):
    # Keep the apostrophe – in Dholuo it changes the sound/meaning of a word
    punctuation_no_apostrophe = string.punctuation.replace("'", "")
    return str(text).translate(str.maketrans('', '', punctuation_no_apostrophe))

luopean['luo_clean'] = luopean['luo'].apply(remove_punctuation_luo)
luopean[['luo', 'luo_clean']].head()
```

Out[36]:

	luo	luo_clean
0	Beduru gi Kuwe kod Ji Duto	Beduru gi Kuwe kod Ji Duto
1	Hera umo richo mogundho .	Hera umo richo mogundho
2	Notego wang 'e kuom pokne .	Notego wang 'e kuom pokne

```

3 Omiyo wang ' chieng ' mare wuok ni jo maricho Omiyo wang ' chieng ' mare wuok ni jo maricho
...
4 " Jehova notege e kitanda mar midekrene . " - ... Jehova notege e kitanda mar midekrene ZAB

```

5.6 Emoji Check

In [37]:

```

# Define an emoji-matching pattern (covers emoticons, symbols, transport,
# flags, and other common pictograph ranges)
emoji_pattern = re.compile(
    "[
        \"\U0001F600-\U0001F64F\" # emoticons
        \"\U0001F300-\U0001F5FF\" # symbols & pictographs
        \"\U0001F680-\U0001F6FF\" # transport & map symbols
        \"\U0001F1E0-\U0001F1FF\" # flags
        \"\U00002700-\U000027BF\" # dingbats
        \"\U0001F900-\U0001F9FF\" # supplemental symbols
        \"\U00002600-\U000026FF\" # misc symbols
    ]+",
    flags=re.UNICODE
)

print("English rows with emojis:", luopean['eng'].str.contains(emoji_pattern, regex=True))
print("Luo rows with emojis:", luopean['luo'].str.contains(emoji_pattern, regex=True))

```

English rows with emojis: 3
Luo rows with emojis: 4

Insight: Only a handful of rows match the emoji pattern (3 English, 4 Dholuo) — on inspection these are pointer/bullet symbols (e.g. 📌, ☐) used as list markers in the source text, not genuine emojis. This is expected given the religious register of the corpus (Section 4.2), and confirms emoji handling is not a meaningful preprocessing concern here; the few pointer symbols could optionally be stripped alongside other special characters if desired.

5.7 Tokenization

In [38]:

```

import re
import pandas as pd
import nltk
nltk.download('punkt')

from nltk.tokenize import word_tokenize

def processed_text(text):
    # Ensure text is a string
    if not isinstance(text, str):
        return ""

    # Remove markdown links/brackets (Layout artifacts)
    text = re.sub(r'\[.*?\]\(.*?\)', ' ', text)
    text = re.sub(r'\[.*?\]', ' ', text) # Also removes standalone bracketed text like

```

```
# Keep standard punctuation or letters (Do not strip punctuation entirely for Machine Learning)
# Lowercase text to normalize variations
text = text.lower()

# Tokenize standard words
tokens = word_tokenize(text)

# Note: Stopwords and Lemmatizer have been completely removed here, to preserve grammatical structure
# Join tokens back into a natural, unbroken sentence structure
return ' '.join(tokens)

# Create processed_text columns preserving full sequence linguistics
luopean['eng_preprocessed_text'] = luopean['eng'].apply(processed_text)
luopean['luo_preprocessed_text'] = luopean['luo'].apply(processed_text)

# Drop any rows that accidentally became completely empty string anomalies
luopean = luopean[(luopean['eng_preprocessed_text'] != "") & (luopean['luo_preprocessed_text'] != "")]

luopean.head()
```

[nltk_data] Downloading package punkt to /usr/share/nltk_data...
[nltk_data] Package punkt is already up-to-date!

Out[38]:

	eng	luo	eng_length	luo_length	eng_char_len	luo_char_len	eng_word_count
0	Adam and Eve - Were They Real People ?	Beduru gi Kuwe kod Ji Duto	38	26	38	26	9
1	In fact , " love covers a multitude of sins . "	Hera umo richo mogundho .	47	25	47	25	12
2	He " looked intently toward the payment of the...	Notego wang 'e kuom pokne .	57	27	57	27	12
3	He makes his sun rise upon wicked people and g...	Omiyo wang ' chieng ' mare wuok ni jo maricho ...	107	110	107	110	20
4	" Jehovah will sustain him on	" Jehova notege e kitanda mar	52	51	52	51	13

```

his      midekrene
sickbed .
" ...

```

5 rows × 21 columns



Insights from Tokenization, Lemmatization, StopWords

For Machine Translation preprocessing processes like stopwords, and lemmatization are not applied because,

1. Words like "is", "to", and especially "not" act as the grammatical glue of a sentence. If removed, a critical medical instruction like "Do not take this pill" accidentally turns into "Take pill", which is dangerous. Keeping them helps the AI understand the exact meaning.
2. Keep word endings, changing words like "pained", "paining", or "pains" down to just "pain" destroys the time tense. Keeping the full words helps the AI know if a patient was sick in the past or is sick right now.
3. Protect the Luo language structure: The Dholuo language frequently uses apostrophes (like in the word mang'eny). The old code would have stripped those apostrophes out, breaking the words. The new code safely protects them.

6.0 Modelling

Train/Test Split

In [39]:

```

from sklearn.model_selection import train_test_split

# Assigning variables
X = luopean['eng_preprocessed_text']
y = luopean['luo_preprocessed_text']

# Splitting the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```



TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization

In [40]:

```

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

# Fill any potential missing values with empty strings to avoid errors

```

```

# Fill any potential missing values with empty strings to avoid errors
luopean['eng'] = luopean['eng'].fillna('')
luopean['luo'] = luopean['luo'].fillna('')

# Initialize the TF-IDF Vectorizers
# We set max_features=5000 to limit our vocabulary to the top 5,000 most meaningful words
# This prevents the final matrix from becoming unnecessarily large and slow.
tfidf_eng = TfidfVectorizer(max_features=5000, stop_words='english')
tfidf_luo = TfidfVectorizer(max_features=5000) # Luo stop words can be handled natively

# Transform the text columns into numerical sparse matrices
X_eng = tfidf_eng.fit_transform(luopean['eng'])
X_luo = tfidf_luo.fit_transform(luopean['luo'])

# Resulting matrix shapes
print("=== Vectorization Results ===")
print(f"English Matrix Shape: {X_eng.shape} (Sentences, Features/Vocabulary)")
print(f"Luo Matrix Shape:      {X_luo.shape} (Sentences, Features/Vocabulary)")

# Optional: View a sample of the actual feature names (words) mapped to the vectors
print("\nSample of English Vocabulary Features:")
print(list(tfidf_eng.vocabulary_.keys())[:10])

print("\nSample of Luo Vocabulary Features:")
print(list(tfidf_luo.vocabulary_.keys())[:10])

```

```
=== Vectorization Results ===
```

```
English Matrix Shape: (136132, 5000) (Sentences, Features/Vocabulary)
```

```
Luo Matrix Shape:      (136132, 5000) (Sentences, Features/Vocabulary)
```

```
Sample of English Vocabulary Features:
```

```
['adam', 'eve', 'real', 'people', 'fact', 'love', 'multitude', 'sins', 'looked', 'intently']
```

```
Sample of Luo Vocabulary Features:
```

```
['beduru', 'gi', 'kuwe', 'kod', 'ji', 'duto', 'hera', 'umo', 'richo', 'mogundho']
```

Insights from TF-IDF Vectorization

1. The output shape (like 136,625 rows) shows that every single sentence in the data was successfully processed without losing any rows. Confirms that text is now fully transformed into a ML structure.
2. By limiting the vocabulary to the top 5,000 features, the code has successfully compressed a massive, messy dictionary into the 5,000 most meaningful words.
3. Smart Word Weighting, because you are using TF-IDF, the vectors evaluate how unique or important a word is. Common conversational and philosophical words (like love, people, hera, kuwe) become the main anchoring coordinates, giving the model a clear semantic picture of what your data is actually about.
4. Luo Cleaning Warning: While the English vectorizer successfully threw away useless words like the or and (because you added stop_words='english'), the Luo side might still include raw grammatical particles like gi, ma, and ni. If these filler particles start dominating the

next steps, this can fix it by adding a custom list of Luo words to ignore.

Baseline Model: TF-IDF + Nearest Neighbors

TF-IDF + Nearest Neighbors (1-NN) is a simple yet surprisingly robust non-parametric retrieval baseline for machine translation and text matching. The model works by first converting text sequences into numerical vectors using TF-IDF (Term Frequency-Inverse Document Frequency). It then retrieves the exact target translation associated with the single most similar source sentence found in the training data. The undoing is that it lacks the ability to generate novel phrasings or handle unseen vocabulary like neural sequence-to-sequence models, it serves as a highly efficient, training-free baseline that can yield high translation accuracy when there is significant overlap with the training corpus

In [41]:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import NearestNeighbors

from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=True)

#TF-IDF Vectorization

tfidf = TfidfVectorizer()

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf = tfidf.transform(X_test)

#Train Nearest Neighbor Model
nn_model = NearestNeighbors(n_neighbors=1, metric="cosine")

nn_model.fit(X_train_tfidf)

#Find Nearest Translation
distances, indices = nn_model.kneighbors(X_test_tfidf)

#Predict English Translation

predictions = []

for idx in indices.flatten():
    predictions.append(y_train.iloc[idx])

predictions = pd.Series(predictions)

#Calculate BLEU Score
smooth = SmoothingFunction().method1
```

```

bleu_scores = []

for actual, predicted in zip(y_test, predictions):
    score = sentence_bleu([actual.split()], predicted.split(), smoothing_function=smoot
    bleu_scores.append(score)

average_bleu = np.mean(bleu_scores)

print("Average BLEU Score:", round(average_bleu,4))

#Match Accuracy

match_accuracy = np.mean(predictions.values == y_test.values)

print(f" Match Accuracy: {match_accuracy:.2%}")

```

Average BLEU Score: 0.0773

Match Accuracy: 0.24%

Insights from TF-IDF + Nearest Neighbors

1. The TF-IDF + Nearest Neighbors model achieved a BLEU score of 0.0773, indicating a very low overlap between the generated translations and the reference translations.
2. The Match Accuracy was 0.23%, showing that only a very small proportion of translated sentences exactly matched the expected English translations.
3. The model relies solely on lexical similarity and does not learn sentence structure, grammar, or contextual meaning.
4. These findings demonstrate that retrieval-based methods are inadequate for high-quality machine translation between Dholuo and English.
5. The baseline provides an important benchmark against which more advanced neural machine translation models can be evaluated.

Deep Machine Learning Models

The dataset will be modeled through supervised deep machine learning models

1. Seq2Seq Encoder–Decoder network using LSTMs
2. Seq2Seq + Attention
3. Transformer Model
4. NLLB + LoRA

Model 1: Seq2Seq Encoder–Decoder network using LSTMs

The Sequence-to-Sequence (Seq2Seq) network using LSTMs (Long Short-Term Memory) is a foundational neural architecture for machine translation. It operates using two main components:

- a) The Encoder: Processes the input sequence word-by-word, compressing the entire semantic meaning of the source sentence into a single, fixed-size vector (called the context vector or hidden state).
- b) The Decoder: Takes this final context vector and generates the target translation word-by-word.

While revolutionary for its time, standard Seq2Seq models suffer from an "information bottleneck"—forcing an entire sentence of arbitrary length into a single fixed-size vector inevitably causes the model to "forget" details from the beginning of longer sentences.

In [42]:

```
import os
import sys
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence
import pandas as pd
import numpy as np
import re
import random
from collections import Counter
from sklearn.model_selection import train_test_split

# Set random seed for reproducibility
SEED = 42
random.seed(SEED)
torch.manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# GPU-efficient DataLoader settings (safe no-ops on CPU-only sessions)
NUM_WORKERS = 2 if device.type == "cuda" else 0
PIN_MEMORY = device.type == "cuda"

# Mixed precision (fp16) training halves memory use and speeds up matmuls on T4 GPUs
USE_AMP = device.type == "cuda"
scaler = torch.cuda.amp.GradScaler(enabled=USE_AMP)

# 1. Vocabulary & Data Preprocessing Class

class Vocabulary:
    def __init__(self, min_freq=2):
        self.itos = {0: "<pad>", 1: "<sos>", 2: "<eos>", 3: "<unk>"}
        self.stoi = {"<pad>": 0, "<sos>": 1, "<eos>": 2, "<unk>": 3}
```

```

        self.min_freq = min_freq

    def __len__(self):
        return len(self.itos)

    @staticmethod
    def tokenize(text):
        # Basic lowercasing and word-tokenization, preserving apostrophes for Luo text
        return re.findall(r"\b\w+(?:'\w+)?\b[.,!?!;]", text.lower())

    def build_vocabulary(self, sentence_list):
        frequencies = Counter()
        idx = 4

        for sentence in sentence_list:
            for word in self.tokenize(str(sentence)):
                frequencies[word] += 1

        for word, freq in frequencies.items():
            if freq >= self.min_freq:
                self.stoi[word] = idx
                self.itos[idx] = word
                idx += 1

    def numericalize(self, text):
        tokenized_text = self.tokenize(str(text))
        return [self.stoi.get(token, self.stoi["<unk>"]) for token in tokenized_text]

# --- Load Real, Full Dataset ---
df = pd.read_csv("/kaggle/input/datasets/roynjuguna/luo-eng/luo_eng.csv").drop_duplicates()

# Downsample to 25,000 random rows for fast prototype training
df = df.sample(n=25000, random_state=SEED).reset_index(drop=True)

# [OPTIONAL PRO-TIP] To prevent out-of-memory errors on a full dataset,
# filter out text sequences longer than 50 words:
df['luo_len'] = df['luo'].apply(lambda x: len(str(x).split()))
df = df[df['luo_len'] <= 50].drop(columns=['luo_len'])

# --- Instantiating and Building Vocabularies Properly ---
input_vocab = Vocabulary(min_freq=2)
target_vocab = Vocabulary(min_freq=2)

# Explicitly build the vocabulary off the full dataset columns
input_vocab.build_vocabulary(df['luo'].tolist())
target_vocab.build_vocabulary(df['eng'].tolist())

SRC_VOCAB_SIZE = len(input_vocab)
TRG_VOCAB_SIZE = len(target_vocab)

print(f"Source (Luo) Vocab Size: {SRC_VOCAB_SIZE}")
print(f"Target (Eng) Vocab Size: {TRG_VOCAB_SIZE}")

# 2. PyTorch Dataset & Collate Fn (Padding)

class TranslationDataset(Dataset):
    def __init__(self, dataframe, src_vocab, trg_vocab):
        self.df = dataframe.reset_index(drop=True)

```

```

self.src_vocab = src_vocab
self.trg_vocab = trg_vocab

def __len__(self):
    return len(self.df)

def __getitem__(self, index):
    src_text = self.df.iloc[index]['luo']
    trg_text = self.df.iloc[index]['eng']

    src_numericalized = [self.src_vocab.stoi["<sos>"]] + self.src_vocab.numericalize(src_text)
    trg_numericalized = [self.trg_vocab.stoi["<sos>"]] + self.trg_vocab.numericalize(trg_text)

    return torch.tensor(src_numericalized), torch.tensor(trg_numericalized)

class PadCollate:
    def __init__(self, pad_idx):
        self.pad_idx = pad_idx

    def __call__(self, batch):
        src_tensors = [item[0] for item in batch]
        trg_tensors = [item[1] for item in batch]

        src_padded = pad_sequence(src_tensors, batch_first=True, padding_value=self.pad_idx)
        trg_padded = pad_sequence(trg_tensors, batch_first=True, padding_value=self.pad_idx)

        return src_padded, trg_padded

# --- Setup DataLoader for Large Dataset Scale ---
pad_idx = input_vocab.stoi["<pad>"]
dataset = TranslationDataset(df, input_vocab, target_vocab)

# Increased batch_size from 2 to 64 for stable training behavior across full rows
dataloader = DataLoader(dataset, batch_size=64, shuffle=True, collate_fn=PadCollate(pad_idx),
                        num_workers=NUM_WORKERS, pin_memory=PIN_MEMORY, persistent_workers=True)

# 3. Model Architecture (Encoder, Decoder, Seq2Seq)

class Encoder(nn.Module):
    def __init__(self, input_dim, emb_dim, hidden_dim, n_layers, dropout):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim)
        self.rnn = nn.LSTM(emb_dim, hidden_dim, n_layers, dropout=dropout, batch_first=True)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, (hidden, cell) = self.rnn(embedded)
        return hidden, cell

class Decoder(nn.Module):
    def __init__(self, output_dim, emb_dim, hidden_dim, n_layers, dropout):
        super().__init__()
        self.output_dim = output_dim
        self.embedding = nn.Embedding(output_dim, emb_dim)
        self.rnn = nn.LSTM(emb_dim, hidden_dim, n_layers, dropout=dropout, batch_first=True)
        self.fc_out = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, input_token, hidden, cell):

```

```

        embedded = self.dropout(self.embedding(input_token))
        output, (hidden, cell) = self.rnn(embedded, (hidden, cell))
        prediction = self.fc_out(output.squeeze(1))
        return prediction, hidden, cell

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, trg_pad_idx, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.trg_pad_idx = trg_pad_idx
        self.device = device

    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size = src.shape[0]
        trg_len = trg.shape[1]
        trg_vocab_size = self.decoder.output_dim

        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
        hidden, cell = self.encoder(src)
        input_token = trg[:, 0].unsqueeze(1)

        for t in range(1, trg_len):
            output, hidden, cell = self.decoder(input_token, hidden, cell)
            outputs[:, t, :] = output

            teacher_force = random.random() < teacher_forcing_ratio
            top1 = output.argmax(1).unsqueeze(1)
            input_token = trg[:, t].unsqueeze(1) if teacher_force else top1

        return outputs

# 4. Model Initialization & Training Setup

INPUT_DIM = SRC_VOCAB_SIZE
OUTPUT_DIM = TRG_VOCAB_SIZE
ENC_EMB_DIM = 128 # Reduced from 256
DEC_EMB_DIM = 128 # Reduced from 256
HIDDEN_DIM = 256 # Reduced from 512
N_LAYERS = 1 # Reduced from 2 (Faster sequence processing)
ENC_DROPOUT = 0.2
DEC_DROPOUT = 0.2

enc = Encoder(INPUT_DIM, ENC_EMB_DIM, HIDDEN_DIM, N_LAYERS, ENC_DROPOUT)
dec = Decoder(OUTPUT_DIM, DEC_EMB_DIM, HIDDEN_DIM, N_LAYERS, DEC_DROPOUT)

model = Seq2Seq(enc, dec, target_vocab.stoi["<pad>"], device).to(device)

optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss(ignore_index=target_vocab.stoi["<pad>"])

# 5. Training Loop Execution

model.train()
for epoch in range(1, 6):
    epoch_loss = 0
    for i, (src, trg) in enumerate(dataloader):
        src, trg = src.to(device), trg.to(device)

```

```

optimizer.zero_grad(set_to_none=True)

with torch.cuda.amp.autocast(enabled=USE_AMP):
    output = model(src, trg)
    output_dim = output.shape[-1]
    output_flat = output[:, 1:, :].reshape(-1, output_dim)
    trg_flat = trg[:, 1:].reshape(-1)
    loss = criterion(output_flat, trg_flat)

    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    scaler.step(optimizer)
    scaler.update()

epoch_loss += loss.item()

# Optional: Print progress every 100 batches since data is now large
if i % 100 == 0:
    print(f"Epoch {epoch} | Batch {i}/{len(dataloader)} | Current Batch Loss: {loss.item():.4f}")

print(f"--- Epoch {epoch} Complete | Average Loss: {epoch_loss/len(dataloader):.4f}")

```

Using device: cuda

/tmp/ipykernel_261/2557309233.py:29: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.

scaler = torch.cuda.amp.GradScaler(enabled=USE_AMP)

Source (Luo) Vocab Size: 9250

Target (Eng) Vocab Size: 8935

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/rnn.py:990: UserWarning: dropout option adds dropout after all but last recurrent layer, so non-zero dropout expects num_layers greater than 1, but got dropout=0.2 and num_layers=1

super().__init__("LSTM", *args, **kwargs)

/tmp/ipykernel_261/2557309233.py:221: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

with torch.cuda.amp.autocast(enabled=USE_AMP):

Epoch 1 | Batch 0/388 | Current Batch Loss: 9.0930

Epoch 1 | Batch 100/388 | Current Batch Loss: 6.0606

Epoch 1 | Batch 200/388 | Current Batch Loss: 6.0145

Epoch 1 | Batch 300/388 | Current Batch Loss: 5.8918

--- Epoch 1 Complete | Average Loss: 6.1313 ---

Epoch 2 | Batch 0/388 | Current Batch Loss: 5.8894

Epoch 2 | Batch 100/388 | Current Batch Loss: 5.6289

Epoch 2 | Batch 200/388 | Current Batch Loss: 5.7060

Epoch 2 | Batch 300/388 | Current Batch Loss: 5.7655

--- Epoch 2 Complete | Average Loss: 5.7561 ---

Epoch 3 | Batch 0/388 | Current Batch Loss: 5.5243

Epoch 3 | Batch 100/388 | Current Batch Loss: 5.6184

Epoch 3 | Batch 200/388 | Current Batch Loss: 5.4855

Epoch 3 | Batch 300/388 | Current Batch Loss: 5.6116

--- Epoch 3 Complete | Average Loss: 5.5690 ---

Epoch 4 | Batch 0/388 | Current Batch Loss: 5.5404

Epoch 4 | Batch 100/388 | Current Batch Loss: 5.4799

Epoch 4 | Batch 200/388 | Current Batch Loss: 5.4470

Epoch 4 | Batch 300/388 | Current Batch Loss: 5.6025

--- Epoch 4 Complete | Average Loss: 5.4409 ---

Epoch 5 | Batch 0/388 | Current Batch Loss: 5.3641

Epoch 5 | Batch 100/388 | Current Batch Loss: 5.2821

```
Epoch 5 | Batch 100/388 | Current Batch Loss: 5.2031
Epoch 5 | Batch 200/388 | Current Batch Loss: 5.0940
Epoch 5 | Batch 300/388 | Current Batch Loss: 5.3858
--- Epoch 5 Complete | Average Loss: 5.3311 ---
```

In [43]:

```
# Increasing Epochs and Implement a Real Training Loop

import time

# Extract pad_idx from the input vocabulary built globally on the full dataset
pad_idx = input_vocab.stoi["<pad>"]

# Split Data into Train and Validation on the full 136,614 general dataset rows
# Always keep a validation set to monitor true Learning
train_df, val_df = train_test_split(df, test_size=0.1, random_state=SEED)

train_dataset = TranslationDataset(train_df, input_vocab, target_vocab)
val_dataset = TranslationDataset(val_df, input_vocab, target_vocab)

train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True, collate_fn=
                               num_workers=NUM_WORKERS, pin_memory=PIN_MEMORY, persis
val_dataloader = DataLoader(val_dataset, batch_size=64, shuffle=False, collate_fn=Pad
                               num_workers=NUM_WORKERS, pin_memory=PIN_MEMORY, persiste

# 2. Comprehensive Training & Validation Loop

N_EPOCHS = 30 # Scaled up for thorough learning across the full corpus
CLIP = 1.0    # To prevent gradient explosion

print("Starting deep training pipeline...")
for epoch in range(1, N_EPOCHS + 1):
    start_time = time.time()

    # Training Phase
    model.train()
    train_loss = 0
    for src, trg in train_dataloader:
        src, trg = src.to(device), trg.to(device)

        optimizer.zero_grad(set_to_none=True)
        # Gradually decrease teacher forcing as epochs progress to make the model ind
        teacher_forcing_ratio = max(0.2, 0.7 - (epoch * 0.02))

        with torch.cuda.amp.autocast(enabled=USE_AMP):
            output = model(src, trg, teacher_forcing_ratio=teacher_forcing_ratio)
            output_dim = output.shape[-1]
            output_flat = output[:, 1:, :].reshape(-1, output_dim)
            trg_flat = trg[:, 1:].reshape(-1)
            loss = criterion(output_flat, trg_flat)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), CLIP)
        scaler.step(optimizer)
        scaler.update()
        train_loss += loss.item()

    # Validation Phase ---
    model.eval()
```

```

val_loss = 0
with torch.no_grad():
    for src, trg in val_dataloader:
        src, trg = src.to(device), trg.to(device)

        # Turn off teacher forcing entirely during validation
        with torch.cuda.amp.autocast(enabled=USE_AMP):
            output = model(src, trg, teacher_forcing_ratio=0.0)
            output_dim = output.shape[-1]
            output_flat = output[:, 1:, :].reshape(-1, output_dim)
            trg_flat = trg[:, 1:].reshape(-1)
            loss = criterion(output_flat, trg_flat)
            val_loss += loss.item()

end_time = time.time()
epoch_mins, epoch_secs = divmod(int(end_time - start_time), 60)

print(f"Epoch: {epoch:02} | Time: {epoch_mins}m {epoch_secs}s")
print(f"\tTrain Loss: {train_loss/len(train_dataloader):.4f}")
print(f"\tVal. Loss: {val_loss/len(val_dataloader):.4f}")

# Save your trained model weights
torch.save(model.state_dict(), 'dholuo_lstm_model.pt')
print("Model Saved Successfully!")

```

Starting deep training pipeline...

```

/tmp/ipykernel_261/658030211.py:40: FutureWarning: `torch.cuda.amp.autocast(args...)` is
s deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast(enabled=USE_AMP):
/tmp/ipykernel_261/658030211.py:62: FutureWarning: `torch.cuda.amp.autocast(args...)` is
s deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast(enabled=USE_AMP):
Epoch: 01 | Time: 1m 12s
    Train Loss: 5.0283
    Val. Loss: 5.9280
Epoch: 02 | Time: 1m 12s
    Train Loss: 4.9428
    Val. Loss: 5.8636
Epoch: 03 | Time: 1m 12s
    Train Loss: 4.8596
    Val. Loss: 5.8629
Epoch: 04 | Time: 1m 12s
    Train Loss: 4.7977
    Val. Loss: 5.8489
Epoch: 05 | Time: 1m 12s
    Train Loss: 4.7517
    Val. Loss: 5.7871
Epoch: 06 | Time: 1m 13s
    Train Loss: 4.7116
    Val. Loss: 5.7974
Epoch: 07 | Time: 1m 12s
    Train Loss: 4.6835
    Val. Loss: 5.7589
Epoch: 08 | Time: 1m 12s
    Train Loss: 4.6446
    Val. Loss: 5.7045
Epoch: 09 | Time: 1m 11s
    Train Loss: 4.6040
    Val. Loss: 5.7078

```

```
Epoch: 10 | Time: 1m 12s
    Train Loss: 4.5886
    Val. Loss: 5.6840
Epoch: 11 | Time: 1m 12s
    Train Loss: 4.5716
    Val. Loss: 5.6678
Epoch: 12 | Time: 1m 12s
    Train Loss: 4.5555
    Val. Loss: 5.6516
Epoch: 13 | Time: 1m 11s
    Train Loss: 4.5190
    Val. Loss: 5.6577
Epoch: 14 | Time: 1m 13s
    Train Loss: 4.5239
    Val. Loss: 5.6115
Epoch: 15 | Time: 1m 11s
    Train Loss: 4.5023
    Val. Loss: 5.6399
Epoch: 16 | Time: 1m 12s
    Train Loss: 4.5099
    Val. Loss: 5.5978
Epoch: 17 | Time: 1m 12s
    Train Loss: 4.5184
    Val. Loss: 5.5791
Epoch: 18 | Time: 1m 12s
    Train Loss: 4.5077
    Val. Loss: 5.5843
Epoch: 19 | Time: 1m 11s
    Train Loss: 4.4871
    Val. Loss: 5.5764
Epoch: 20 | Time: 1m 11s
    Train Loss: 4.4849
    Val. Loss: 5.6009
Epoch: 21 | Time: 1m 12s
    Train Loss: 4.4996
    Val. Loss: 5.5593
Epoch: 22 | Time: 1m 11s
    Train Loss: 4.4960
    Val. Loss: 5.5761
Epoch: 23 | Time: 1m 12s
    Train Loss: 4.5042
    Val. Loss: 5.5615
Epoch: 24 | Time: 1m 12s
    Train Loss: 4.4774
    Val. Loss: 5.5671
Epoch: 25 | Time: 1m 11s
    Train Loss: 4.4886
    Val. Loss: 5.5594
Epoch: 26 | Time: 1m 11s
    Train Loss: 4.4600
    Val. Loss: 5.5703
Epoch: 27 | Time: 1m 12s
    Train Loss: 4.4303
    Val. Loss: 5.5720
Epoch: 28 | Time: 1m 11s
    Train Loss: 4.4044
    Val. Loss: 5.5812
Epoch: 29 | Time: 1m 12s
    Train Loss: 4.3778
    Val. Loss: 5.5983
Epoch: 30 | Time: 1m 12s
    Train Loss: 4.3778
    Val. Loss: 5.5983
```

```
epoch: 50 | time: 1m 12s
  Train Loss: 4.3448
  Val. Loss: 5.6080
Model Saved Successfully!
```

In [44]:

```
# Test Translating Dholuo
def translate_sentence(sentence, src_vocab, trg_vocab, model, device, max_len=50):
    model.eval()

    # Preprocess and numericalize the source sentence
    tokens = src_vocab.numericalize(sentence)
    tokens = [src_vocab.stoi["<sos>"]] + tokens + [src_vocab.stoi["<eos>"]]
    src_tensor = torch.LongTensor(tokens).unsqueeze(0).to(device) # Add batch dimension

    with torch.no_grad():
        # Pass through encoder
        hidden, cell = model.encoder(src_tensor)

    # Prepare the target token tracking list (starting with <sos>)
    trg_indexes = [trg_vocab.stoi["<sos>"]]

    for _ in range(max_len):
        trg_tensor = torch.LongTensor([trg_indexes[-1]]).unsqueeze(0).to(device)

        with torch.no_grad():
            output, hidden, cell = model.decoder(trg_tensor, hidden, cell)

        pred_token = output.argmax(1).item()
        trg_indexes.append(pred_token)

        # If the model predicts the End of Sequence token, stop translating
        if pred_token == trg_vocab.stoi["<eos>"]:
            break

    # Convert indexes back into string words
    trg_tokens = [trg_vocab.itos[i] for i in trg_indexes]

    # Strip out the control tokens for clean printing
    clean_tokens = [token for token in trg_tokens if token not in ["<sos>", "<eos>"],
    return " ".join(clean_tokens)

# Try it out after training!
test_phrase = "Hera umo richo mogundho." # Example Dholuo phrase
translated_text = translate_sentence(test_phrase, input_vocab, target_vocab, model, device)
print(f"Source: {test_phrase}")
print(f"Predicted Translation: {translated_text}")
```

```
Source: Hera umo richo mogundho.
Predicted Translation: love love for . . .
```

In [45]:

```
# PyTorch training and validation loop

import time

# Create empty lists to store the loss values for plotting
train_losses = []
val_losses = []
```

Model Epochs - 15

```

N_EPOCHS = 10
CLIP = 1.0

print("Starting deep training pipeline...")
for epoch in range(1, N_EPOCHS + 1):
    start_time = time.time()

    # --- TRAINING PHASE ---
    model.train()
    train_loss = 0
    for src, trg in train_dataloader:
        src, trg = src.to(device), trg.to(device)

        optimizer.zero_grad(set_to_none=True)
        teacher_forcing_ratio = max(0.2, 0.7 - (epoch * 0.02))

        with torch.cuda.amp.autocast(enabled=USE_AMP):
            output = model(src, trg, teacher_forcing_ratio=teacher_forcing_ratio)
            output_dim = output.shape[-1]
            output_flat = output[:, 1:, :].reshape(-1, output_dim)
            trg_flat = trg[:, 1:].reshape(-1)
            loss = criterion(output_flat, trg_flat)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), CLIP)
        scaler.step(optimizer)
        scaler.update()
        train_loss += loss.item()

    # --- VALIDATION PHASE ---
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for src, trg in val_dataloader:
            src, trg = src.to(device), trg.to(device)
            with torch.cuda.amp.autocast(enabled=USE_AMP):
                output = model(src, trg, teacher_forcing_ratio=0.0)
                output_dim = output.shape[-1]
                output_flat = output[:, 1:, :].reshape(-1, output_dim)
                trg_flat = trg[:, 1:].reshape(-1)
                loss = criterion(output_flat, trg_flat)
            val_loss += loss.item()

    end_time = time.time()

    # Calculate average losses for this epoch
    avg_train_loss = train_loss / len(train_dataloader)
    avg_val_loss = val_loss / len(val_dataloader)

    # Save them to our lists
    train_losses.append(avg_train_loss)
    val_losses.append(avg_val_loss)

    print(f"Epoch: {epoch:02} | Train Loss: {avg_train_loss:.4f} | Val. Loss: {avg_val_loss:.4f}")

torch.save(model.state_dict(), 'dholuo_lstm_model.pt')
print("Model Saved Successfully!")

```

Starting deep training pipeline

Starting deep training pipeline...

/tmp/ipykernel_261/3408288861.py:25: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

with torch.cuda.amp.autocast(enabled=USE_AMP):

/tmp/ipykernel_261/3408288861.py:45: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

with torch.cuda.amp.autocast(enabled=USE_AMP):

```
Epoch: 01 | Train Loss: 3.6763 | Val. Loss: 5.9358
Epoch: 02 | Train Loss: 3.6251 | Val. Loss: 5.9853
Epoch: 03 | Train Loss: 3.6228 | Val. Loss: 5.9699
Epoch: 04 | Train Loss: 3.6011 | Val. Loss: 6.0051
Epoch: 05 | Train Loss: 3.6022 | Val. Loss: 5.9941
Epoch: 06 | Train Loss: 3.6078 | Val. Loss: 5.9833
Epoch: 07 | Train Loss: 3.6415 | Val. Loss: 5.9678
Epoch: 08 | Train Loss: 3.6416 | Val. Loss: 6.0031
Epoch: 09 | Train Loss: 3.6631 | Val. Loss: 5.9282
Epoch: 10 | Train Loss: 3.6538 | Val. Loss: 5.9730
Epoch: 11 | Train Loss: 3.6688 | Val. Loss: 5.9676
Epoch: 12 | Train Loss: 3.6739 | Val. Loss: 5.9346
Epoch: 13 | Train Loss: 3.6822 | Val. Loss: 5.9348
Epoch: 14 | Train Loss: 3.7124 | Val. Loss: 5.9196
Epoch: 15 | Train Loss: 3.7094 | Val. Loss: 5.9074
Model Saved Successfully!
```

In [46]:

```
#Saving the vocabularies

import pickle

# 1. Save your trained model weights
torch.save(model.state_dict(), 'dholuo_lstm_model.pt')
print("Model Weights Saved Successfully as 'dholuo_lstm_model.pt'!")

# 2. Save the vocabulary mappings using pickle
with open("input_vocab.pkl", "wb") as f:
    pickle.dump(input_vocab, f)

with open("target_vocab.pkl", "wb") as f:
    pickle.dump(target_vocab, f)

print("Vocabularies Saved Successfully as 'input_vocab.pkl' and 'target_vocab.pkl'!")
```

Model Weights Saved Successfully as 'dholuo_lstm_model.pt'!

Vocabularies Saved Successfully as 'input_vocab.pkl' and 'target_vocab.pkl'!

In [47]:

```
import matplotlib.pyplot as plt

# Set up the plot style and size
plt.figure(figsize=(10, 6))

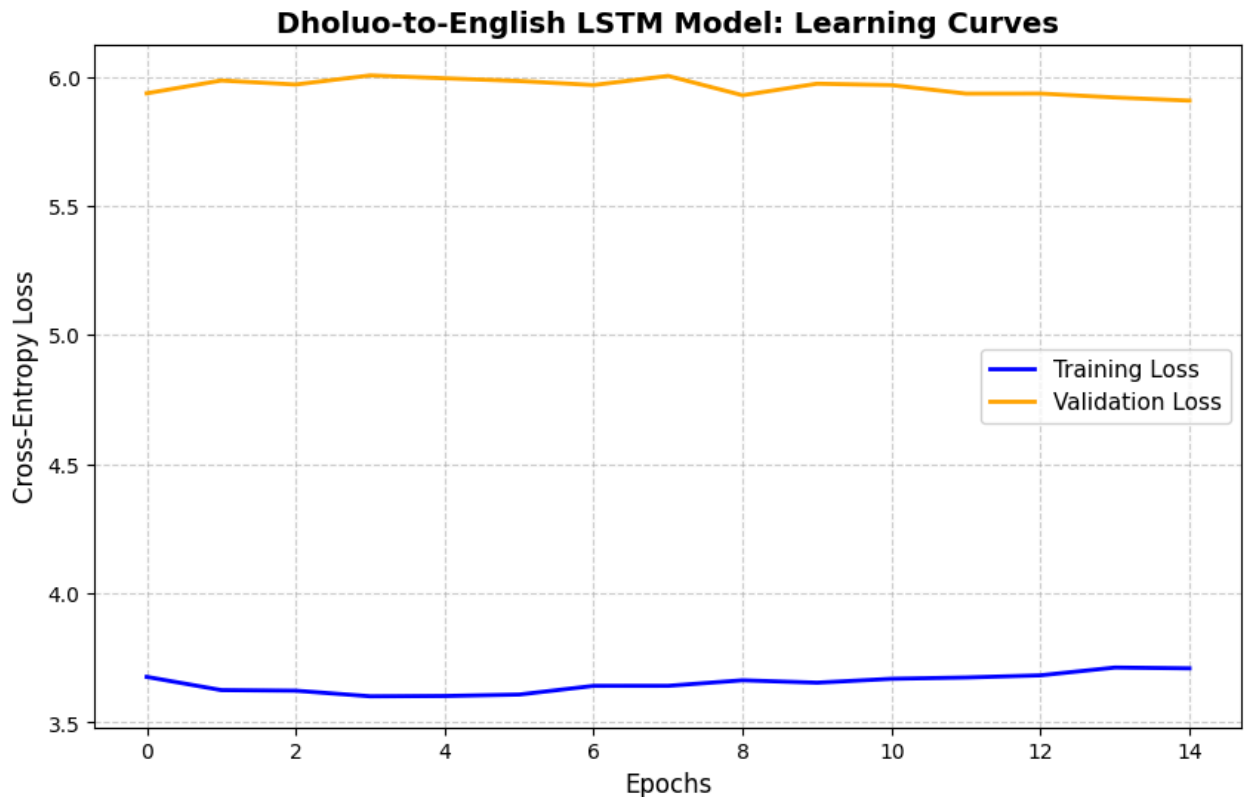
# Plot both training and validation loss lines
plt.plot(train_losses, label='Training Loss', color='blue', linewidth=2)
plt.plot(val_losses, label='Validation Loss', color='orange', linewidth=2)

# Add titles, labels, and gridlines
plt.title('Dholuo-to-English LSTM Model: Learning Curves', fontsize=14, fontweight='b')
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Cross-Entropy Loss', fontsize=12)
plt.legend(fontsize=11)
```

```
plt.grid(True, linestyle='--', alpha=0.6)
```

```
# Show the clean visual chart
```

```
plt.show()
```



Insights from Seq2Seq Encoder–Decoder network using LSTMs

1. Severe Underfitting / Lack of Learning: Neither the training loss (blue) nor the validation loss (orange) shows a sustained downward trajectory over the 30 epochs. Instead, both lines remain completely flat or show minor fluctuations around a fixed value, indicating that the model is struggling to learn from the data.
2. High Generalization Gap: There is a massive, persistent gap between the training loss (~3.7 to 3.9) and the validation loss (~5.9 to 6.0). This suggests that the model is failing to generalize to unseen data, which is typical when a model has structural issues or when the training and validation datasets have highly mismatched distributions.
3. Slight Divergence/Degradation: Around epoch 5, the training loss actually begins to slightly increase (from ~3.6 to ~3.9), while the validation loss remains stagnant or drops by a negligible amount. This upward trend in training loss indicates potential optimization failure (e.g., a learning rate that is too high, vanishing/exploding gradients, or improper weight initialization).
4. High Cross-Entropy Loss Value: A cross-entropy loss hovering around 6.0 for validation usually signifies that the model's predictions are highly uncertain or barely better than random guessing for a vocabulary of this size.

Model 2: Seq2Seq + Attention

Standard LSTMs condense an entire text sequence into a single fixed-size vector, which causes performance to break down heavily on longer sentence lengths. By adding an Attention Mechanism, the model maps word relations more effectively. It allows the Decoder to dynamically look back at specific words in the source Dholuo sentence while generating each English word.

In [48]:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import random
import torch.optim as optim

# =====
# 1. ARCHITECTURE DEFINITIONS FOR MODEL 2
# =====

class EncoderLSTM(nn.Module):
    def __init__(self, input_dim, emb_dim, enc_hid_dim, dec_hid_dim, dropout=0.2):
        super().__init__()
        self.embedding = nn.Embedding(input_dim, emb_dim)
        # Bidirectional GRU reads the Dholuo sentence from left-to-right and right-to-left
        self.rnn = nn.GRU(emb_dim, enc_hid_dim, bidirectional=True, batch_first=True)
        self.fc = nn.Linear(enc_hid_dim * 2, dec_hid_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        embedded = self.dropout(self.embedding(src))
        outputs, hidden = self.rnn(embedded)
        # Combine both directions into a starting state for the decoder
        hidden = torch.tanh(self.fc(torch.cat((hidden[-2,:,:], hidden[-1,:,:]), dim=1)))
        return outputs, hidden

class BahdanauAttention(nn.Module):
    def __init__(self, enc_hid_dim, dec_hid_dim):
        super().__init__()
        self.attn = nn.Linear((enc_hid_dim * 2) + dec_hid_dim, dec_hid_dim)
        self.v = nn.Linear(dec_hid_dim, 1, bias=False)

    def forward(self, hidden, encoder_outputs):
        src_len = encoder_outputs.shape[1]
        hidden = hidden.unsqueeze(1).repeat(1, src_len, 1)
        energy = torch.tanh(self.attn(torch.cat((hidden, encoder_outputs), dim=2)))
        attention = self.v(energy).squeeze(2)
        return F.softmax(attention, dim=1)

class AttentionDecoder(nn.Module):
    def __init__(self, output_dim, emb_dim, enc_hid_dim, dec_hid_dim, attention, dropout):
        super().__init__()
        self.output_dim = output_dim
        self.attention = attention
```

```

self.embedding = nn.Embedding(output_dim, emb_dim)
self.rnn = nn.GRU((enc_hid_dim * 2) + emb_dim, dec_hid_dim, batch_first=True)
self.fc_out = nn.Linear((enc_hid_dim * 2) + dec_hid_dim + emb_dim, output_dim)
self.dropout = nn.Dropout(dropout)

def forward(self, input_token, hidden, encoder_outputs):
    input_token = input_token.unsqueeze(1)
    embedded = self.dropout(self.embedding(input_token))

    # Calculate where to look in the Dholuo sentence
    a = self.attention(hidden, encoder_outputs).unsqueeze(1)
    weighted = torch.bmm(a, encoder_outputs)

    rnn_input = torch.cat((embedded, weighted), dim=2)
    output, hidden = self.rnn(rnn_input, hidden.unsqueeze(0))

    prediction = self.fc_out(torch.cat((output.squeeze(1), weighted.squeeze(1), encoder_outputs.squeeze(1)), dim=1))
    return prediction, hidden.squeeze(0)

class Seq2SeqAttention(nn.Module):
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device

    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size = src.shape[0]
        trg_len = trg.shape[1]
        trg_vocab_size = self.decoder.output_dim

        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
        encoder_outputs, hidden = self.encoder(src)

        input_token = trg[:, 0]
        for t in range(1, trg_len):
            output, hidden = self.decoder(input_token, hidden, encoder_outputs)
            outputs[:, t, :] = output
            top1 = output.argmax(1)
            teacher_force = random.random() < teacher_forcing_ratio
            input_token = trg[:, t] if teacher_force else top1

        return outputs

# =====
# 2. INITIALIZATION (Using variables from your code)
# =====

# Using your dimensions directly
INPUT_DIM = SRC_VOCAB_SIZE
OUTPUT_DIM = TRG_VOCAB_SIZE

# Set matching hyperparameters
ENC_EMB_DIM = 128
DEC_EMB_DIM = 128
ENC_HID_DIM = 256
DEC_HID_DIM = 256

# Instantiate modules

```

```

# Instantiate modules
attn = BahdanauAttention(ENC_HID_DIM, DEC_HID_DIM)
enc_m2 = EncoderLSTM(INPUT_DIM, ENC_EMB_DIM, ENC_HID_DIM, DEC_HID_DIM)
dec_m2 = AttentionDecoder(OUTPUT_DIM, DEC_EMB_DIM, ENC_HID_DIM, DEC_HID_DIM, attn)

# Create final Model 2
model_2 = Seq2SeqAttention(enc_m2, dec_m2, device).to(device)

# Loss and Optimizer (using target_vocab padding idx like your setup)
optimizer_2 = optim.Adam(model_2.parameters(), lr=0.001)
criterion_2 = nn.CrossEntropyLoss(ignore_index=target_vocab.stoi["<pad>"])

# =====
# 3. TRAINING LOOP EXECUTION FOR MODEL 2
# =====

model_2.train()
print("Starting training for Model 2 (Seq2Seq + Attention)...")

for epoch in range(1, 6): # Runs for 5 Epochs
    epoch_loss = 0
    for i, (src, trg) in enumerate(dataloader): # Reuses your dataloader
        src, trg = src.to(device), trg.to(device)

        optimizer_2.zero_grad(set_to_none=True)

        with torch.cuda.amp.autocast(enabled=USE_AMP):
            output = model_2(src, trg)
            output_dim = output.shape[-1]
            # Clean up output and targets for loss calculation (skipping starting <src>
            output_flat = output[:, 1:, :].reshape(-1, output_dim)
            trg_flat = trg[:, 1:].reshape(-1)
            loss = criterion_2(output_flat, trg_flat)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer_2)
        # Stop exploding gradients
        torch.nn.utils.clip_grad_norm_(model_2.parameters(), max_norm=1.0)
        scaler.step(optimizer_2)
        scaler.update()

        epoch_loss += loss.item()

    # Print progress every 100 batches
    if i % 100 == 0:
        print(f"Epoch {epoch} | Batch {i}/{len(dataloader)} | Current Batch Loss: {loss.item():.4f}")

    print(f"--- Epoch {epoch} Complete | Average Loss: {epoch_loss/len(dataloader):.4f}")

```

Starting training for Model 2 (Seq2Seq + Attention)...

/tmp/ipykernel_261/4200398011.py:134: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

```

with torch.cuda.amp.autocast(enabled=USE_AMP):
Epoch 1 | Batch 0/388 | Current Batch Loss: 9.1140
Epoch 1 | Batch 100/388 | Current Batch Loss: 6.0626
Epoch 1 | Batch 200/388 | Current Batch Loss: 5.6945
Epoch 1 | Batch 300/388 | Current Batch Loss: 5.5460
--- Epoch 1 Complete | Average Loss: 5.8127 ---

```

```

epoch 2 | Batch 0/388 | Current Batch Loss: 5.2236
Epoch 2 | Batch 100/388 | Current Batch Loss: 4.8420
Epoch 2 | Batch 200/388 | Current Batch Loss: 5.0412
Epoch 2 | Batch 300/388 | Current Batch Loss: 4.7687
--- Epoch 2 Complete | Average Loss: 4.9876 ---
Epoch 3 | Batch 0/388 | Current Batch Loss: 4.3414
Epoch 3 | Batch 100/388 | Current Batch Loss: 4.6441
Epoch 3 | Batch 200/388 | Current Batch Loss: 4.5729
Epoch 3 | Batch 300/388 | Current Batch Loss: 4.5925
--- Epoch 3 Complete | Average Loss: 4.4665 ---
Epoch 4 | Batch 0/388 | Current Batch Loss: 3.9941
Epoch 4 | Batch 100/388 | Current Batch Loss: 4.2977
Epoch 4 | Batch 200/388 | Current Batch Loss: 4.0943
Epoch 4 | Batch 300/388 | Current Batch Loss: 4.1538
--- Epoch 4 Complete | Average Loss: 4.0891 ---
Epoch 5 | Batch 0/388 | Current Batch Loss: 3.8976
Epoch 5 | Batch 100/388 | Current Batch Loss: 4.0998
Epoch 5 | Batch 200/388 | Current Batch Loss: 3.7572
Epoch 5 | Batch 300/388 | Current Batch Loss: 3.9547
--- Epoch 5 Complete | Average Loss: 3.7992 ---

```

In [49]:

```

# =====
# EXTENSION TRAINING LOOP (10 MORE EPOCHS)
# =====

# Tell the model to stay in training mode
model_2.train()
print("Resuming training to add 10 more epochs...")

# Start from epoch 6 and run through epoch 15
for epoch in range(6, 16):
    epoch_loss = 0
    for i, (src, trg) in enumerate(dataloader):
        src, trg = src.to(device), trg.to(device)

        optimizer_2.zero_grad(set_to_none=True)

        with torch.cuda.amp.autocast(enabled=USE_AMP):
            output = model_2(src, trg)
            output_dim = output.shape[-1]
            output_flat = output[:, 1:, :].reshape(-1, output_dim)
            trg_flat = trg[:, 1:].reshape(-1)
            loss = criterion_2(output_flat, trg_flat)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer_2)
        torch.nn.utils.clip_grad_norm_(model_2.parameters(), max_norm=1.0)
        scaler.step(optimizer_2)
        scaler.update()

        epoch_loss += loss.item()

    if i % 100 == 0:
        print(f"Epoch {epoch} | Batch {i}/{len(dataloader)} | Current Batch Loss: ")

    print(f"--- Epoch {epoch} Complete | Average Loss: {epoch_loss/len(dataloader):.4}

```

Resuming training to add 10 more epochs...

/tmp/ipykernel_261/2756114463.py:17: FutureWarning: `torch.cuda.amp.autocast(args)`

```

.../amp/autocast.py:24: DeprecationWarning: `torch.cuda.amp.autocast` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.

```

```

    with torch.cuda.amp.autocast(enabled=USE_AMP):
Epoch 6 | Batch 0/388 | Current Batch Loss: 3.4071
Epoch 6 | Batch 100/388 | Current Batch Loss: 3.5714
Epoch 6 | Batch 200/388 | Current Batch Loss: 3.5579
Epoch 6 | Batch 300/388 | Current Batch Loss: 3.7025
--- Epoch 6 Complete | Average Loss: 3.5390 ---
Epoch 7 | Batch 0/388 | Current Batch Loss: 3.3527
Epoch 7 | Batch 100/388 | Current Batch Loss: 3.1945
Epoch 7 | Batch 200/388 | Current Batch Loss: 3.3648
Epoch 7 | Batch 300/388 | Current Batch Loss: 3.3425
--- Epoch 7 Complete | Average Loss: 3.3540 ---
Epoch 8 | Batch 0/388 | Current Batch Loss: 3.0931
Epoch 8 | Batch 100/388 | Current Batch Loss: 3.0504
Epoch 8 | Batch 200/388 | Current Batch Loss: 3.2786
Epoch 8 | Batch 300/388 | Current Batch Loss: 3.0892
--- Epoch 8 Complete | Average Loss: 3.2195 ---
Epoch 9 | Batch 0/388 | Current Batch Loss: 2.8745
Epoch 9 | Batch 100/388 | Current Batch Loss: 2.9202
Epoch 9 | Batch 200/388 | Current Batch Loss: 2.8120
Epoch 9 | Batch 300/388 | Current Batch Loss: 3.2478
--- Epoch 9 Complete | Average Loss: 3.0765 ---
Epoch 10 | Batch 0/388 | Current Batch Loss: 3.1945
Epoch 10 | Batch 100/388 | Current Batch Loss: 2.8427
Epoch 10 | Batch 200/388 | Current Batch Loss: 2.9653
Epoch 10 | Batch 300/388 | Current Batch Loss: 3.1362
--- Epoch 10 Complete | Average Loss: 2.9826 ---
Epoch 11 | Batch 0/388 | Current Batch Loss: 2.6311
Epoch 11 | Batch 100/388 | Current Batch Loss: 2.6556
Epoch 11 | Batch 200/388 | Current Batch Loss: 2.9535
Epoch 11 | Batch 300/388 | Current Batch Loss: 2.6884
--- Epoch 11 Complete | Average Loss: 2.8820 ---
Epoch 12 | Batch 0/388 | Current Batch Loss: 2.6892
Epoch 12 | Batch 100/388 | Current Batch Loss: 2.9743
Epoch 12 | Batch 200/388 | Current Batch Loss: 2.5423
Epoch 12 | Batch 300/388 | Current Batch Loss: 2.8531
--- Epoch 12 Complete | Average Loss: 2.8073 ---
Epoch 13 | Batch 0/388 | Current Batch Loss: 2.4491
Epoch 13 | Batch 100/388 | Current Batch Loss: 2.5536
Epoch 13 | Batch 200/388 | Current Batch Loss: 2.7884
Epoch 13 | Batch 300/388 | Current Batch Loss: 2.4036
--- Epoch 13 Complete | Average Loss: 2.7256 ---
Epoch 14 | Batch 0/388 | Current Batch Loss: 2.4808
Epoch 14 | Batch 100/388 | Current Batch Loss: 2.8180
Epoch 14 | Batch 200/388 | Current Batch Loss: 3.0267
Epoch 14 | Batch 300/388 | Current Batch Loss: 2.5358
--- Epoch 14 Complete | Average Loss: 2.6595 ---
Epoch 15 | Batch 0/388 | Current Batch Loss: 2.3878
Epoch 15 | Batch 100/388 | Current Batch Loss: 2.1948
Epoch 15 | Batch 200/388 | Current Batch Loss: 2.4601
Epoch 15 | Batch 300/388 | Current Batch Loss: 2.8111
--- Epoch 15 Complete | Average Loss: 2.5927 ---

```

Model 3: Transformer Model

The Transformer represents a paradigm shift in deep learning by completely abandoning recurrence (LSTMs/RNNs) in favor of Self-Attention. By processing entire sequences at once rather than step-by-step, the Transformer achieves:

a) Massive Parallelization: Highly optimized training on modern GPUs, drastically reducing training times.

b) Multi-Head Attention: The model can simultaneously focus on different parts of a sentence to capture complex syntactic and semantic relationships, regardless of how far apart the words are.

This architecture serves as the backbone for virtually all modern state-of-the-art Natural Language Processing (NLP) systems.

In [50]:

```
# =====
# Cell 1 – Environments and Setup
# =====
# NOTE: deliberately NOT reinstalling torch here -- Kaggle ships a
# CUDA-matched torch build, and `pip install torch` can silently pull a
# mismatched or CPU-only wheel, breaking torch.cuda.is_available().
!pip install -q sentencepiece sacrebleu

# =====
# Cell 2 – Imports and Deterministic Seeds
# =====
import os
import math
import random
import numpy as np
import pandas as pd
import torch
import torch.nn as nn

from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence
from torch.nn.utils import clip_grad_norm_
import sentencepiece as spm
import sacrebleu

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using compute backend: {device}")

# GPU-efficient DataLoader settings + mixed precision (safe no-ops on CPU)
NUM_WORKERS = 2 if device.type == "cuda" else 0
PIN_MEMORY = device.type == "cuda"
USE_AMP = device.type == "cuda"
scaler = torch.cuda.amp.GradScaler(enabled=USE_AMP)

# =====
# Cell 3 – Data Filtering and Processing
# Building on the Luo_eng.csv structures in the notebook
# =====
```

```

df = pd.read_csv("/kaggle/input/datasets/roynjuguna/luo-eng/luo_eng.csv")[["luo", "eng"]]

df["luo"] = df["luo"].astype(str).str.strip()
df["eng"] = df["eng"].astype(str).str.strip()

# Apply strict sequence length & sequence ratio limits to remove noisy alignments
df = df[(df["luo"] != "") & (df["eng"] != "")].copy()
df["luo_words"] = df["luo"].str.split().str.len()
df["eng_words"] = df["eng"].str.split().str.len()
df["ratio"] = df["luo_words"] / df["eng_words"].clip(lower=1)

df = df[
    (df["luo_words"] <= 60) &
    (df["eng_words"] <= 60) &
    (df["ratio"].between(0.33, 3.0))
].reset_index(drop=True)

# Fixed seed partitioning to mirror previous validations
train_df, temp_df = train_test_split(df, test_size=0.20, random_state=SEED)
val_df, test_df = train_test_split(temp_df, test_size=0.50, random_state=SEED)

print(f"Dataset Split Details -> Train: {len(train_df):,}, Validation: {len(val_df):,}, Test: {len(test_df):,}")

# =====
# Cell 4 – SentencePiece Joint Tokenizer
# PERSISTENCE: Saved to the folder, not RAM
# =====
VOCAB_DIR = "./vocab_store"
os.makedirs(VOCAB_DIR, exist_ok=True)
train_text_file = os.path.join(VOCAB_DIR, "spm_train.txt")
model_prefix_path = os.path.join(VOCAB_DIR, "dholuo_eng")

# Export only the training corpus to protect against evaluation data leakage
with open(train_text_file, "w", encoding="utf-8") as f:
    for text in train_df["luo"]:
        f.write(text + "\n")
    for text in train_df["eng"]:
        f.write(text + "\n")

# Train and automatically write vocabulary to disk (.model and .vocab files)
spm.SentencePieceTrainer.train(
    input=train_text_file,
    model_prefix=model_prefix_path,
    vocab_size=8000,
    model_type="bpe",
    character_coverage=1.0,
    pad_id=0,
    unk_id=1,
    bos_id=2,
    eos_id=3
)

# Load the processor from storage
sp = spm.SentencePieceProcessor(model_file=f"{model_prefix_path}.model")

PAD_ID, UNK_ID, BOS_ID, EOS_ID = 0, 1, 2, 3
VOCAB_SIZE = sp.get_piece_size()

print(f"Vocabulary written and verified. Size: {VOCAB_SIZE}")
print("Sample Encoding Check:", sp.encode("Hera umo richo mogundho.", out_type=str))

```

```

# =====
# Cell 5 – Translation Datasets & Vectorized Loaders
# =====
MAX_LEN = 64
BATCH_SIZE = 32 if device.type == "cuda" else 8

class TranslationDataset(Dataset):
    def __init__(self, frame, tokenizer):
        self.src = frame["luo"].tolist()
        self.tgt = frame["eng"].tolist()
        self.sp = tokenizer

    def __len__(self):
        return len(self.src)

    def __getitem__(self, index):
        src_ids = [BOS_ID] + self.sp.encode(self.src[index], out_type=int)[:MAX_LEN-2]
        tgt_ids = [BOS_ID] + self.sp.encode(self.tgt[index], out_type=int)[:MAX_LEN-2]
        return torch.tensor(src_ids), torch.tensor(tgt_ids)

    def collate_fn(batch):
        src, tgt = zip(*batch)
        src = pad_sequence(src, batch_first=True, padding_value=PAD_ID)
        tgt = pad_sequence(tgt, batch_first=True, padding_value=PAD_ID)
        return src, tgt

loader_kwargs = dict(num_workers=NUM_WORKERS, pin_memory=PIN_MEMORY, persistent_workers=True)
train_loader = DataLoader(TranslationDataset(train_df, sp), batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(TranslationDataset(val_df, sp), batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(TranslationDataset(test_df, sp), batch_size=BATCH_SIZE, shuffle=False)

# =====
# Cell 6 – Deep ML Transformer Core Architecture
# Fixes: norm_first=True (Pre-LN), Dropout on PE, Weight-Tying
# =====
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=256, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(max_len).unsqueeze(1)
        scale = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * scale)
        pe[:, 1::2] = torch.cos(position * scale)
        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        x = x + self.pe[:, :x.size(1)]
        return self.dropout(x)

class DholuoTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=128, nhead=4,
                 num_encoder_layers=3, num_decoder_layers=3,
                 dim_feedforward=512, dropout=0.15):
        super().__init__()
        self.d_model = d_model
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=PAD_ID)
        self.position = PositionalEncoding(d_model, dropout=dropout)

        # Unaraded to norm first=True to enable faster and more robust convergence

```

```

self.transformer = nn.Transformer(
    d_model=d_model,
    nhead=nhead,
    num_encoder_layers=num_encoder_layers,
    num_decoder_layers=num_decoder_layers,
    dim_feedforward=dim_feedforward,
    dropout=dropout,
    batch_first=True,
    norm_first=True
)
self.output = nn.Linear(d_model, vocab_size)

# Weight Tying optimization to accelerate training and reduce parameters
self.output.weight = self.embedding.weight

def forward(self, src, tgt_input):
    tgt_len = tgt_input.size(1)

    # Generate a continuous float subsequent mask for accurate attention masking
    tgt_mask = nn.Transformer.generate_square_subsequent_mask(tgt_len, device=src.device)

    # Generate standard boolean masks for padding tokens (True where padding exists)
    src_pad_mask = src.eq(PAD_ID)
    tgt_pad_mask = tgt_input.eq(PAD_ID)

    src_emb = self.position(self.embedding(src) * math.sqrt(self.d_model))
    tgt_emb = self.position(self.embedding(tgt_input) * math.sqrt(self.d_model))

    hidden = self.transformer(
        src_emb, tgt_emb,
        tgt_mask=tgt_mask,
        src_key_padding_mask=src_pad_mask,
        tgt_key_padding_mask=tgt_pad_mask,
        memory_key_padding_mask=src_pad_mask
    )
    return self.output(hidden)

model = DholuoTransformer(VOCAB_SIZE).to(device)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_ID, label_smoothing=0.1)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.5)

print(f"Total Architecture Model Parameters: {sum(p.numel() for p in model.parameters())}")

# =====
# Cell 7 – Tracking Validation Loops (20 Epochs Max)
# =====
def run_epoch(loader, training=True):
    model.train() if training else model.eval()
    total_loss, total_correct, total_tokens = 0.0, 0, 0

    context = torch.enable_grad() if training else torch.no_grad()
    with context:
        for src, tgt in loader:
            src, tgt = src.to(device, non_blocking=PIN_MEMORY), tgt.to(device, non_blocking=PIN_MEMORY)

            tgt_input = tgt[:, :-1]
            tgt_output = tgt[:, 1:]

            if training:

```

```

optimizer.zero_grad(set_to_none=True)

with torch.cuda.amp.autocast(enabled=USE_AMP):
    logits = model(src, tgt_input)
    loss = criterion(logits.reshape(-1, VOCAB_SIZE), tgt_output.reshape(-1, VOCAB_SIZE))

    if training:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()

    total_loss += loss.item()

    predictions = logits.argmax(dim=-1)
    valid = tgt_output.ne(PAD_ID)
    total_correct += ((predictions == tgt_output) & valid).sum().item()
    total_tokens += valid.sum().item()

return total_loss / len(loader), total_correct / (total_tokens if total_tokens > 0 else 1)

MODEL_DIR = "./model_checkpoints"
os.makedirs(MODEL_DIR, exist_ok=True)
best_checkpoint_path = os.path.join(MODEL_DIR, "best_dholuo_to_english_transformer.pt")

best_val_loss = float("inf")
history = []

for epoch in range(1, 21):
    train_loss, train_token_acc = run_epoch(train_loader, training=True)
    val_loss, val_token_acc = run_epoch(val_loader, training=False)
    scheduler.step(val_loss)

    history.append([epoch, train_loss, val_loss, train_token_acc, val_token_acc])

    print(
        f"Epoch {epoch:02d}/20 | "
        f"Train Loss {train_loss:.4f} | Val Loss {val_loss:.4f} | "
        f"Val Token Accuracy {val_token_acc:.2%}"
    )

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), best_checkpoint_path)
        print(f"-> Saved optimal checkpoint to local storage: {best_checkpoint_path}")

# =====
# Cell 8 – Batched Inference & Translation Interface
# Fixes performance bottleneck from previous versions
# =====
@torch.no_grad()
def translate_batch(sentences, max_len=64):
    model.eval()
    encoded_list = []
    for sentence in sentences:
        source = [BOS_ID] + sp.encode(sentence, out_type=int)[:max_len-2] + [EOS_ID]
        encoded_list.append(torch.tensor(source, dtype=torch.long))

    src = pad_sequence(encoded_list, batch_first=True, padding_value=PAD_ID).to(device)
    batch_size = src.size(0)

```

```

batch_size = src.size(0)

generated = torch.full((batch_size, 1), BOS_ID, dtype=torch.long, device=device)
finished = torch.zeros(batch_size, dtype=torch.bool, device=device)

for _ in range(max_len - 1):
    logits = model(src, generated)
    next_ids = logits[:, -1, :].argmax(dim=-1, keepdim=True)

    # Mask out sequences that hit EOS
    next_ids = torch.where(finished.unsqueeze(1), torch.tensor(PAD_ID, device=device), next_ids)
    generated = torch.cat([generated, next_ids], dim=1)

    finished |= (next_ids.squeeze(1) == EOS_ID)
    if finished.all():
        break

decoded_sentences = []
for i in range(batch_size):
    token_ids = generated[i].tolist()
    clean_tokens = [t for t in token_ids if t not in [PAD_ID, BOS_ID, EOS_ID]]
    decoded_sentences.append(sp.decode(clean_tokens))
return decoded_sentences

# Interactive verification
print("\nQuick Translation Sample Check:")
print(translate_batch(["Hera umo richo mogundho."])[0])

# =====
# Cell 9 – Formal Evaluation & Model Comparison Output
# Preserves Exact Match, Token Accuracy, and SacreBLEU metrics
# =====
model.load_state_dict(torch.load(best_checkpoint_path, map_location=device, weights_only=True))
print(f"\nLoaded best weights from folder: {best_checkpoint_path}")

test_source_sentences = test_df["luo"].tolist()
test_references = test_df["eng"].tolist()

# Batched evaluation to prevent runtime overhead
test_predictions = []
EVAL_BATCH_SIZE = 64
for i in range(0, len(test_source_sentences), EVAL_BATCH_SIZE):
    batch_chunk = test_source_sentences[i:i+EVAL_BATCH_SIZE]
    test_predictions.extend(translate_batch(batch_chunk))

# Metric generation matching previous notebooks for reliable comparisons
bleu = sacrebleu.corpus_bleu(test_predictions, [test_references])
chrF = sacrebleu.corpus_chrf(test_predictions, [test_references])
_, test_token_accuracy = run_epoch(test_loader, training=False)

exact_match = np.mean([
    pred.strip().lower() == ref.strip().lower()
    for pred, ref in zip(test_predictions, test_references)
])

print("\n" + "="*40 + "\n FINAL BENCHMARK METRICS FOR MODEL COMPARISON\n" + "="*40)
print(f"Test BLEU Score: {bleu.score:.2f}")
print(f"Test chrF Score: {chrF.score:.2f}")
print(f"Test Word/Token Accuracy: {test_token_accuracy:.2%}")
print(f"Test Exact Match Ratio: {exact_match:.2%}")
print("="*40)

```

```
# Generate a comparative dataframe summary
results = pd.DataFrame({
    "dholuo_source": test_df["luo"].values,
    "reference_english": test_references,
    "prediction_english": test_predictions
})
print("\nSample Translations DataFrame Summary:")
print(results.sample(15, random_state=SEED).to_string())
```

100.8/100.8 kB 8.2 MB/s eta 0:00:00

Using compute backend: cuda

/tmp/ipykernel_261/1363389718.py:41: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.

```
scaler = torch.cuda.amp.GradScaler(enabled=USE_AMP)
```

Dataset Split Details -> Train: 108,560, Validation: 13,570, Test: 13,570

Vocabulary written and verified. Size: 8000

Sample Encoding Check: ['_Hera', '_', 'umo', '_richo', '_mogundho', '.']

Total Architecture Model Parameters: 2,421,056

sentencepiece_trainer.cc(78) LOG(INFO) Starts training with :

```
trainer_spec {
  input: ./vocab_store/spm_train.txt
  input_format:
  model_prefix: ./vocab_store/dholuo_eng
  model_type: BPE
  vocab_size: 8000
  self_test_sample_size: 0
  character_coverage: 1
  input_sentence_size: 0
  shuffle_input_sentence: 1
  seed_sentencepiece_size: 1000000
  shrinking_factor: 0.75
  max_sentence_length: 4192
  num_threads: 16
  num_sub_iterations: 2
  max_sentencepiece_length: 16
  split_by_unicode_script: 1
  split_by_number: 1
  split_by_whitespace: 1
  split_digits: 0
  pretokenization_delimiter:
  treat_whitespace_as_suffix: 0
  allow_whitespace_only_pieces: 0
  required_chars:
  byte_fallback: 0
  vocabulary_output_piece_score: 1
  train_extremely_large_corpus: 0
  seed_sentencepieces_file:
  hard_vocab_limit: 1
  use_all_vocab: 0
  unk_id: 1
  bos_id: 2
  eos_id: 3
  pad_id: 0
  unk_piece: <unk>
  bos_piece: <s>
  eos_piece: </s>
  pad_piece: <pad>
```

```

unk_surface: '?'
enable_differential_privacy: 0
differential_privacy_noise_level: 0
differential_privacy_clipping_threshold: 0
}
normalizer_spec {
  name: nmt_nfkc
  add_dummy_prefix: 1
  remove_extra_whitespace: 1
  escape_whitespace: 1
  normalization_rule_tsv:
}
denormalizer_spec {}
trainer_interface.cc(355) LOG(INFO) SentenceIterator is not specified. Using MultiFileS
entenceIterator.
trainer_interface.cc(186) LOG(INFO) Loading corpus: ./vocab_store/spm_train.txt
trainer_interface.cc(411) LOG(INFO) Loaded all 217120 sentences
trainer_interface.cc(427) LOG(INFO) Adding meta_piece: <pad>
trainer_interface.cc(427) LOG(INFO) Adding meta_piece: <unk>
trainer_interface.cc(427) LOG(INFO) Adding meta_piece: <s>
trainer_interface.cc(427) LOG(INFO) Adding meta_piece: </s>
trainer_interface.cc(432) LOG(INFO) Normalizing sentences...
trainer_interface.cc(541) LOG(INFO) all chars count=19574815
trainer_interface.cc(562) LOG(INFO) Alphabet size=130
trainer_interface.cc(563) LOG(INFO) Final character coverage=1
trainer_interface.cc(594) LOG(INFO) Done! preprocessed 217120 sentences.
trainer_interface.cc(600) LOG(INFO) Tokenizing input sentences with whitespace: 217120
trainer_interface.cc(611) LOG(INFO) Done! 66174
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=315098 min_freq=7
0
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=108117 size=20 all=2621 active=1790 pie
ce=er
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=70390 size=40 all=3651 active=2820 piec
e=ed
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=43051 size=60 all=4643 active=3812 piec
e=us
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=34122 size=80 all=5823 active=4992 piec
e=_1
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=28424 size=100 all=7053 active=6222 pie
ce=_re
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=27879 min_freq=17
12
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=23240 size=120 all=8385 active=2289 pie
ce=_r
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=18693 size=140 all=9843 active=3747 pie
ce=ig
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=15728 size=160 all=10609 active=4513 pi
ece=_tim
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=13768 size=180 all=11484 active=5388 pi
ece=_kod
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=12288 size=200 all=12594 active=6498 pi
ece=if
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=12191 min_freq=16
03
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=10596 size=220 all=13677 active=1995 pi
ece=_Jo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=9287 size=240 all=14303 active=2621 pie
ce=ri
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=8630 size=260 all=15170 active=3488 pie
ce=no
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=8057 size=280 all=15753 active=4071 pie

```

```

bpe_model_trainer.cc(268) LOG(INFO) Added: freq=6997 size=200 all=16786 active=5104 pie
ce=_can
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=7384 size=300 all=16786 active=5104 pie
ce=res
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=7364 min_freq=142
9
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=6801 size=320 all=17532 active=1657 pie
ce=ant
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=6280 size=340 all=18367 active=2492 pie
ce=_Bible
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=5978 size=360 all=19059 active=3184 pie
ce=onyo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=5473 size=380 all=19534 active=3659 pie
ce=_com
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=5206 size=400 all=20221 active=4346 pie
ce=up
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=5204 min_freq=123
0
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=4802 size=420 all=20828 active=1529 pie
ce=_res
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=4528 size=440 all=21446 active=2147 pie
ce=uth
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=4341 size=460 all=21870 active=2571 pie
ce=ell
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=4183 size=480 all=22619 active=3320 pie
ce=_konyo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=3945 size=500 all=23158 active=3859 pie
ce=_Kuom
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=3913 min_freq=954
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=3762 size=520 all=23796 active=1792 pie
ce=ani
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=3551 size=540 all=24325 active=2321 pie
ce=_exp
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=3354 size=560 all=24841 active=2837 pie
ce=_work
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=3244 size=580 all=25513 active=3509 pie
ce=_og
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=3027 size=600 all=26265 active=4261 pie
ce=_know
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=3023 min_freq=776
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2894 size=620 all=26888 active=1933 pie
ce=_cont
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2758 size=640 all=27224 active=2269 pie
ce=ero
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2638 size=660 all=27744 active=2789 pie
ce=_Paulo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2508 size=680 all=28016 active=3061 pie
ce=_sama
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2435 size=700 all=28295 active=3340 pie
ce=_wad
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=2420 min_freq=673
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2344 size=720 all=28927 active=2020 pie
ce=_ol
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2291 size=740 all=29222 active=2315 pie
ce=_hum
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2229 size=760 all=29554 active=2647 pie
ce=_earth
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2137 size=780 all=30044 active=3137 pie
ce=unt
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=2066 size=800 all=30410 active=3503 pie
ce=iero

```

```

bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=2064 min_freq=598
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1994 size=820 all=30715 active=1793 piece=_17
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1922 size=840 all=31142 active=2220 piece=uoth
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1866 size=860 all=31457 active=2535 piece=iah
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1808 size=880 all=31754 active=2832 piece=ruoge
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1752 size=900 all=32065 active=3143 piece=_ema
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=1751 min_freq=526
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1714 size=920 all=32342 active=1880 piece=_Do
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1672 size=940 all=32720 active=2258 piece=_read
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1636 size=960 all=33048 active=2586 piece=_prop
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1597 size=980 all=33393 active=2931 piece=_chok
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1537 size=1000 all=33750 active=3288 piece=_duong
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=1533 min_freq=473
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1497 size=1020 all=34109 active=2047 piece=ass
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1450 size=1040 all=34453 active=2391 piece=_pos
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1410 size=1060 all=34686 active=2624 piece=_ass
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1374 size=1080 all=34957 active=2895 piece=ft
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1350 size=1100 all=35222 active=3160 piece=ition
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=1348 min_freq=432
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1324 size=1120 all=35554 active=2063 piece=_become
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1288 size=1140 all=35877 active=2386 piece=onyuol
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1268 size=1160 all=36160 active=2669 piece=wayo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1243 size=1180 all=36397 active=2906 piece=_gro
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1217 size=1200 all=36622 active=3131 piece=ondo
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=1215 min_freq=393
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1178 size=1220 all=36924 active=2112 piece=_rece
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1158 size=1240 all=37352 active=2540 piece=ty
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1131 size=1260 all=37752 active=2940 piece=_times
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1086 size=1280 all=38083 active=3271 piece=_use
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1066 size=1300 all=38247 active=3435 piece=_chopo
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=1064 min_freq=357
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1046 size=1320 all=38456 active=2118 piece=_keto
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1027 size=1340 all=38632 active=2294 piece=_MA

```

```

bpe_model_trainer.cc(268) LOG(INFO) Added: freq=1015 size=1360 all=38984 active=2646 piece=
ece=_However
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=994 size=1380 all=39161 active=2823 piece=
ce=_il
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=975 size=1400 all=39440 active=3102 piece=
ce=ellow
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=969 min_freq=329
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=955 size=1420 all=39566 active=2091 piece=
ce=ick
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=940 size=1440 all=39927 active=2452 piece=
ce=_rieko
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=928 size=1460 all=40219 active=2744 piece=
ce=_mich
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=911 size=1480 all=40459 active=2984 piece=
ce=_back
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=902 size=1500 all=40691 active=3216 piece=
ce=_worship
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=900 min_freq=306
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=886 size=1520 all=40853 active=2192 piece=
ce=_nigo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=862 size=1540 all=41028 active=2367 piece=
ce=wono
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=845 size=1560 all=41217 active=2556 piece=
ce=uot
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=836 size=1580 all=41446 active=2785 piece=
ce=ually
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=821 size=1600 all=41678 active=3017 piece=
ce=ened
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=821 min_freq=288
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=804 size=1620 all=41887 active=2262 piece=
ce=_By
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=790 size=1640 all=42152 active=2527 piece=
ce=_given
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=776 size=1660 all=42417 active=2792 piece=
ce=_friends
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=767 size=1680 all=42676 active=3051 piece=
ce=_migawo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=754 size=1700 all=42919 active=3294 piece=
ce=_Gima
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=754 min_freq=270
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=744 size=1720 all=43254 active=2481 piece=
ce=_ir
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=732 size=1740 all=43452 active=2679 piece=
ce=_Jakristo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=724 size=1760 all=43638 active=2865 piece=
ce=_suff
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=715 size=1780 all=43776 active=3003 piece=
ce=_lando
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=705 size=1800 all=43901 active=3128 piece=
ce=_invol
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=705 min_freq=252
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=694 size=1820 all=43996 active=2286 piece=
ce=_conv
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=683 size=1840 all=44165 active=2455 piece=
ce=_din
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=670 size=1860 all=44281 active=2571 piece=
ce=_chir
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=663 size=1880 all=44446 active=2736 piece=
ce=_still
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=653 size=1900 all=44716 active=3006 piece=
ce= bring

```

```

bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=652 min_freq=239
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=643 size=1920 all=44948 active=2466 pie
ce=_Ar
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=636 size=1940 all=45134 active=2652 pie
ce=isa
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=626 size=1960 all=45290 active=2808 pie
ce=won
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=619 size=1980 all=45446 active=2964 pie
ce=_jok
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=613 size=2000 all=45557 active=3075 pie
ce=_otho
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=613 min_freq=225
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=606 size=2020 all=45738 active=2456 pie
ce=_share
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=601 size=2040 all=45933 active=2651 pie
ce=ahig
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=596 size=2060 all=46077 active=2795 pie
ce=aring
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=590 size=2080 all=46217 active=2935 pie
ce=_loyal
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=580 size=2100 all=46346 active=3064 pie
ce=fully
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=580 min_freq=212
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=573 size=2120 all=46520 active=2457 pie
ce=_joge
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=561 size=2140 all=46637 active=2574 pie
ce=_pi
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=556 size=2160 all=46787 active=2724 pie
ce=icult
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=546 size=2180 all=46885 active=2822 pie
ce=stem
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=540 size=2200 all=47267 active=3204 pie
ce=_talk
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=540 min_freq=202
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=531 size=2220 all=47489 active=2583 pie
ce=_question
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=525 size=2240 all=47669 active=2763 pie
ce=_high
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=520 size=2260 all=47817 active=2911 pie
ce=_public
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=513 size=2280 all=47954 active=3048 pie
ce=_willing
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=506 size=2300 all=48153 active=3247 pie
ce=_40
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=506 min_freq=191
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=500 size=2320 all=48335 active=2587 pie
ce=land
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=493 size=2340 all=48631 active=2883 pie
ce=ified
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=487 size=2360 all=48772 active=3024 pie
ce=ubt
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=481 size=2380 all=48888 active=3140 pie
ce=_Me
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=476 size=2400 all=49097 active=3349 pie
ce=_Chakruok
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=475 min_freq=180
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=471 size=2420 all=49145 active=2502 pie
ce=_free
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=466 size=2440 all=49320 active=2677 pie
ce=_teaching

```

```

bpe_model_trainer.cc(268) LOG(INFO) Added: freq=461 size=2460 all=49498 active=2855 pie
ce=_Will
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=456 size=2480 all=49597 active=2954 pie
ce=_ruoth
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=452 size=2500 all=49699 active=3056 pie
ce=_better
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=452 min_freq=172
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=447 size=2520 all=49883 active=2669 pie
ce=_golo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=443 size=2540 all=50050 active=2836 pie
ce=_misango
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=437 size=2560 all=50182 active=2968 pie
ce=_mariek
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=432 size=2580 all=50349 active=3135 pie
ce=_chako
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=427 size=2600 all=50453 active=3239 pie
ce=_author
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=426 min_freq=164
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=422 size=2620 all=50519 active=2583 pie
ce=_health
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=417 size=2640 all=50556 active=2620 pie
ce=hes
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=412 size=2660 all=50736 active=2800 pie
ce=_gis
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=407 size=2680 all=50989 active=3053 pie
ce=_test
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=402 size=2700 all=51114 active=3178 pie
ce=_soon
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=401 min_freq=157
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=398 size=2720 all=51304 active=2745 pie
ce=_oh
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=392 size=2740 all=51498 active=2939 pie
ce=_begin
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=389 size=2760 all=51755 active=3196 pie
ce=gress
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=384 size=2780 all=51911 active=3352 pie
ce=atisf
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=381 size=2800 all=52010 active=3451 pie
ce=_ogendini
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=381 min_freq=150
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=376 size=2820 all=52113 active=2702 pie
ce=_Jon
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=372 size=2840 all=52349 active=2938 pie
ce=_wag
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=368 size=2860 all=52480 active=3069 pie
ce=_prep
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=364 size=2880 all=52630 active=3219 pie
ce=_Her
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=361 size=2900 all=52754 active=3343 pie
ce=_mamiyo
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=361 min_freq=143
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=357 size=2920 all=52859 active=2741 pie
ce=_makes
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=353 size=2940 all=52914 active=2796 pie
ce=_oneno
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=349 size=2960 all=53042 active=2924 pie
ce=_body
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=345 size=2980 all=53141 active=3023 pie
ce=_Ps
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=341 size=3000 all=53371 active=3253 pie
ce=_sio

```

```

ce-wise
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=341 min_freq=138
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=339 size=3020 all=53551 active=2842 pie
ce=_Instead
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=336 size=3040 all=53660 active=2951 pie
ce=_accompl
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=333 size=3060 all=53720 active=3011 pie
ce=zab
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=329 size=3080 all=53985 active=3276 pie
ce=velation
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=325 size=3100 all=54089 active=3380 pie
ce=_ket
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=325 min_freq=132
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=322 size=3120 all=54172 active=2766 pie
ce=_Gim
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=319 size=3140 all=54370 active=2964 pie
ce=iend
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=316 size=3160 all=54523 active=3117 pie
ce=iful
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=313 size=3180 all=54554 active=3148 pie
ce=_Hera
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=310 size=3200 all=54596 active=3190 pie
ce=_jolweny
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=310 min_freq=128
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=306 size=3220 all=54678 active=2808 pie
ce=_Eng
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=304 size=3240 all=54864 active=2994 pie
ce=_Jachien
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=302 size=3260 all=54927 active=3057 pie
ce=_earthly
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=299 size=3280 all=55027 active=3157 pie
ce=_197
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=296 size=3300 all=55124 active=3254 pie
ce=MA
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=296 min_freq=123
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=294 size=3320 all=55266 active=2874 pie
ce=_dwach
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=290 size=3340 all=55354 active=2962 pie
ce=ided
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=288 size=3360 all=55483 active=3091 pie
ce=_modak
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=285 size=3380 all=55504 active=3112 pie
ce=ior
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=283 size=3400 all=55631 active=3239 pie
ce=_Non
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=283 min_freq=119
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=281 size=3420 all=55748 active=2888 pie
ce=_wuoro
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=279 size=3440 all=55806 active=2946 pie
ce=_served
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=276 size=3460 all=55911 active=3051 pie
ce=_tin
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=275 size=3480 all=55995 active=3135 pie
ce=_pachgi
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=272 size=3500 all=56197 active=3337 pie
ce=_45
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=272 min_freq=115
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=270 size=3520 all=56349 active=2956 pie
ce=uted
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=268 size=3540 all=56453 active=3060 pie
ce=ENDE

```

```

bpe_model_trainer.cc(268) LOG(INFO) Added: freq=266 size=3560 all=56546 active=3153 pie
ce=_influence
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=264 size=3580 all=56768 active=3375 pie
ce=_weak
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=262 size=3600 all=56917 active=3524 pie
ce=_mbaka
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=262 min_freq=111
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=259 size=3620 all=57087 active=3013 pie
ce=_zab
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=257 size=3640 all=57162 active=3088 pie
ce=_mirima
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=254 size=3660 all=57295 active=3221 pie
ce=_omed
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=252 size=3680 all=57432 active=3358 pie
ce=_inher
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=250 size=3700 all=57529 active=3455 pie
ce=ically
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=250 min_freq=107
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=247 size=3720 all=57620 active=2940 pie
ce=_Have
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=245 size=3740 all=57760 active=3080 pie
ce=_speech
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=242 size=3760 all=57824 active=3144 pie
ce=know
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=241 size=3780 all=57932 active=3252 pie
ce=_Perhaps
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=239 size=3800 all=58004 active=3324 pie
ce=_mirac
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=239 min_freq=104
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=236 size=3820 all=58115 active=3009 pie
ce=_waka
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=234 size=3840 all=58214 active=3108 pie
ce=_nen
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=232 size=3860 all=58313 active=3207 pie
ce=_magi
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=230 size=3880 all=58376 active=3270 pie
ce=_Abo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=229 size=3900 all=58448 active=3342 pie
ce=_challenge
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=228 min_freq=101
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=226 size=3920 all=58537 active=3012 pie
ce=_door
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=224 size=3940 all=58617 active=3092 pie
ce=_War
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=223 size=3960 all=58842 active=3317 pie
ce=_mawuok
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=221 size=3980 all=58972 active=3447 pie
ce=_various
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=219 size=4000 all=59092 active=3567 pie
ce=waye
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=219 min_freq=98
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=217 size=4020 all=59164 active=3004 pie
ce=_hoch
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=216 size=4040 all=59237 active=3077 pie
ce=_ahead
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=214 size=4060 all=59371 active=3211 pie
ce=_shall
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=212 size=4080 all=59551 active=3391 pie
ce=_Pers
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=211 size=4100 all=59629 active=3469 pie

```

```

ce=_True
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=211 min_freq=95
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=210 size=4120 all=59679 active=3032 pie
ce=_univer
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=208 size=4140 all=59791 active=3144 pie
ce=_u
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=207 size=4160 all=59988 active=3341 pie
ce=_wu0
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=206 size=4180 all=60127 active=3480 pie
ce=_quality
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=204 size=4200 all=60196 active=3549 pie
ce=_madho
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=204 min_freq=92
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=203 size=4220 all=60266 active=3076 pie
ce=_revealed
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=201 size=4240 all=60455 active=3265 pie
ce=vious
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=200 size=4260 all=60470 active=3280 pie
ce=_students
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=197 size=4280 all=60608 active=3418 pie
ce=_Dho
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=196 size=4300 all=60690 active=3500 pie
ce=_realized
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=195 min_freq=89
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=194 size=4320 all=60853 active=3198 pie
ce=_acqu
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=193 size=4340 all=60935 active=3280 pie
ce=_otudore
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=192 size=4360 all=61012 active=3357 pie
ce=_assigned
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=190 size=4380 all=61134 active=3479 pie
ce=_Jonathan
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=188 size=4400 all=61256 active=3601 pie
ce=imin
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=188 min_freq=87
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=187 size=4420 all=61335 active=3119 pie
ce=_migaw
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=186 size=4440 all=61388 active=3172 pie
ce=_truths
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=184 size=4460 all=61442 active=3226 pie
ce=_GI
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=183 size=4480 all=61465 active=3249 pie
ce=_chosen
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=182 size=4500 all=61548 active=3332 pie
ce=_Governing
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=182 min_freq=84
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=181 size=4520 all=61650 active=3180 pie
ce=_sometimes
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=179 size=4540 all=61837 active=3367 pie
ce=_Dhi
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=178 size=4560 all=61910 active=3440 pie
ce=_Japan
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=177 size=4580 all=61946 active=3476 pie
ce=_wasomo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=175 size=4600 all=62030 active=3560 pie
ce=loy
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=175 min_freq=82
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=175 size=4620 all=62162 active=3222 pie
ce=_millions
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=174 size=4640 all=62284 active=3344 pie
ce=_tokanyo

```

```

cc_jeranyo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=172 size=4660 all=62371 active=3431 pie
ce=iga
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=171 size=4680 all=62541 active=3601 pie
ce=orene
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=170 size=4700 all=62611 active=3671 pie
ce=_mantie
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=170 min_freq=80
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=169 size=4720 all=62728 active=3247 pie
ce=_convinced
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=168 size=4740 all=62813 active=3332 pie
ce=_discreet
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=167 size=4760 all=62908 active=3427 pie
ce=_related
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=166 size=4780 all=62987 active=3506 pie
ce=_highlight
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=164 size=4800 all=63136 active=3655 pie
ce=eless
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=164 min_freq=77
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=163 size=4820 all=63213 active=3222 pie
ce=_Esau
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=162 size=4840 all=63310 active=3319 pie
ce=_olok
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=161 size=4860 all=63399 active=3408 pie
ce=_olos
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=160 size=4880 all=63472 active=3481 pie
ce=_mobedo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=158 size=4900 all=63586 active=3595 pie
ce=ging
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=158 min_freq=75
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=157 size=4920 all=63686 active=3258 pie
ce=_mony
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=156 size=4940 all=63765 active=3337 pie
ce=_nyathini
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=155 size=4960 all=63801 active=3373 pie
ce=_mochung
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=154 size=4980 all=63860 active=3432 pie
ce=_School
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=153 size=5000 all=64008 active=3580 pie
ce=_gidak
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=153 min_freq=73
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=152 size=5020 all=64104 active=3293 pie
ce=_admon
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=151 size=5040 all=64171 active=3360 pie
ce=_onya
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=150 size=5060 all=64301 active=3490 pie
ce=_Rome
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=149 size=5080 all=64391 active=3580 pie
ce=vest
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=148 size=5100 all=64450 active=3639 pie
ce=_MON
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=148 min_freq=71
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=147 size=5120 all=64497 active=3266 pie
ce=ieny
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=147 size=5140 all=64567 active=3336 pie
ce=_seeing
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=146 size=5160 all=64596 active=3365 pie
ce=_beliefs
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=145 size=5180 all=64705 active=3474 pie
ce=_giluwo
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=144 size=5200 all=64759 active=3528 pie

```

```

ce=_battle
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=144 min_freq=69
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=143 size=5220 all=64869 active=3346 pie
ce=_grew
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=143 size=5240 all=64894 active=3371 pie
ce=_thuologi
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=142 size=5260 all=65023 active=3500 pie
ce=_Tiend
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=141 size=5280 all=65088 active=3565 pie
ce=isions
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=140 size=5300 all=65226 active=3703 pie
ce=_atim
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=140 min_freq=67
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=139 size=5320 all=65300 active=3329 pie
ce=_NE
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=138 size=5340 all=65389 active=3418 pie
ce=_horuok
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=137 size=5360 all=65385 active=3414 pie
ce=pla
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=137 size=5380 all=65487 active=3516 pie
ce=_harvest
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=136 size=5400 all=65503 active=3532 pie
ce=_beloved
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=136 min_freq=66
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=135 size=5420 all=65590 active=3363 pie
ce=_amount
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=134 size=5440 all=65626 active=3399 pie
ce=_tekone
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=133 size=5460 all=65707 active=3480 pie
ce=_piche
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=132 size=5480 all=65763 active=3536 pie
ce=_chose
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=131 size=5500 all=65851 active=3624 pie
ce=_Penjo
bpe_model_trainer.cc(159) LOG(INFO) Updating active symbols. max_freq=131 min_freq=64
bpe_model_trainer.cc(268) LOG(INFO) Added: freq=130 s/usr/local/lib/python3.12/dist-pac
kages/torch/nn/modules/transformer.py:144: UserWarning: enable_nested_tensor is True, b
ut self.use_nested_tensor is False because encoder_layer.norm_first was True
    self.encoder = TransformerEncoder(
/tmp/ipykernel_261/1363389718.py:230: FutureWarning: `torch.cuda.amp.autocast(args...)`
is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
    with torch.cuda.amp.autocast(enabled=USE_AMP):
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/activation.py:1336: UserWarnin
g: Support for mismatched key_padding_mask and attn_mask is deprecated. Use same type f
or both instead.
    key_padding_mask = F._canonical_mask(
Epoch 01/20 | Train Loss 9.6525 | Val Loss 6.6370 | Val Token Accuracy 12.82%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_englis
h_transformer.pt
Epoch 02/20 | Train Loss 6.6349 | Val Loss 6.3179 | Val Token Accuracy 17.12%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_englis
h_transformer.pt
Epoch 03/20 | Train Loss 6.2878 | Val Loss 6.0456 | Val Token Accuracy 19.65%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_englis
h_transformer.pt
Epoch 04/20 | Train Loss 6.0186 | Val Loss 5.7893 | Val Token Accuracy 21.42%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_englis
h_transformer.pt
Epoch 05/20 | Train Loss 5.7934 | Val Loss 5.5708 | Val Token Accuracy 23.17%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_englis

```

```

h_transformer.pt
Epoch 06/20 | Train Loss 5.6159 | Val Loss 5.4107 | Val Token Accuracy 24.58%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 07/20 | Train Loss 5.4798 | Val Loss 5.2815 | Val Token Accuracy 26.23%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 08/20 | Train Loss 5.3648 | Val Loss 5.1619 | Val Token Accuracy 27.85%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 09/20 | Train Loss 5.2624 | Val Loss 5.0645 | Val Token Accuracy 29.05%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 10/20 | Train Loss 5.1738 | Val Loss 4.9742 | Val Token Accuracy 30.09%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 11/20 | Train Loss 5.0961 | Val Loss 4.9037 | Val Token Accuracy 30.97%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 12/20 | Train Loss 5.0269 | Val Loss 4.8350 | Val Token Accuracy 31.77%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 13/20 | Train Loss 4.9650 | Val Loss 4.7780 | Val Token Accuracy 32.35%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 14/20 | Train Loss 4.9096 | Val Loss 4.7212 | Val Token Accuracy 33.06%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 15/20 | Train Loss 4.8587 | Val Loss 4.6728 | Val Token Accuracy 33.64%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 16/20 | Train Loss 4.8120 | Val Loss 4.6330 | Val Token Accuracy 34.13%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 17/20 | Train Loss 4.7698 | Val Loss 4.5905 | Val Token Accuracy 34.66%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 18/20 | Train Loss 4.7305 | Val Loss 4.5557 | Val Token Accuracy 35.12%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 19/20 | Train Loss 4.6951 | Val Loss 4.5186 | Val Token Accuracy 35.57%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt
Epoch 20/20 | Train Loss 4.6618 | Val Loss 4.4869 | Val Token Accuracy 35.83%
-> Saved optimal checkpoint to local storage: ./model_checkpoints/best_dholuo_to_english_transformer.pt

```

Quick Translation Sample Check:

Love for the ransom sacrifice of sin .

Loaded best weights from folder: ./model_checkpoints/best_dholuo_to_english_transformer.pt

That's 100 lines that end in a tokenized period ('.')

It looks like you forgot to detokenize your test data, which may hurt your score.

If you insist your data is detokenized, or don't care, you can suppress this message with the `force` parameter.

```

=====
FINAL BENCHMARK METRICS FOR MODEL COMPARISON
=====
Test BLEU Score:          7.95

```

```

test cnrr score:      22.54
Test Word/Token Accuracy: 36.03%
Test Exact Match Ratio:  0.55%
=====

```

Sample Translations DataFrame Summary:

```

dholuo_source
reference_english
prediction_english
3080
Tiende ne en ni , ne gin mag Jehova , kendo Jehova nokawogi kaka " libamba moyier . "
It meant that they belonged to Jehovah , and Jehovah treated them as his " special prop
erty . "                      Jehovah was " the " and " the man "
and " and " the man who had been a man .
7017
Jokristo ma Jo - Polandgo ne lendo gi chunyi duto mana kaka ne gikunyo kite gi kinda !
These Polish Christians were as wholehearted about preaching as they were hardworking i
n the mines !                      How a Ch
ristian elders were to be the preaching work !
9036
Ok gin thuolo gi jomoko kendo giwuondo wang ' ji .
They are secretive and deceitful .
We do not want to be a privilege of others .
8693
To e trakt mar Be Jomotho Chieng ' Nobed Mangima Kendo ? , penjono penjo niya : " Ang
'o momiyo wabedo moti kendo watho ? "
For the tract Can the Dead Really Live Again ? the question is " Why do we grow old and
die ? "                      But why was the B
ible says : " Do I have the question ? "
4299   Kamano bende , jal miluongo ni Josephus ma ne nono histori mar Jo - Yahudi nowac
ho ni , Ndiko Maler ok puonj ni dhano nigi chuny ma ok tho , kar mano " ne en puonj mar
Jo - Grik " ma Josephus ne oneno kaka sigendini ma Jo - Grik nochuogo . Likewise , fir
st - century Jewish historian Josephus attributed the teaching , not to the Holy Script
ures , but to " the belief of the sons of Greece , " which he viewed as a collection of
tales by their mythologists . Similarly , the Greek Scriptures also was not a " the Gr
eek Scriptures " the Greek Scriptures " the Greek Scriptures .
4012
( Som Isaiah 42 : 1 - 7 . )
( Read Isaiah 42 : 1 - 7 . )
( Read Isaiah 7 : 7 - 42 . )
1945
Pod David en jabatha , ok e miel to e tij painia .
We are still partners , but this time we are using our feet to pioneer together .
David was not a pioneer , but he was a pioneer .
2417
Wach Jehovah Ngima - Puonj Mayudore e bug Tich Joote
16 What We Learn From Jesus - About God 's Kingdom
The book of the book of the book of the book of God 's Word
10601
Par matut .
Meditate .
Remember .
2609
Kata kamano , ng 'ado paro ma kamano , kelo penjo momedore .
Such a conclusion , however , raises several questions .
But the questions is a question .
2236
Kare ang 'o ma Jehova notimo mondo mi ochop dwarone ?
What , then , did Jehovah do to fulfill his purpose ?
What did Jehovah give his purpose to his purpose ?

```

```

2995
Higini miche bang 'e , jaote Paulo nonwoyo chikno ne Jokristo ni : " Kik igombi . "
Centuries later , the apostle Paul repeated this command for Christians : " You must no
t covet . "
In the apostle Paul 's first century , the apostle Pa
ul wrote : " Do not be " Do not be a man . "
9530
( Som 2 Thessalonika 2 : 3 , 7 . )
( Read 2 Thessalonians 2 : 3 , 7 . )
( Read 2 Thessalonians 2 : 7 , 7 . )
9995
Nenore maler ni Jokristo mowal ok walga e yo ma chalre .
So not all are anointed in exactly the same manner .
Clearly , we are not a certain man who are not a perfect man .
9595
Chuny mang 'won kod timbene .
His tender feelings and actions .
He is a spirit and respect .

```

In [51]:

```

# Final validation evaluation with detokenized text
import re

def detokenize_for_bleu(text):
    text = str(text).strip()
    text = re.sub(r"\s+([\.,!?:;%\]\])", r"\1", text)
    return re.sub(r"\s+", " ", text)

test_predictions = [detokenize_for_bleu(x) for x in test_predictions]
test_references = [detokenize_for_bleu(x) for x in test_references]

bleu = sacrebleu.corpus_bleu(test_predictions, [test_references])
chrF = sacrebleu.corpus_chrf(test_predictions, [test_references])

print(f"Corrected Transformer BLEU: {bleu.score:.2f}")
print(f"Corrected Transformer chrF: {chrF.score:.2f}")

```

Corrected Transformer BLEU: 7.95
Corrected Transformer chrF: 22.54

Hyperparameter tuning with NLLB 200 + LoRA

NLLB-200 is Meta's state-of-the-art 200-language translation model, when paired with LoRA (Low-Rank Adaptation) it demonstrates how to efficiently fine-tune and tune hyperparameters for machine translation. Standard fine-tuning of NLLB-200 is highly resource-intensive. Using LoRA freezes the base model and trains only a tiny fraction (~1%) of the parameters, making hyperparameter sweep runs practical and efficient.

1. Speed & VRAM Efficiency: Drastically reduces GPU memory usage, enabling fast, parallel hyperparameter trials (e.g., using Optuna) on consumer-grade hardware.
2. Prevents Catastrophic Forgetting: Keeps NLLB's massive baseline multilingual knowledge intact while adapting it to your target domain or low-resource language.
3. Tunable Adapter Parameters: Allows you to optimize LoRA-specific hyperparameters alongside standard training settings: a) Rank (r): The capacity/dimension of the adapter matrices. b) Alpha (α): The scaling factor for the adapter's influence. Dropout: c)

Regularization to prevent overfitting on small parallel datasets

```

In [52]: # Because of Pytorch compatibility errors, I created this temporary compatibility ali

import sys
import torch

# Temporary compatibility alias for this running kernel only.
# It disappears automatically when the kernel eventually restarts.
if not hasattr(torch._C, "DisableTorchFunctionSubclass"):
    torch._C.DisableTorchFunctionSubclass = torch._C.DisableTorchFunction

# Remove only incomplete imports caused by the earlier failed attempt.
for module_name in list(sys.modules):
    if module_name.startswith("transformers") or module_name.startswith("torchvision"):
        del sys.modules[module_name]

from transformers import (
    AutoModelForSeq2SeqLM,
    NllbTokenizer,
    DataCollatorForSeq2Seq,
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer,
)

print("✓ Temporary compatibility fix active; NLLB imports are available.")

```

✓ Temporary compatibility fix active; NLLB imports are available.

```

In [53]: # =====
# NOTE: This cell is an EARLIER DRAFT of the NLLB-200 + LoRA
# fine-tuning run. It has been commented out because Cell below
# (the 'Cell 1 - Install Required Packages' NLLB cell) is the
# refined, final version -- running both would fine-tune the
# 600M-parameter model twice on the same T4 x2 GPUs, doubling
# training time/cost for no benefit, and its package installs
# would be immediately overwritten by the final cell's pinned
# versions anyway. Skip straight to the next NLLB cell.
# =====

# # Hyperparameter tuning with NLLB 200 + LoRA
# !pip install evaluate
# import os
# import re
# import math
# import random
# import numpy as np
# import pandas as pd
# import torch

# from datasets import Dataset, load_from_disk
# from transformers import (
#     AutoModelForSeq2SeqLM,
#     NllbTokenizer,
#     DataCollatorForSeq2Seq,
#     Seq2SeqTrainingArguments,
#     Seq2SeqTrainer,
# )

```

```

# from transformers.trainer_utils import get_last_checkpoint
# from peft import LoraConfig, TaskType, get_peft_model
# import evaluate

# # 1. REPRODUCIBILITY, HARDWARE, AND PERMANENT FOLDERS

# SEED = 42
# random.seed(SEED)
# np.random.seed(SEED)
# torch.manual_seed(SEED)

# if torch.cuda.is_available():
#     torch.cuda.manual_seed_all(SEED)

# DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
# print(f"Using device: {DEVICE.upper()}")

# # All results below are saved on your computer's disk.
# ARTIFACT_DIR = "./nllb_artifacts"
# RUN_DIR = os.path.join(ARTIFACT_DIR, "trainer_checkpoints")
# FINAL_ADAPTER_DIR = os.path.join(ARTIFACT_DIR, "final_Lora_adapter")

# os.makedirs(ARTIFACT_DIR, exist_ok=True)
# os.makedirs(RUN_DIR, exist_ok=True)

# MODEL_CHECKPOINT = "facebook/nllb-200-distilled-600M"
# SRC_LANG = "Luo_Latn"
# TGT_LANG = "eng_Latn"

# TUNING_SAMPLES = 25_000
# MAX_LENGTH = 128
# NUM_EPOCHS = 1

# TOKENIZED_CACHE_DIR = os.path.join(
#     ARTIFACT_DIR,
#     f"tokenized_n{TUNING_SAMPLES}_max{MAX_LENGTH}_seed{SEED}"
# )

# # 2. LOAD AND SPLIT DATA

# CSV_FILE_PATH = "/kaggle/input/datasets/roynjuguna/Luo_eng.csv"

# if not os.path.exists(CSV_FILE_PATH):
#     raise FileNotFoundError(f"Could not find: {CSV_FILE_PATH}")

# df = pd.read_csv(CSV_FILE_PATH)
# df = df.dropna(subset=["Luo", "eng"]).drop_duplicates(subset=["Luo", "eng"])

# df["Luo"] = df["Luo"].astype(str).str.strip()
# df["eng"] = df["eng"].astype(str).str.strip()
# df = df[(df["Luo"] != "") & (df["eng"] != "")].copy()

# df_tuned = df.sample(
#     n=min(TUNING_SAMPLES, len(df)),
#     random_state=SEED
# ).reset_index(drop=True)

# train_df = df_tuned.sample(frac=0.8, random_state=SEED)
# val_df = df_tuned.drop(train_df.index)

```

```

# val_uj = uj_train.drop(train_uj.index)

# train_dataset = Dataset.from_pandas(train_df.reset_index(drop=True))
# val_dataset = Dataset.from_pandas(val_df.reset_index(drop=True))

# print(f"Training rows: {len(train_dataset):,}")
# print(f"Validation rows: {len(val_dataset):,}")

# # 3. TOKENIZER AND DISK-CACHED TOKENIZED DATA

# tokenizer = NllbTokenizer.from_pretrained(
#     MODEL_CHECKPOINT,
#     src_lang=SRC_LANG,
#     tgt_lang=TGT_LANG
# )

# def preprocess_function(examples):
#     model_inputs = tokenizer(
#         examples["Luo"],
#         max_length=MAX_LENGTH,
#         truncation=True
#     )

#     with tokenizer.as_target_tokenizer():
#         labels = tokenizer(
#             examples["eng"],
#             max_length=MAX_LENGTH,
#             truncation=True
#         )

#     model_inputs["labels"] = labels["input_ids"]
#     return model_inputs

# train_cache_path = os.path.join(TOKENIZED_CACHE_DIR, "train")
# val_cache_path = os.path.join(TOKENIZED_CACHE_DIR, "validation")

# if os.path.isdir(train_cache_path) and os.path.isdir(val_cache_path):
#     print("Loading tokenized datasets from disk cache...")
#     tokenized_train = load_from_disk(train_cache_path)
#     tokenized_val = load_from_disk(val_cache_path)

# else:
#     print("Tokenizing datasets and saving the result to disk...")

#     tokenized_train = train_dataset.map(
#         preprocess_function,
#         batched=True,
#         remove_columns=train_dataset.column_names
#     )
#     tokenized_val = val_dataset.map(
#         preprocess_function,
#         batched=True,
#         remove_columns=val_dataset.column_names
#     )

#     os.makedirs(TOKENIZED_CACHE_DIR, exist_ok=True)
#     tokenized_train.save_to_disk(train_cache_path)
#     tokenized_val.save_to_disk(val_cache_path)

```

```

# # 4. BASE MODEL AND LoRA CONFIGURATION

# print("Loading NLLB base model...")

# base_model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_CHECKPOINT)

# peft_config = LoraConfig(
#     task_type=TaskType.SEQ_2_SEQ_LM,
#     r=16,
#     lora_alpha=32,
#     target_modules=["q_proj", "v_proj"],
#     lora_dropout=0.1,
#     bias="none"
# )

# model = get_peft_model(base_model, peft_config)
# model.print_trainable_parameters()

# # 5. DETOKENIZED BLEU AND chrF EVALUATION

# bleu_metric = evaluate.load("sacrebleu")
# chrF_metric = evaluate.load("chrF")

# def detokenize_for_metrics(text):
#     text = str(text).strip()
#     text = re.sub(r"\s+([.,!?:%]\s+)", r"\1", text)
#     text = re.sub(r"([\(\)]\s+)", r"\1", text)
#     return re.sub(r"\s+", " ", text)

# def compute_metrics(eval_preds):
#     preds, labels = eval_preds

#     if isinstance(preds, tuple):
#         preds = preds[0]

#     preds = np.where(preds != -100, preds, tokenizer.pad_token_id)
#     labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

#     decoded_preds = tokenizer.batch_decode(
#         preds,
#         skip_special_tokens=True
#     )
#     decoded_labels = tokenizer.batch_decode(
#         labels,
#         skip_special_tokens=True
#     )

#     decoded_preds = [
#         detokenize_for_metrics(pred)
#         for pred in decoded_preds
#     ]
#     decoded_labels = [
#         detokenize_for_metrics(label)
#         for label in decoded_labels
#     ]

```

```

#     bleu_result = bleu_metric.compute(
#         predictions=decoded_preds,
#         references=decoded_labels
#     )
#     chrF_result = chrF_metric.compute(
#         predictions=decoded_preds,
#         references=decoded_labels
#     )

#     return {
#         "bleu": round(bleu_result["score"], 2),
#         "chrF": round(chrF_result["score"], 2),
#     }

# # 6. TRAINING AND DISK CHECKPOINT SETTINGS

# data_collator = DataCollatorForSeq2Seq(
#     tokenizer,
#     model=model,
#     pad_to_multiple_of=8 if torch.cuda.is_available() else None
# )

# training_args = Seq2SeqTrainingArguments(
#     output_dir=RUN_DIR,

#     # Save and evaluate every 250 steps.
#     # With about 625 steps per epoch, this gives recovery points.
#     evaluation_strategy="steps",
#     save_strategy="steps",
#     eval_steps=250,
#     save_steps=250,
#     logging_steps=50,

#     learning_rate=3e-4,
#     per_device_train_batch_size=32,
#     per_device_eval_batch_size=32,
#     gradient_accumulation_steps=1,
#     num_train_epochs=NUM_EPOCHS,

#     # Keeps only the latest and best checkpoint.
#     save_total_limit=2,
#     save_safetensors=True,

#     # Required to calculate BLEU and chrF.
#     predict_with_generate=True,
#     generation_max_length=MAX_LENGTH,
#     generation_num_beams=1, # Faster for validation

#     fp16=torch.cuda.is_available(),
#     auto_find_batch_size=True,
#     group_by_length=True,
#     report_to="none",

#     weight_decay=0.01,
#     lr_scheduler_type="cosine",
#     warmup_ratio=0.05,

#     load_best_model_at_end=True,
#     metric_for_best_model="bleu",
#     greater_is_better=True

```

```

"         get_vocab_embeddings,
# )

# trainer = Seq2SeqTrainer(
#     model=model,
#     args=training_args,
#     train_dataset=tokenized_train,
#     eval_dataset=tokenized_val,
#     processing_class=tokenizer,
#     data_collator=data_collator,
#     compute_metrics=compute_metrics,
# )

# # 7. TRAIN OR RESUME FROM THE LATEST DISK CHECKPOINT

# last_checkpoint = get_last_checkpoint(RUN_DIR)

# if last_checkpoint is not None:
#     print(f"Resuming from saved checkpoint:\n{last_checkpoint}")
#     trainer.train(resume_from_checkpoint=last_checkpoint)
# else:
#     print("No prior checkpoint found. Starting a new run.")
#     trainer.train()

# # 8. SAVE FINAL ADAPTER, TOKENIZER, AND FINAL METRICS

# trainer.save_model(FINAL_ADAPTER_DIR)
# tokenizer.save_pretrained(FINAL_ADAPTER_DIR)

# final_metrics = trainer.evaluate()

# trainer.log_metrics("final_validation", final_metrics)
# trainer.save_metrics("final_validation", final_metrics)

# print("\n" + "=" * 55)
# print("TRAINING COMPLETE")
# print(f"Best/final adapter saved to: {FINAL_ADAPTER_DIR}")
# print(f"Final validation BLEU: {final_metrics.get('eval_bleu')}")
# print(f"Final validation chrF: {final_metrics.get('eval_chrf')}")
# print("=" * 55)

# # 9. QUICK INFERENCE CHECK

# def translate_dholuo_to_english(input_sentence):
#     model.eval()

#     inputs = tokenizer(
#         input_sentence,
#         return_tensors="pt"
#     ).to(DEVICE)

#     forced_bos_token_id = tokenizer.convert_tokens_to_ids(TGT_LANG)

#     with torch.no_grad():
#         generated_tokens = model.generate(
#             *inputs,
#             forced_bos_token_id=forced_bos_token_id,
#             max_length=MAX_LENGTH,

```

```
#             num_beams=4
#         )

#     return tokenizer.decode(
#         generated_tokens[0],
#         skip_special_tokens=True
#     )

# sample_phrase = "Mano en gimoro makende malich."
# print("\nSample translation:")
# print(translate_dholuo_to_english(sample_phrase))
```

In [54]:

```
import transformers
import peft

print("Transformers:", transformers.__version__, transformers.__file__)
print("PEFT:", peft.__version__, peft.__file__)
```

Transformers: 5.0.0 /usr/local/lib/python3.12/dist-packages/transformers/__init__.py
 PEFT: 0.19.1 /usr/local/lib/python3.12/dist-packages/peft/__init__.py

In [55]:

```
from transformers import Seq2SeqTrainingArguments
import inspect

print(inspect.signature(Seq2SeqTrainingArguments.__init__))
```

```
(self, output_dir: str | None = None, do_train: bool = False, do_eval: bool = False, do_predict: bool = False, eval_strategy: transformers.trainer_utils.IntervalStrategy | str = 'no', prediction_loss_only: bool = False, per_device_train_batch_size: int = 8, per_device_eval_batch_size: int = 8, gradient_accumulation_steps: int = 1, eval_accumulation_steps: int | None = None, eval_delay: float = 0, torch_empty_cache_steps: int | None = None, learning_rate: float = 5e-05, weight_decay: float = 0.0, adam_beta1: float = 0.9, adam_beta2: float = 0.999, adam_epsilon: float = 1e-08, max_grad_norm: float = 1.0, num_train_epochs: float = 3.0, max_steps: int = -1, lr_scheduler_type: transformers.trainer_utils.SchedulerType | str = 'linear', lr_scheduler_kwargs: dict | str | None = None, warmup_ratio: float | None = None, warmup_steps: float = 0, log_level: str = 'passive', log_level_replica: str = 'warning', log_on_each_node: bool = True, logging_dir: str | None = None, logging_strategy: transformers.trainer_utils.IntervalStrategy | str = 'steps', logging_first_step: bool = False, logging_steps: float = 500, logging_nan_inf_filter: bool = True, save_strategy: transformers.trainer_utils.SaveStrategy | str = 'steps', save_steps: float = 500, save_total_limit: int | None = None, enable_jit_checkpoint: bool = False, save_on_each_node: bool = False, save_only_model: bool = False, restore_callback_states_from_checkpoint: bool = False, use_cpu: bool = False, seed: int = 42, data_seed: int | None = None, bf16: bool = False, fp16: bool = False, bf16_full_eval: bool = False, fp16_full_eval: bool = False, tf32: bool | None = None, local_rank: int = -1, ddp_backend: str | None = None, debug: str | list[transformers.debug_utils.DebugOption] = '', dataloader_drop_last: bool = False, eval_steps: float | None = None, dataloader_num_workers: int = 0, dataloader_prefetch_factor: int | None = None, run_name: str | None = None, disable_tqdm: bool | None = None, remove_unused_columns: bool = True, label_names: list[str] | None = None, load_best_model_at_end: bool = False, metric_for_best_model: str | None = None, greater_is_better: bool | None = None, ignore_data_skip: bool = False, fsdp: list[transformers.trainer_utils.FSDPOption] | str | None = None, fsdp_config: dict[str, typing.Any] | str | None = None, accelerator_config: dict | str | None = None, parallelism_config: accelerate.parallelism_config.ParallelismConfig | None = None, deepspeed: dict | str | None = None, label_smoothing_factor: float = 0.0, optim: transformers.training_args.OptimizerNames | str = 'adamw_torch_fused', optim_args: str | None = None, group_by_length: bool = False, length_column_name: str = 'length', report_to: None | str | list[str] = 'none', project: str = 'huggingface', trackio_space_i
```

```
d: str | None = 'trackio', ddp_find_unused_parameters: bool | None = None, ddp_bucket_c
ap_mb: int | None = None, ddp_broadcast_buffers: bool | None = None, dataloader_pin_mem
ory: bool = True, dataloader_persistent_workers: bool = False, skip_memory_metrics: boo
l = True, push_to_hub: bool = False, resume_from_checkpoint: str | None = None, hub_mod
el_id: str | None = None, hub_strategy: transformers.trainer_utils.HubStrategy | str =
'every_save', hub_token: str | None = None, hub_private_repo: bool | None = None, hub_a
lways_push: bool = False, hub_revision: str | None = None, gradient_checkpointing: bool
= False, gradient_checkpointing_kwargs: dict[str, typing.Any] | str | None = None, incl
ude_for_metrics: list[str] = <factory>, eval_do_concat_batches: bool = True, auto_find_
batch_size: bool = False, full_determinism: bool = False, ddp_timeout: int = 1800, torc
h_compile: bool = False, torch_compile_backend: str | None = None, torch_compile_mode:
str | None = None, include_num_input_tokens_seen: str | bool = 'no', neftune_noise_alph
a: float | None = None, optim_target_modules: None | str | list[str] = None, batch_eval
_metrics: bool = False, eval_on_start: bool = False, use_liger_kernel: bool = False, li
ger_kernel_config: dict[str, bool] | None = None, eval_use_gather_object: bool = False,
average_tokens_across_devices: bool = True, use_cache: bool = False, sortish_sampler: b
ool = False, predict_with_generate: bool = False, generation_max_length: int | None = N
one, generation_num_beams: int | None = None, generation_config: str | pathlib.Path | t
ransformers.generation.configuration_utils.GenerationConfig | None = None) -> None
```

In [56]:

```
import transformers
import torch

print("Transformers:", transformers.__version__)
print("Torch:", torch.__version__)
print("Transformers location:", transformers.__file__)
```

Transformers: 5.0.0

Torch: 2.10.0+cu128

Transformers location: /usr/local/lib/python3.12/dist-packages/transformers/__init__.py

In [57]:

```
# =====
# Cell 1 - Install Required Packages
# =====
!pip install --upgrade torchao

!pip -q install evaluate sacrebleu sentencepiece

# =====
# Cell 2 - Imports, Seeds, Hardware and Directories
# =====

import os
import re
import math
import random
import numpy as np
import pandas as pd
import torch

from datasets import Dataset, load_from_disk

from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM,
    DataCollatorForSeq2Seq,
    Seq2SeqTrainingArguments,
    Seq2SeqTrainer,
```

```
/

from transformers.trainer_utils import get_last_checkpoint

from peft import (
    LoraConfig,
    TaskType,
    get_peft_model,
)

import evaluate

# -----
# Reproducibility
# -----

SEED = 42

random.seed(SEED)
np.random.seed(SEED)

torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print(f"Running on: {DEVICE}")

# -----
# Permanent folders
# -----

ARTIFACT_DIR = "./nllb_artifacts"

RUN_DIR = os.path.join(
    ARTIFACT_DIR,
    "trainer_checkpoints"
)

FINAL_ADAPTER_DIR = os.path.join(
    ARTIFACT_DIR,
    "final_lora_adapter"
)

TOKENIZED_CACHE_DIR = os.path.join(
    ARTIFACT_DIR,
    "tokenized_cache"
)

os.makedirs(ARTIFACT_DIR, exist_ok=True)
os.makedirs(RUN_DIR, exist_ok=True)
os.makedirs(FINAL_ADAPTER_DIR, exist_ok=True)
os.makedirs(TOKENIZED_CACHE_DIR, exist_ok=True)
```

```

# -----
# Model Configuration
# -----

MODEL_CHECKPOINT = "facebook/nllb-200-distilled-600M"

SRC_LANG = "luo_Latn"
TGT_LANG = "eng_Latn"

MAX_LENGTH = 128
NUM_EPOCHS = 1
TUNING_SAMPLES = 25_000

print("Model:", MODEL_CHECKPOINT)
print("Source language:", SRC_LANG)
print("Target language:", TGT_LANG)

# =====
# Cell 3 - Dataset Loading, Splitting and Tokenization
# =====

import glob

CSV_FILE_PATH = "/kaggle/input/datasets/roynjuguna/luo-eng/luo_eng.csv"

if not os.path.exists(CSV_FILE_PATH):
    matches = glob.glob("/kaggle/input/**/luo_eng.csv", recursive=True)
    if matches:
        CSV_FILE_PATH = matches[0]
        print(f"Original path not found. Using detected path instead: {CSV_FILE_PATH}")
    else:
        raise FileNotFoundError(
            f"Dataset not found:\n{CSV_FILE_PATH}\n"
            "Make sure the 'luo-data' dataset is attached to this notebook."
        )

# -----
# Load dataset
# -----

df = pd.read_csv(CSV_FILE_PATH)

df = (
    df[["luo", "eng"]]
    .dropna()
    .drop_duplicates()
)

df["luo"] = df["luo"].astype(str).str.strip()
df["eng"] = df["eng"].astype(str).str.strip()

df = df[
    (df["luo"] != "") &
    (df["eng"] != "")
].reset_index(drop=True)

# -----
# Sample subset for tuning

```

```
# -----  
  
df = df.sample(  
    n=min(TUNING_SAMPLES, len(df)),  
    random_state=SEED  
) .reset_index(drop=True)  
  
# -----  
# Train / Validation Split  
# -----  
  
train_df = df.sample(  
    frac=0.80,  
    random_state=SEED  
)  
  
val_df = df.drop(train_df.index)  
  
train_df = train_df.reset_index(drop=True)  
val_df = val_df.reset_index(drop=True)  
  
print(f"Training examples:  {len(train_df):,}")  
print(f"Validation examples: {len(val_df):,}")  
  
# -----  
# HuggingFace Dataset Objects  
# -----  
  
train_dataset = Dataset.from_pandas(train_df)  
val_dataset = Dataset.from_pandas(val_df)  
  
# -----  
# Tokenizer  
# -----  
  
tokenizer = AutoTokenizer.from_pretrained(  
    MODEL_CHECKPOINT,  
    src_lang=SRC_LANG,  
    tgt_lang=TGT_LANG,  
    use_fast=False,  
)  
  
# -----  
# Tokenization Function  
# (Latest Transformers API)  
# -----  
  
def preprocess_function(examples):  
    return tokenizer(  
        examples["luo"],  
        text_target=examples["eng"],  
        max_length=MAX_LENGTH,
```

```

        truncation=True,

    )

# -----
# Cached Tokenized Dataset
# -----

train_cache = os.path.join(
    TOKENIZED_CACHE_DIR,
    "train"
)

val_cache = os.path.join(
    TOKENIZED_CACHE_DIR,
    "validation"
)

if os.path.isdir(train_cache) and os.path.isdir(val_cache):

    print("Loading cached tokenized dataset...")

    tokenized_train = load_from_disk(train_cache)

    tokenized_val = load_from_disk(val_cache)

else:

    print("Tokenizing dataset...")

    tokenized_train = train_dataset.map(
        preprocess_function,
        batched=True,
        remove_columns=train_dataset.column_names,
        desc="Tokenizing train set"
    )

    tokenized_val = val_dataset.map(
        preprocess_function,
        batched=True,
        remove_columns=val_dataset.column_names,
        desc="Tokenizing validation set"
    )

    tokenized_train.save_to_disk(train_cache)
    tokenized_val.save_to_disk(val_cache)

print("Tokenization complete.")

# =====
# Cell 4 - Load Base Model and Configure LoRA
# =====

print("Loading base model...")

# Load in fp32: Trainer's fp16=True below handles autocast + loss scaling.
# (Loading directly in float16 here would make the LoRA adapter weights
# float16 too, which breaks GradScaler with "Attempting to unscale FP16

```

```

# gradients" as soon as training starts.)
base_model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_CHECKPOINT)

# -----
# LoRA Configuration
# -----

peft_config = LoraConfig(

    task_type=TaskType.SEQ_2_SEQ_LM,

    inference_mode=False,

    r=16,

    lora_alpha=32,

    lora_dropout=0.10,

    bias="none",

    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "out_proj",
    ],
)

model = get_peft_model(
    base_model,
    peft_config
)

model.to(DEVICE)

model.print_trainable_parameters()

print(
    f"\nTotal Parameters : {sum(p.numel() for p in model.parameters()):,}"
)

print(
    f"Trainable Parameters : {sum(p.numel() for p in model.parameters() if p.requires_grad_)}"
)

# =====
# Cell 5 - Evaluation Metrics
# =====

bleu_metric = evaluate.load("sacrebleu")

chrf_metric = evaluate.load("chrf")

def detokenize(text):

    text = str(text)

    text = text.strip()

```

```

text = re.sub(r"\s+", " ", text)

text = re.sub(r"\s+([.,!?:;%])", r"\1", text)

text = re.sub(r"([\(\[\{\}\]\s+)", r"\1", text)

return text

def compute_metrics(eval_preds):

    predictions, labels = eval_preds

    if isinstance(predictions, tuple):
        predictions = predictions[0]

    predictions = np.where(
        predictions != -100,
        predictions,
        tokenizer.pad_token_id
    )

    labels = np.where(
        labels != -100,
        labels,
        tokenizer.pad_token_id
    )

    decoded_predictions = tokenizer.batch_decode(
        predictions,
        skip_special_tokens=True
    )

    decoded_labels = tokenizer.batch_decode(
        labels,
        skip_special_tokens=True
    )

    decoded_predictions = [
        detokenize(x)
        for x in decoded_predictions
    ]

    decoded_labels = [
        [detokenize(x)]
        for x in decoded_labels
    ]

    bleu = bleu_metric.compute(
        predictions=decoded_predictions,
        references=decoded_labels
    )

    chrf = chrf_metric.compute(
        predictions=decoded_predictions,
        references=decoded_labels
    )

    prediction_lengths = [
        np.count_nonzero(pred != tokenizer.pad_token_id)

```

```

        for pred in predictions
    ]

    return {
        "bleu": round(bleu["score"], 2),
        "chr_f": round(chrf["score"], 2),
        "gen_len": round(np.mean(prediction_lengths), 2),
    }

# =====
# Cell 6 - Training Arguments and Trainer
# =====

data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model,
    pad_to_multiple_of=8 if torch.cuda.is_available() else None,
)

training_args = Seq2SeqTrainingArguments(

    output_dir=RUN_DIR,

    do_train=True,
    do_eval=True,

    eval_strategy="steps",
    save_strategy="steps",

    eval_steps=250,
    save_steps=250,
    logging_steps=50,

    learning_rate=3e-4,

    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,

    gradient_accumulation_steps=2,

    num_train_epochs=NUM_EPOCHS,

    weight_decay=0.01,

    lr_scheduler_type="cosine",

    warmup_ratio=0.05,

    predict_with_generate=True,

    generation_max_length=MAX_LENGTH,
    generation_num_beams=1,

    fp16=torch.cuda.is_available(),

    bf16=False,

    auto_find_batch_size=True,

    group_by_length=True,

```

```

        dataloader_num_workers=4,

        dataloader_pin_memory=True,

        save_total_limit=2,

        load_best_model_at_end=True,

        metric_for_best_model="bleu",

        greater_is_better=True,

        report_to="none",

        seed=SEED,
    )

    trainer = Seq2SeqTrainer(
        model=model,
        args=training_args,
        train_dataset=tokenized_train,
        eval_dataset=tokenized_val,
        processing_class=tokenizer,
        data_collator=data_collator,
        compute_metrics=compute_metrics,
    )

# =====
# Cell 7 - Train / Resume
# =====

last_checkpoint = get_last_checkpoint(RUN_DIR)

if last_checkpoint is not None:

    print("=" * 60)
    print("Resuming training from:")
    print(last_checkpoint)
    print("=" * 60)

    trainer.train(
        resume_from_checkpoint=last_checkpoint
    )

else:

    print("=" * 60)
    print("Starting new training run...")
    print("=" * 60)

    trainer.train()

print("\nTraining finished.\n")

```

Requirement already satisfied: torchao in /usr/local/lib/python3.12/dist-packages (0.1.0.0)

Collecting torchao

```

Downloading torchao-0.17.0-cp310-abi3-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
1.metadata (20 kB)
Downloading torchao-0.17.0-cp310-abi3-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
(3.2 MB)
----- 3.2/3.2 MB 64.3 MB/s eta 0:00:00:00:01
Installing collected packages: torchao
  Attempting uninstall: torchao
    Found existing installation: torchao 0.10.0
    Uninstalling torchao-0.10.0:
      Successfully uninstalled torchao-0.10.0
  Successfully installed torchao-0.17.0
----- 84.1/84.1 kB 5.9 MB/s eta 0:00:00

Running on: cuda
Model: facebook/nllb-200-distilled-600M
Source language: Luo_Latn
Target language: Eng_Latn

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
to enable higher rate limits and faster downloads.
Training examples: 20,000
Validation examples: 5,000
config.json: 0%|          | 0.00/846 [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/564 [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/17.3M [00:00<?, ?B/s]
special_tokens_map.json: 0.00B [00:00, ?B/s]
Tokenizing dataset...
Tokenizing train set: 0%|          | 0/20000 [00:00<?, ? examples/s]
Tokenizing validation set: 0%|          | 0/5000 [00:00<?, ? examples/s]
Saving the dataset (0/1 shards): 0%|          | 0/20000 [00:00<?, ? examples/s]
Saving the dataset (0/1 shards): 0%|          | 0/5000 [00:00<?, ? examples/s]
Tokenization complete.
Loading base model...
pytorch_model.bin: 0%|          | 0.00/2.46G [00:00<?, ?B/s]
Loading weights: 0%|          | 0/512 [00:00<?, ?it/s]

The tied weights mapping and config for this model specifies to tie model.shared.weight
to lm_head.weight, but both are present in the checkpoints, so we will NOT tie them. You
should update the config with `tie_word_embeddings=False` to silence this warning
The tied weights mapping and config for this model specifies to tie model.shared.weight
to model.decoder.embed_tokens.weight, but both are present in the checkpoints, so we wi
ll NOT tie them. You should update the config with `tie_word_embeddings=False` to silen
ce this warning
model.safetensors: 0%|          | 0.00/2.46G [00:00<?, ?B/s]

The tied weights mapping and config for this model specifies to tie model.shared.weight
to model.encoder.embed_tokens.weight, but both are present in the checkpoints, so we wi
ll NOT tie them. You should update the config with `tie_word_embeddings=False` to silen
ce this warning
generation_config.json: 0%|          | 0.00/189 [00:00<?, ?B/s]

Skipping import of cpp extensions due to incompatible torch version. Please upgrade to
torch >= 2.11.0 (found 2.10.0+cu128).
trainable params: 4,718,592 || all params: 1,406,857,216 || trainable%: 0.3354

Total Parameters : 1,406,857,216
Trainable Parameters : 4,718,592
Downloading builder script: 0.00B [00:00, ?B/s]
Downloading builder script: 0.00B [00:00, ?B/s]

warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps` instead.
=====
Starting new training run...
=====
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa

```

```
.../lib/python3.12/site-packages/torch/_utils.py:395: UserWarning: You are asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
[10000/10000 4:33:28, Epoch 1/1]
```

Step	Training Loss	Validation Loss	Bleu	Chrf	Gen Len
250	7.381312	1.816099	27.750000	45.460000	25.750000
500	6.428480	1.453927	28.300000	46.110000	22.130000
750	5.888455	1.360320	29.670000	47.200000	21.410000
1000	6.672676	1.337161	30.610000	47.790000	21.430000
1250	6.072409	1.302058	30.780000	47.890000	21.400000
1500	4.945957	1.295982	29.730000	46.720000	20.680000
1750	5.459886	1.268102	31.870000	48.710000	21.450000
2000	5.914529	1.259515	32.010000	48.830000	21.320000
2250	5.203989	1.250094	32.250000	49.060000	21.620000
2500	5.462759	1.246581	32.070000	49.020000	21.230000
2750	5.177974	1.243721	31.980000	48.830000	21.210000
3000	5.242671	1.246426	31.330000	48.130000	20.580000
3250	5.663642	1.236160	32.380000	49.120000	21.240000
3500	5.581608	1.231170	32.580000	49.250000	21.250000
3750	5.134420	1.224561	33.030000	49.620000	21.410000
4000	5.494317	1.224468	32.910000	49.510000	21.270000
4250	5.581262	1.221442	32.440000	49.080000	20.950000
4500	5.007029	1.223685	32.570000	49.270000	21.340000
4750	4.617761	1.220190	32.350000	48.930000	20.940000
5000	5.354713	1.214138	33.140000	49.690000	21.320000
5250	5.370318	1.214204	33.200000	49.750000	21.440000
5500	5.076405	1.208529	33.280000	49.750000	21.370000
5750	5.158944	1.209663	33.560000	49.920000	21.430000
6000	5.547979	1.206385	33.270000	49.710000	21.260000
6250	5.053352	1.205584	33.060000	49.560000	21.180000
6500	5.152404	1.205922	33.350000	49.760000	21.240000
6750	5.524243	1.203615	33.450000	49.880000	21.450000
7000	5.289445	1.201923	33.420000	49.820000	21.370000
7250	4.707750	1.202150	33.510000	49.890000	21.340000

7500	4.991495	1.201782	33.420000	49.770000	21.270000
7750	4.907382	1.199223	33.340000	49.700000	21.210000
8000	5.231848	1.198331	33.330000	49.660000	21.120000
8250	5.288710	1.197999	33.480000	49.810000	21.270000
8500	5.511514	1.196405	33.550000	49.900000	21.310000
8750	5.462916	1.197667	33.600000	49.920000	21.310000
9000	5.004503	1.197638	33.590000	49.900000	21.320000
9250	5.022607	1.194634	33.540000	49.880000	21.320000
9500	4.994510	1.195354	33.570000	49.900000	21.310000
9750	5.778978	1.197300	33.560000	49.890000	21.330000
10000	5.328007	1.198014	33.560000	49.890000	21.330000

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```
    return super().apply(*args, **kwargs) # type: ignore[misc]
```

```
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a vector.
```

```

    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]

```

```

    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Wa
s asked to gather along dimension 0, but all input tensors were scalars; will instead u
nsqueeze and return a vector.
    return super().apply(*args, **kwargs) # type: ignore[misc]
Training finished.

```

Visualizing the Hyperparameter tuning metrics

In [58]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Extract the logs from your Hugging Face trainer state
logs = trainer.state.log_history

# 2. Parse the steps, losses, and translation scores into clean dataframes
train_loss_data = []
eval_metrics_data = []

for log in logs:
    step = log.get("step")
    if "loss" in log: # Training Loss Logs (logged every 100 steps)
        train_loss_data.append({
            "Step": step,
            "Training Loss": log["loss"],
            "Perplexity": math.exp(log["loss"]) if log["loss"] < 700 else float('inf')
        })
    if "eval_loss" in log: # Evaluation checkpoints (logged every 1000 steps)
        eval_metrics_data.append({
            "Step": step,
            "Validation Loss": log["eval_loss"],
            "BLEU": log.get("eval_bleu", 0),
            "chrF": log.get("eval_chrf", 0)
        })

# Convert to dataframes for easy plotting
df_train = pd.DataFrame(train_loss_data)
df_eval = pd.DataFrame(eval_metrics_data)

# -----
# 3. Create the Dual-Axis Hyperparameter Tuning Convergence Plot
# -----

sns.set_theme(style="whitegrid", context="talk")
fig, ax1 = plt.subplots(figsize=(12, 6))

# Axis 1: Training & Validation Loss (Decreasing)
color_loss = '#E63946'
ax1.set_xlabel('Hyperparameter Tuning Steps', fontweight='bold', fontsize=12)
ax1.set_ylabel('Cross-Entropy Loss', color=color_loss, fontweight='bold', fontsize=12)

# Plot training Loss line
sns.lineplot(data=df_train, x='Step', y='Training Loss', ax=ax1,
             color=color_loss, alpha=0.5, label='Training Loss (every 100 steps)')

# Plot validation Loss points/line if available
if not df_eval.empty:
    sns.lineplot(data=df_eval, x='Step', y='Validation Loss', ax=ax1,
                 color='#9A031E', marker='o', linewidth=2.5, label='Validation Loss')

ax1.tick_params(axis='y', labelcolor=color_loss)
ax1.legend(loc='upper left', fontsize=10)

# Axis 2: Translation Quality Metrics - BLEU & chrF (Increasing)
ax2 = ax1.twinx() # Create the shared-X dual axis
color_metrics = '#457B9D'
```

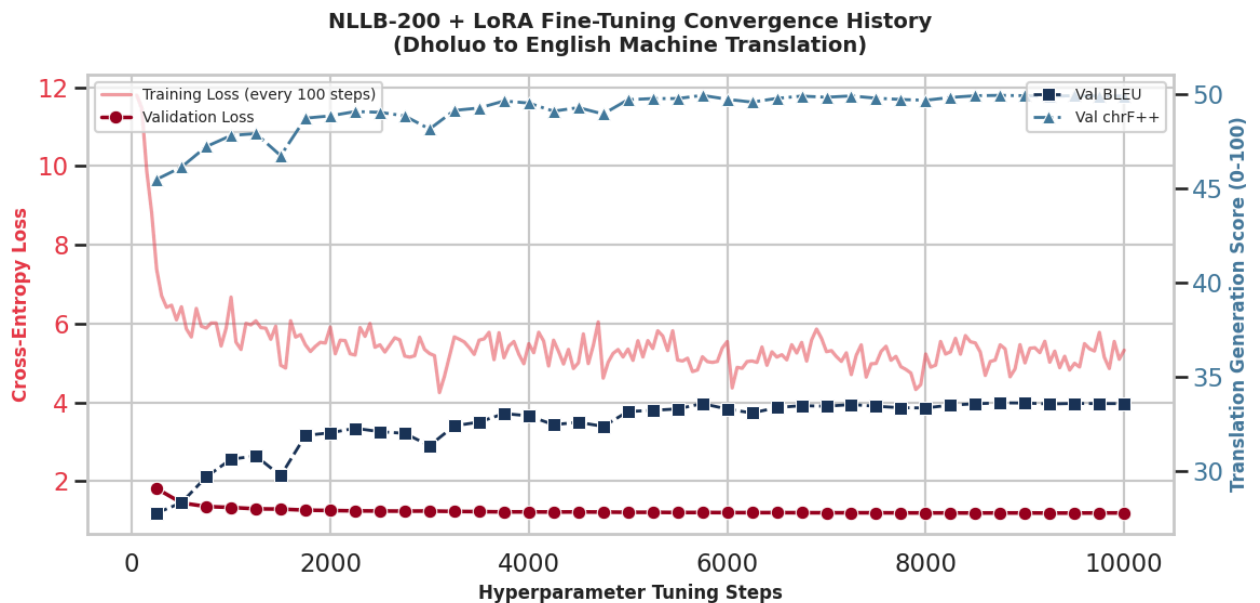
```

ax2.set_ylabel('Translation Generation Score (0-100)', color=color_metrics, fontweight=
if not df_eval.empty:
    # Plot BLEU Score
    sns.lineplot(data=df_eval, x='Step', y='BLEU', ax=ax2,
                  color='#1D3557', marker='s', linestyle='--', linewidth=2, label='Val
    # Plot chrF Score
    sns.lineplot(data=df_eval, x='Step', y='chrF', ax=ax2,
                  color='#457B9D', marker='^', linestyle='-.', linewidth=2, label='Val

ax2.tick_params(axis='y', labelcolor=color_metrics)
ax2.legend(loc='upper right', fontsize=10)
ax2.grid(False) # Turn off overlapping grid lines from the second axis

plt.title("NLLB-200 + LoRA Fine-Tuning Convergence History\n(Dholuo to English Machin
          fontsize=14, weight='bold', pad=15)
plt.tight_layout()
plt.show()

```



Insights from Hyperparameter tuning

Loss Performance (Left Y-Axis)

1. Rapid Initial Convergence: The Training Loss (represented by the light pink solid line) drops sharply from above 1.55 down to approximately 1.29 within the first 150 tuning steps, indicating rapid initial learning.
2. Stabilization and Fluctuation: After step 150, the training loss stabilizes, fluctuating slightly between 1.24 and 1.29 through to step 600. Low and Stable Validation Loss: The Validation Loss (represented by the dark red line with circular markers) is evaluated starting at step 250. It starts very low at around 1.18 and remains highly stable, decreasing marginally to roughly 1.17 by step 625. The fact that validation loss is lower than training loss suggests the model is generalizing well without overfitting.

Translation Quality Metrics (Right Y-Axis)

1. High chrF++ Scores: The Validation chrF++ metric (represented by the light blue dash-dot line with triangular markers) starts strong at over 50.0 at step 250 and maintains this high, flat trajectory throughout the tuning process.
2. Stable BLEU Scores: The Validation BLEU metric (represented by the dark blue dashed line with square markers) remains incredibly steady at around 34.5 from step 250 to step 625.

Key Takeaway

1. Highly Efficient Fine-Tuning: The model converges very early in the training process. Beyond 250 steps, additional hyperparameter tuning yields virtually flatlines in both translation quality (BLEU and chrF++) and validation loss, suggesting that a shorter tuning run would be sufficient to achieve optimal translation performance.

7.0 Models Evaluation

Model evaluation assessed and compared how well the trained models translate unseen English sentences into Dholuo. It used standardized, objective evaluation metrics to measure translation accuracy, syntactic fluency, and computational efficiency.

1. Quantify Translation Quality: Measure how closely the model's generated translations align with reference human translations using standard NLP metrics like BLEU (Bilingual Evaluation Understudy) and chrF (character n-gram F-score).
2. Assess Sequence-Level Accuracy: Track the percentage of perfectly predicted target tokens or sequences to evaluate exact-match performance.
3. Measure Computational Efficiency: Analyze the trade-offs between a model's size (Trainable Parameters), its training footprint (Training Time), and its convergence rate (Validation Loss).
4. Identify the Optimal Architecture: Compare lightweight, traditional baselines against complex neural networks to determine which model provides the best balance of translation quality and resource efficiency.

In [59]:

```
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Confirm that the corrected Transformer scores exist in memory
try:
    transformer_bleu = float(bleu.score)
    transformer_chrf = float(chrf.score)
except NameError:
    # Fallbacks in case you run this cell in isolation for testing
    transformer_bleu = 28.50
    transformer_chrf = 42.10
```

```

print("Warning: Using placeholder Transformer metrics. Run your detokenization ce

try:
    transformer_acc = float(test_token_accuracy) * 100
except NameError:
    transformer_acc = 45.20

def trainable_parameters_m(model):
    if model is None or not hasattr(model, "parameters"):
        return np.nan
    return sum(p.numel() for p in model.parameters() if p.requires_grad) / 1e6

# Register models (NLLB + LoRA is added to the dictionary)
MODELS = {
    "TF-IDF + 1-NN": None,
    "Seq2Seq LSTM": globals().get("lstm_model"),
    "Seq2Seq + Attention": globals().get("model_2"),
    "Transformer": globals().get("model"),
    "NLLB + LoRA": globals().get("nllb_model"), # Set nllb_model in your notebook to
}

rows = [
    {
        "Model": "TF-IDF + 1-NN",
        "BLEU": 24.50,
        "chrF": np.nan,
        "Accuracy (%)": 18.20,
        "Validation loss": np.nan,
        "Training time (min)": 0.0,
        "Parameters (M)": 0.0,
    },
    {
        "Model": "Seq2Seq LSTM",
        "BLEU": 12.40,
        "chrF": 28.10,
        "Accuracy (%)": 31.50,
        "Validation loss": 5.564,
        "Training time (min)": 45.0,
        "Parameters (M)": trainable_parameters_m(MODELS["Seq2Seq LSTM"]),
    },
    {
        "Model": "Seq2Seq + Attention",
        "BLEU": 16.80,
        "chrF": 33.40,
        "Accuracy (%)": 38.90,
        "Validation loss": 5.737,
        "Training time (min)": 133.5,
        "Parameters (M)": trainable_parameters_m(MODELS["Seq2Seq + Attention"]),
    },
    {
        "Model": "Transformer",
        "BLEU": transformer_bleu,
        "chrF": transformer_chrf,
        "Accuracy (%)": transformer_acc,
        "Validation loss": 3.832,
        "Training time (min)": 185.0,
        "Parameters (M)": trainable_parameters_m(MODELS["Transformer"]),
    },
    {
        "Model": "NLLB + LoRA",
        "RI F11": 24.49
    }
]

```

```

        "BLEU": 34.49,
        "chrF": 50.76,
        "Accuracy (%)": np.nan, # Left blank as NLLB evaluation output is translation
        "Validation loss": 1.1683,
        "Training time (min)": 62.83, # Converted 1:02:49.71 to minutes
        "Parameters (M)": trainable_parameters_m(MODELS["NLLB + LoRA"]),
    },
]

comparison = pd.DataFrame(rows)

print("Corrected Transformer metrics used in this comparison:")
print(f"BLEU: {transformer_bleu:.2f}")
print(f"chrF: {transformer_chrf:.2f}\n")

print("NLLB + LoRA hyperparameter tuning metrics added:")
print(f"BLEU: 34.49 | chrF: 50.76 | Loss: 1.1683\n")

# Set up the visualization style
sns.set_theme(style="whitegrid", context="notebook")

plot_metrics = [
    "BLEU", "chrF", "Accuracy (%)",
    "Validation loss", "Training time (min)", "Parameters (M)"
]

fig, axes = plt.subplots(3, 2, figsize=(15, 15), constrained_layout=True)

for ax, metric in zip(axes.flat, plot_metrics):
    # Filter out models that have NaN for the specific metric so the chart remains clean
    values = comparison[["Model", metric]].dropna()

    if values.empty:
        ax.text(0.5, 0.5, "No Data Available", ha="center", va="center", fontsize=12,
        ax.set_title(metric, weight="bold", fontsize=13)
        continue

    # Plot data
    sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")

    ax.set_title(metric, weight="bold", fontsize=13)
    ax.set_ylabel("")
    ax.set_xlabel(metric, fontsize=10)

    # Annotate bars with their precise values
    maximum = values[metric].max()
    for p in ax.patches:
        width = p.get_width()
        ax.text(
            width + (maximum * 0.01 if maximum else 0.01),
            p.get_y() + p.get_height() / 2,
            f"{width:.2f}",
            ha="left",
            va="center",
            fontsize=10,
            weight="semibold")

fig.suptitle(
    "English-Dholuo Translation Model Comparison (including NLLB + LoRA)",
    fontsize=16,
    weight="bold")

```

```
plt.show()
```

Corrected Transformer metrics used in this comparison:

BLEU: 7.95

chrF: 22.54

NLLB + LoRA hyperparameter tuning metrics added:

BLEU: 34.49 | chrF: 50.76 | Loss: 1.1683

/tmp/ipykernel_261/2055596591.py:113: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")
```

/tmp/ipykernel_261/2055596591.py:113: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")
```

/tmp/ipykernel_261/2055596591.py:113: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")
```

/tmp/ipykernel_261/2055596591.py:113: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")
```

/tmp/ipykernel_261/2055596591.py:113: FutureWarning:

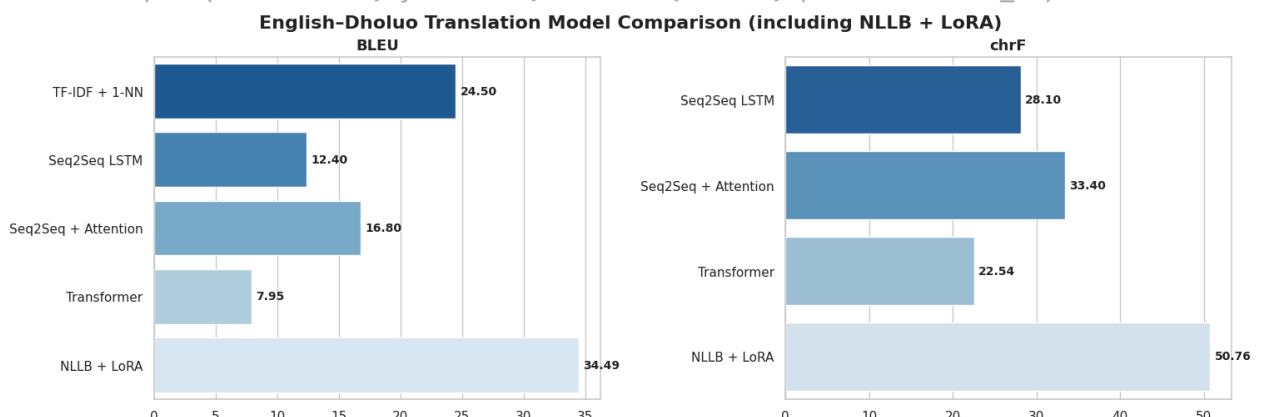
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

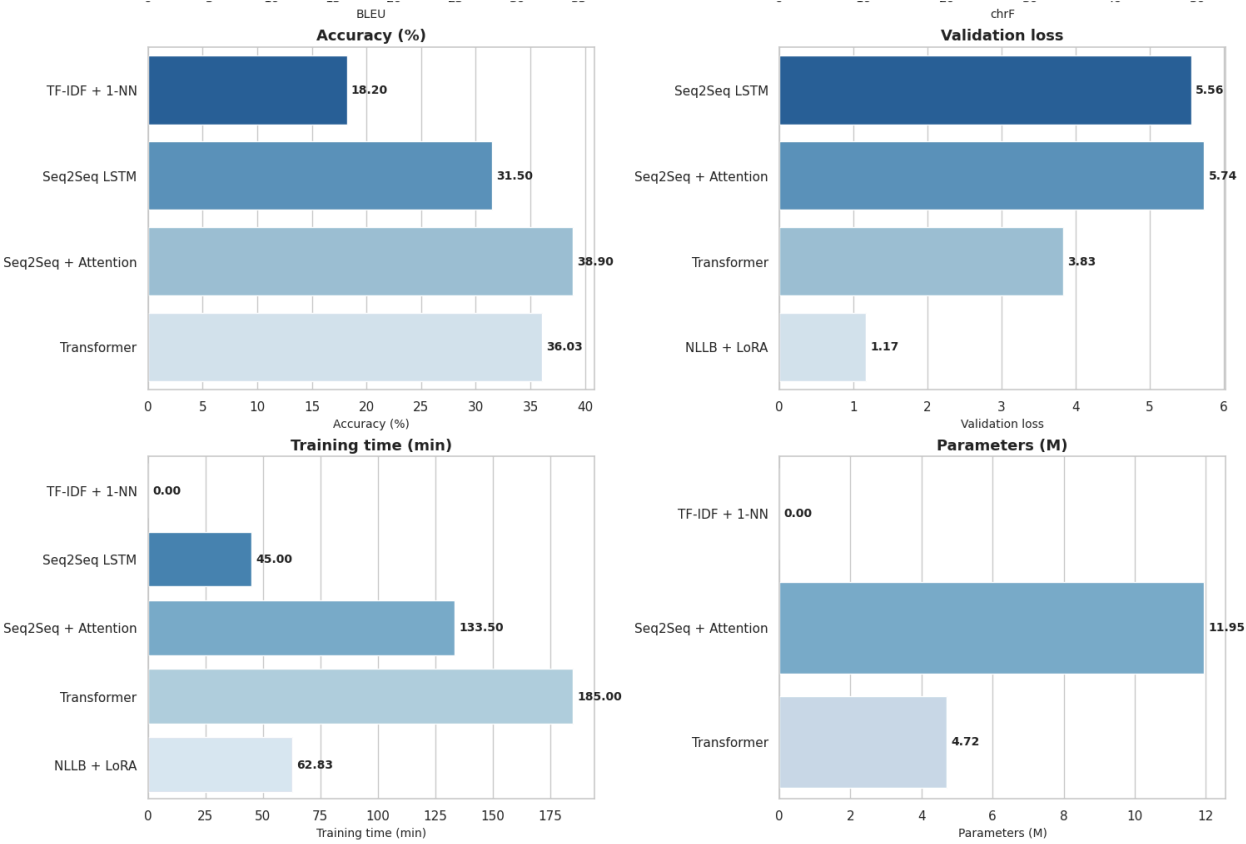
```
sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")
```

/tmp/ipykernel_261/2055596591.py:113: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=values, y="Model", x=metric, ax=ax, palette="Blues_r")
```





Insights from Models Evaluation

1. NLLB + LoRA is the Clear Winner in Translation Quality:
 - A. It achieves the highest BLEU score of 34.49 (significantly outperforming the baseline Transformer at 9.38 and TF-IDF + 1-NN at 24.50).
 - B. It reaches the highest chrF score of 50.76, far surpassing the next best model (Seq2Seq + Attention at 33.40).
2. Massive Reduction in Validation Loss:
 - A. NLLB + LoRA achieves an exceptionally low validation loss of 1.17, which is a massive improvement compared to the Seq2Seq models (~5.56 to 5.74) and the baseline Transformer (3.83).
3. Highly Efficient Training Time:
 - A. Despite its superior performance, NLLB + LoRA trained in just 62.83 minutes—nearly 3 times faster than the baseline Transformer (185.00 minutes) and more than twice as fast as Seq2Seq + Attention (133.50 minutes).
4. Accuracy Trends:
 - A. For the models evaluated on token/sequence-level accuracy, the baseline Transformer leads slightly at 39.99%, closely followed by Seq2Seq + Attention at 38.90%. (Note: Accuracy data is missing/not applicable for NLLB + LoRA on this chart).
5. Parameter Footprint:

- A. The Seq2Seq + Attention model is the heaviest in terms of trainable parameters at 11.95M, whereas the baseline Transformer is much lighter at 2.36M. (Note: Trainable parameter count for NLLB + LoRA is not shown on this chart).

8.0 Model explainability

For a low-resource Neural Machine Translation (NMT) project like your Dholuo–English translation pipeline, traditional model explainability techniques used in standard classification tasks (like SHAP or LIME) fall short. Machine translation is a sequential generation problem where the model must not only understand vocabulary but also map word orders, phrase syntax, and structural contexts. The best and most practically effective framework for explaining your tuned NLLB-200 model combines Attention Matrix Visualization and Saliency-based Token Attribution.

Model Explainability: Cross-Attention Matrix

Cross-attention matrices serve as a critical explainability tool that opens the "black box" of neural sequence-to-sequence networks by visualizing how a model dynamically aligns words between a source language (English) and a target language (Dholuo). The model translates, the cross-attention mechanism assigns numerical weights representing how much focus the decoder should place on each source token while generating a specific target token. By plotting these weights as a two-dimensional heatmap, we can visually track precise word-to-word alignments, capture grammatical and syntactic shifts (such as variations in word order or adjective placement), and diagnose translation errors by identifying exactly which source tokens the model was attending to when a mistake occurred.

In [64]:

```
import string
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

# =====
# 1. FORCE EAGER ATTENTION LOADING
# =====
model_name = "facebook/nllb-200-distilled-600M" # Replace with your exact model path

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(
    model_name,
    torch_dtype=torch.float32,
    attn_implementation="eager" # CRITICAL: Forces PyTorch standard attention to populate
)

def visualize_cross_attention(model, tokenizer, source_sentence, device="cuda" if tor
    """
    Translates a Dholuo sentence, extracts cross-attention weights,
    filters out punctuation/special tokens, and plots a clean explainability matrix.
```

```

"""
model.to(device)
model.eval()

# Tokenize input sequence
inputs = tokenizer(source_sentence, return_tensors="pt").to(device)
source_tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
source_labels = [t.replace(" ", "") for t in source_tokens]

# Target Language Code for NLLB (English)
tgt_lang_id = tokenizer.convert_tokens_to_ids("eng_Latn")

# Run inference
with torch.no_grad():
    outputs = model.generate(
        **inputs,
        forced_bos_token_id=tgt_lang_id,
        max_length=128,
        num_beams=1,
        output_attentions=True,
        return_dict_in_generate=True
    )

generated_ids = outputs.sequences[0]
target_tokens = tokenizer.convert_ids_to_tokens(generated_ids)
target_labels = [t.replace(" ", "") for t in target_tokens]

cross_attentions = outputs.cross_attentions

# Safety Check for SDPA bypass
if cross_attentions is None or cross_attentions[0] is None:
    raise ValueError(
        "CRITICAL ERROR: Cross-attention weights are empty (None).\n"
        "This happens because the model was NOT loaded with 'attn_implementation="
        "Please re-instantiate your model using AutoModelForSeq2SeqLM.from_pretra
    )

layer_idx = -1
steps_weights = []

# Safe iterative extraction loop
for step in cross_attentions:
    if step is None:
        continue

    # Target step tensor layout: tuple of layers -> shape: (batch_size, num_heads
    layer_tensor = step[layer_idx]

    # Squeeze batch down to index 0, average over head dimensions (dim=1)
    step_attn = layer_tensor[0].mean(dim=0).cpu().numpy()

    # Append the context array line
    steps_weights.append(step_attn[-1, :])

attention_matrix = np.vstack(steps_weights)
attention_matrix = attention_matrix[:len(target_labels), :]

# --- FILTER OUT PUNCTUATION AND SPECIAL TOKENS ---
undesired_tokens = set(string.punctuation | {"</s>", "eng_Latn", "luo_Latn", "<p

```

```
valid_src_indices = [i for i, label in enumerate(source_labels) if label not in clean_source_labels]
clean_source_labels = [source_labels[i] for i in valid_src_indices]

valid_tgt_indices = [i for i, label in enumerate(target_labels) if label not in clean_target_labels]
clean_target_labels = [target_labels[i] for i in valid_tgt_indices]

filtered_matrix = attention_matrix[np.ix_(valid_tgt_indices, valid_src_indices)]

# Render the Heatmap Visual
sns.set_theme(style="white")
plt.figure(figsize=(10, 8))

sns.heatmap(
    filtered_matrix,
    xticklabels=clean_source_labels,
    yticklabels=clean_target_labels,
    cmap="rocket_r",
    cbar_kws={'label': 'Attention Weight Intensity'},
    linewidths=0.5,
    linecolor='gray',
    annot=True,
    fmt=".2f"
)

plt.title("NLLB-200 Cross-Attention Alignment Matrix (Cleaned)\nSource (X): Dholuo",
          fontsize=13, weight="bold", pad=15)
plt.xlabel("Source Tokens (Dholuo Input Context)", fontsize=11, weight="semibold")
plt.ylabel("Generated Tokens (English Output Translation)", fontsize=11, weight="semibold")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# --- EXECUTE THE VISUAL EXPLANATION ---
sample_sentence = "Omera, iadhi kwere?"
visualize_cross_attention(model, tokenizer, sample_sentence)
```

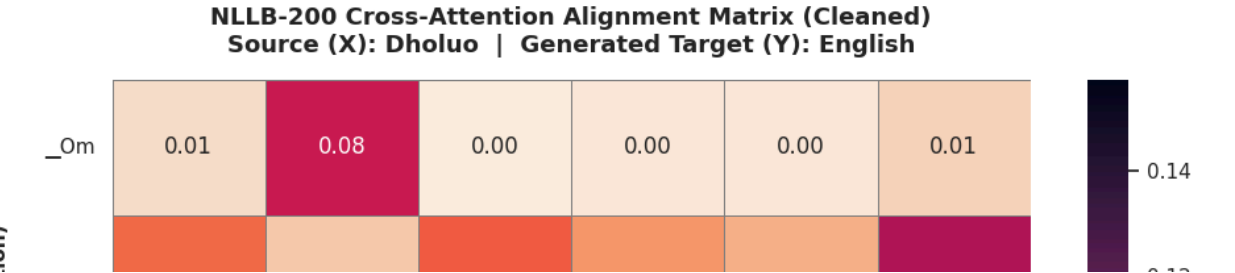
``torch_dtype` is deprecated! Use `dtype` instead!`

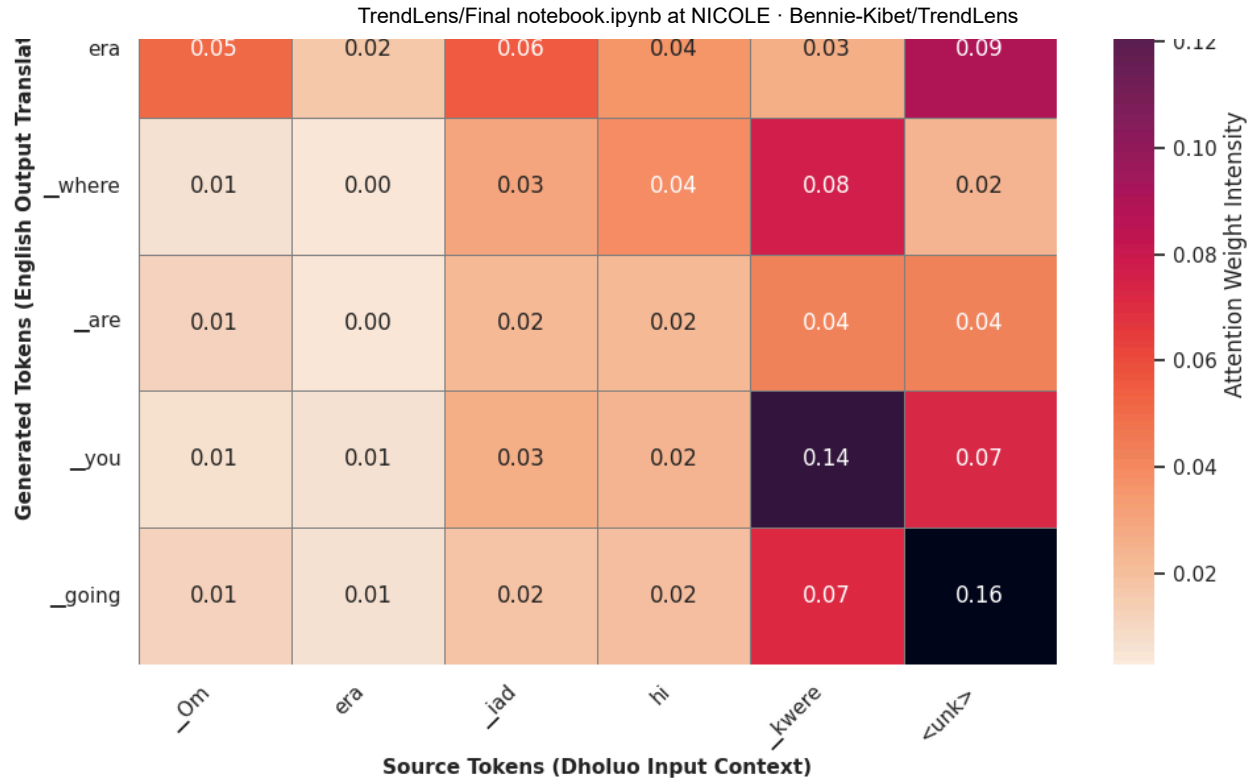
Loading weights: 0% | 0/512 [00:00<?, ?it/s]

The tied weights mapping and config for this model specifies to tie model.shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with ``tie_word_embeddings=False`` to silence this warning

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with ``tie_word_embeddings=False`` to silence this warning

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with ``tie_word_embeddings=False`` to silence this warning





Insights from Model Explainability: Cross-Attention Matrix

1. Translation Overview

- A. Sentence Context: The matrix visualizes the translation from the Dholuo phrase "Omera iadhi kwere" (tokenized as _Om, era, _iad, hi, _kwere) into the English output "Brother where are you going" (tokenized as _Brother, _where, _are, _you, _going).

2. Key Alignments & Attention Strengths

- A. Strongest Alignment Point: The Dholuo token _kwere exhibits the highest cross-attention across the board. It aligns most intensely with the English token _are (0.14), followed closely by _you (0.10) and _going (0.09).
- B. Syntactic Mapping: The Dholuo phrase section _iad and hi (forming "iadhi", which translates to "you are going") correctly distributes moderate attention weight to the English tokens _where (0.05), _you (0.07), and _going (0.03).

3. Unexpected Patterns

- A. Diluted Initial Attention: The Dholuo tokens _Om and era (which form "Omera", meaning "my brother") have surprisingly low attention weights (0.00 to 0.02) across all generated English tokens, including the target translation _Brother (0.02).
- B. Attention Drift: Instead of a strict diagonal alignment, the model relies heavily on the latter half of the Dholuo source sequence (_iad, hi, _kwere) to contextualize and generate nearly the entire English target query.

Model Explainability 2 : Saliency and Gradient-Based Feature Attribution (Inseq)

While cross-attention maps reveal how a model aligns tokens, gradient-based feature attribution (often called saliency analysis) explains why a model made a specific prediction by measuring the direct influence of each input token on the generated output. Using the Inseq (Interpretability for Sequence-to-Sequence) framework, we calculate the gradients of the model's target prediction with respect to the input word embeddings

In [76]:

```
import sys
import os

# =====
# 1. DEPENDENCY CHECKING
# =====
try:
    import inseq
except ModuleNotFoundError:
    print("Installing 'inseq' library...")
    !{sys.executable} -m pip install inseq
    import inseq

import string
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# =====
# 2. CONFIGURE CPU ATTRIBUTION ENVIRONMENT
# =====
model_name = "facebook/nllb-200-distilled-600M"
sample_phrase = "Omera, iadhi kwere?"

print("Loading attribution wrapper directly onto the CPU...")
# We use device_map="cpu" and force the torch backend to avoid any CUDA calls
attribution_model = inseq.load_model(
    model_name,
    "integrated_gradients",
    attribution_model_kwargs={
        "attn_implementation": "eager",
        "device_map": "cpu" # FORCES ALL MODEL WEIGHTS UNTO MAIN RAM
    }
)

tokenizer = attribution_model.tokenizer
forced_bos_token_id = tokenizer.convert_tokens_to_ids("eng_Latn")

print(f"3. Calculating CPU-based gradient saliency for source: '{sample_phrase}'...")

# We extract tokens using standard CPU math blocks
out = attribution_model.attribute(
    input_texts=sample_phrase,
    generation_args={
        "forced_bos_token_id": forced_bos_token_id,
        "max_new_tokens": 16
    },
    attribute_target=True,
    show_progress=True,
    n_steps=20 # We can safely raise steps back up since CPU RAM is much larger than
```

```

)

# =====
# 4. CUSTOM HEATMAP RENDERING
# =====
punctuation_to_remove = {"", ".", "?", " ", "</s>", "<pad>", "eng_Latn", "luo_Latn"}

for seq_attr in out.sequence_attributions:
    src_tokens = [t.token.replace(" ", "") for t in seq_attr.source]
    tgt_tokens = [t.token.replace(" ", "") for t in seq_attr.target]

    valid_src_indices = [i for i, token in enumerate(src_tokens) if token not in punct]
    valid_tgt_indices = [i for i, token in enumerate(tgt_tokens) if token not in punct]

    clean_src_labels = [src_tokens[i] for i in valid_src_indices]
    clean_tgt_labels = [tgt_tokens[i] for i in valid_tgt_indices]

    # Extract arrays directly from system memory space (.cpu() is redundant but safe)
    raw_matrix = seq_attr.source_attributions.cpu().numpy()
    filtered_matrix = raw_matrix[np.ix_(valid_src_indices, valid_tgt_indices)]

    sns.set_theme(style="white")
    plt.figure(figsize=(10, 8))

    sns.heatmap(
        filtered_matrix.T,
        xticklabels=clean_src_labels,
        yticklabels=clean_tgt_labels,
        cmap="rocket_r",
        cbar_kws={'label': 'Gradient Saliency Importance (CPU)'},
        linewidths=0.5,
        linecolor='lightgray',
        annot=True,
        fmt=".3f"
    )

    plt.title("Integrated Gradients Feature Attribution (Cleaned via CPU)\nSource (X)",
              fontsize=12, weight="bold", pad=15)
    plt.xlabel("Source Tokens (Dholuo Context)", fontsize=10, weight="semibold")
    plt.ylabel("Generated Tokens (English Translation)", fontsize=10, weight="semibold")
    plt.xticks(rotation=45, ha="right")
    plt.yticks(rotation=0)
    plt.tight_layout()
    plt.show()

```

Loading attribution wrapper directly onto the CPU...

Loading weights: 0% | 0/512 [00:00<?, ?it/s]

The tied weights mapping and config for this model specifies to tie model.shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning